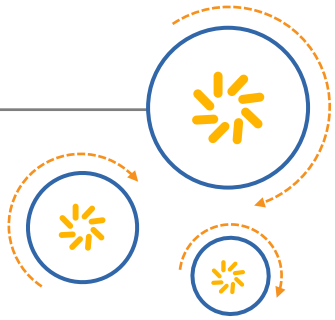
Qualcomm Technologies, Inc.



Diagnostic System

User Guide

80-NA157-61 J

September 15, 2015

QUALCOMM®
2016-05-17 06:26:10 PDT
deon_zhang@askey.com.tw

Confidential and Proprietary – Qualcomm Technologies, Inc.

© 2012--2015 Qualcomm Technologies, Inc. and/or its affiliated companies. All rights reserved.

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to: DocCtrlAgent@qualcomm.com.

Not to be used, copied, reproduced, or modified in whole or in part, nor its contents revealed in any manner to others without the express written permission of Qualcomm Technologies, Inc.

MSM is a product of Qualcomm Technologies, Inc. Other Qualcomm products referenced herein are products of Qualcomm Technologies, Inc. or its subsidiaries.

QUALCOMM®
2016-05-17 06:26:10 PDT
deon_zhang@askey.com.tw

Restricted Distribution: Not to be distributed to anyone who is not an employee of either Qualcomm Technologies, Inc. or its affiliated companies without the express approval of Qualcomm Configuration Management.

Qualcomm, MSM, and QXDM Professional are trademarks of Qualcomm Incorporated, registered in the United States and other countries. All Qualcomm Incorporated trademarks are used with permission. Other product and brand names may be trademarks or registered trademarks of their respective owners.

This technical data may be subject to U.S. and international export, re-export, or transfer (“export”) laws. Diversion contrary to U.S. and international law is strictly prohibited.

Qualcomm Technologies, Inc.
5775 Morehouse Drive
San Diego, CA 92121
U.S.A.

Revision history

Revision	Date	Description
A	Aug 2012	Initial release.
B	Oct 2013	Numerous changes were made to this document; it should be read in its entirety.
C	Nov 2013	Updated Chapter 7.
D	Feb 2014	Numerous changes were made to this document; it should be read in its entirety.
E	Apr 2014	Added Sections 4.5, 4.6, 6.2, and 6.3, Chapters 9 and 10, and Appendixes A and B; updated Sections 2.3.2, 2.3.3, 2.3.4 and Chapter 7.
F	Sep 2014	Added Section 2.3.3.1, 8.2.1.1, and 8.2.1.2 and updated Chapters 3 and 7 and Sections 4.1.1, 6.2.2, 8.1, and 8.3.1.
G	Nov 2014	Added a note after Figure 2-1. Added Sections 2.4 and 2.5. Updated Sections 4.1.1 and 6.1.1. Added Section 8.3.9. Updated Section 10.1.1.
H	Jun 2015	Moved and updated Event reporting service section (now Section 2.3.2). Added Chapter 5. Updated Sections 3.3.1, 4.4.1, 9.1.5.1, and 9.2.1. Added Sections 2.6, 4.4.5, 4.4.6, 4.5.3, 4.5.4, 4.5.5, 8.6, 9.1.5.2, 9.2.5, 9.3.4, 9.3.12, 10.2.2, and 11.1.2. Updated Figure 9-3.
J	Sep 2015	Updated Sections 2.3.3.3 and 4.10.1. Added Sections 4.3.3, 4.7, and 5.2.

Note: There is no Rev. I, O, Q, S, X, or Z per Mil. standards.

QUALCOMM
Document Control: 10 Print

Contents

1 Introduction 10

1.1 Purpose 10
1.2 Conventions 10
1.3 Technical assistance 10

2 Diagnostic services overview 11

2.1 Central routing 11
2.1.1 Handling common requests 14
2.2 Working with Diag services 14
2.3 Diag services APIs 15
2.3.1 Command service 15
2.3.2 Event reporting service 16
2.3.3 Debug Message service 18
2.3.4 Logging service 23
2.4 ADSP Diag architecture 24
2.4.1 LSM layer initialization 24
2.4.2 Logs/F3s/Events 24
2.4.3 Command requests/responses 24
2.5 Diag architecture on MSM8916 and MDM9x45 25
2.6 HDLC removal 26

3 Common customization examples 31

3.1 Defining new packets 31
3.1.1 Defining a new Subsystem Command (Linux, AMSS) 31
3.1.2 Defining a new Log code (Linux, AMSS) 36
3.1.3 Defining a new Event ID (Linux, AMSS) 37
3.1.4 Defining new message SSID 39
3.2 Changing a QXDM Pro database to support newly added packets 43
3.3 Adding listeners (AMSS only) 46
3.3.1 Adding log listeners 46
3.3.2 Adding event listeners 48
3.3.3 Adding F3 listeners 50

4 On-device logging 52

4.1 Using diag_mdlog 52
4.2 Command line options avaialable 53
4.3 Running on-device logging (diag_mdlog) 55
4.3.1 Running diag_mdlog using ADB 55
4.3.2 Running diag_mdlog on the device 55
4.3.3 Starting mdlog from another application 55

4.4 Troubleshooting SD card issues ... 56
4.5 Diag_mdlog output file ... 56
4.5.1 Converting .qmdl/.qmdl2 files to .isf for use in QXDM Pro ... 56
4.6 Diag masks ... 57
4.6.1 Creating a mask config (.cfg/.cfg2) file ... 57
4.6.2 Loading a mask config (.cfg/.cfg2) file in QXDM Pro ... 59
4.6.3 Saving Diag stress test command requests ... 62
4.6.4 Saving Diag task profile requests ... 64
4.6.5 Reading the CFG file ... 66
4.7 Qshrink4 support ... 69
4.8 OTA logs support in the CFG file ... 70
4.9 Low power modes ... 71
4.9.1 NRT mode ... 72
4.9.2 Real-time mode ... 72
4.10 Initializing diag_mdlog at boot time ... 73
4.10.1 Required code changes for initializing diag_mdlog at boot time ... 73

5 Buffering mode ... 75

5.1 Buffering mode functions ... 75
5.1.1 diag_configure_peripheral_buffering_tx_mode ... 75
5.1.2 diag_peripheral_buffering_drain_immediately ... 76
5.2 Sample buffering application ... 77

6 Socket logging ... 79

6.1 Initial setup ... 80
6.1.1 Ping device from PC ... 80
6.1.2 Ping PC from device ... 80
6.2 Application usage ... 81
6.2.1 Running the diag_socket_log application ... 81
6.2.2 Stopping the diag_socket_log application ... 81
6.3 QPST configuration ... 82

7 UART logging ... 83

7.1 Application usage ... 83
7.1.1 Running the diag_uart_log application ... 83
7.1.2 Fused target enhancements ... 84
7.1.3 Examples ... 84
7.2 Required code changes for enabling UART logging ... 85
7.2.1 TN builds – MDM9x15, MDM9x25, MDM9x35 ... 85
7.2.2 LE/LA builds ... 86
7.3 Tools tested ... 88
7.4 Limitations ... 89

8 Diag consumer interface ... 91

8.1 Overview ... 91
8.2 Initialization ... 92
8.3 Command/response packets ... 93

8.4 Logs and events 94
8.5 Limitations 95
8.6 DCI functions 95
8.6.1 diag_register_dci_client() 97
8.6.2 diag_release_dci_client() 99
8.6.3 diag_get_dci_support_list() 99
8.6.4 diag_get_dci_support_list_proc() 100
8.6.5 diag_send_dci_async_request() 101
8.6.6 diag_register_dci_stream() 102
8.6.7 diag_register_dci_stream_proc() 102
8.6.8 diag_log_stream_config() 103
8.6.9 diag_event_stream_config() 104
8.6.10 diag_get_health_stats() 104
8.6.11 diag_get_health_stats_proc() 105
8.6.12 diag_get_log_status() 106
8.6.13 diag_get_event_status() 106
8.6.14 diag_disable_all_logs() 107
8.6.15 diag_disable_all_events() 107
8.6.16 diag_dci_vote_real_time() 108
8.6.17 diag_dci_get_real_time_status() 108
8.6.18 diag_register_dci_signal_data() 109
8.6.19 diag_deregister_dci_signal_data() 110
8.7 Security 110
8.7.1 APSS protections 110
8.7.2 MPSS protections 111
8.8 NRT mode 111
8.8.1 NRT/Buffering mode 112
8.8.2 Power management on the APSS 113

9 Debugging 114

9.1 QPST disconnect/No Phone status 114
9.1.1 Check for host driver issues 115
9.1.2 Check for tools issues 116
9.1.3 Check QPST port trace 117
9.1.4 Check QXDM connection 124
9.1.5 Check device status 126
9.2 Linux APSS 135
9.2.1 Command/Response routing 136
9.2.2 F3 masks 139
9.2.3 Log masks 140
9.2.4 Kernel logs 141
9.2.5 Logcat 141
9.3 Modem processor 141
9.3.1 Command/Response routing 141
9.3.2 Log packet generation 143
9.3.3 Event report generation 145
9.3.4 Diag I/O communication channels 146
9.3.5 F3 messages generation 148
9.3.6 Diag signals 150

9.3.7 DSM debugging 151
9.3.8 Sleep voting 153
9.3.9 Timestamps 154
9.3.10 Maximum Packet sizes 154
9.3.11 Error fatal in diagbuf_check_overrun_pattern 155
9.3.12 Debugging listeners 156

10 Code structure 158

10.1 L4/Rex 158
10.1.1 Source files 158
10.1.2 Callflow 159
10.2 Linux 161
10.2.1 LA source files 161
10.2.2 Call flow 161
10.2.3 LE source files 167

11 Case study 168

11.1 Packet drop 168
11.1.1 Creating an IPC buffer on APSS 168
11.1.2 Extracting IPC logs 168
11.1.3 Using MPM timecount 168

A Defining custom Android permissions for Diag 174

A.1 Enabling use of custom permissions by a system APK 174
A.1.1 Declaring custom Android permission 174
A.1.2 Mapping custom Android permission to Linux permission 174
A.1.3 Additional modifications to the APK's Android.mk file 175
A.1.4 Proper placement of the APK's mapping.xml file 175
A.1.5 Making the APK part of the system image 175
A.2 Enabling use of custom permissions by a nonsystem APK 176
A.2.1 Using custom Android permissions 176
A.2.2 Signing an unsigned APK 176

B References 177

B.1 Related documents 177
B.2 Acronyms and terms 177

Figures

Figure 2-1 Central routing flow 13
Figure 2-2 Retrieve database IDs 22
Figure 2-3 Read a database using Diag commands 22
Figure 2-4 ADSP Diag architecture 25
Figure 2-5 Diag architecture on MSM8916 and MDM9x45 25
Figure 2-6 Old vs. new packet formats 27
Figure 2-7 Negotiation protocol used by device and tool for HDLC off 28
Figure 2-8 QPST HDLC mode switching options 30
Figure 4-1 Block diagram of the mdlog application operation 52
Figure 4-2 Flow diagram for using the diag_mdlog command line options 54
Figure 4-3 Difference between the QMDL and QMDL2 formats 57
Figure 4-4 Files needed for successful Qshrink decoding. 69
Figure 4-5 Item List Config window in QXDM Pro 71
Figure 6-1 Architecture for socket logging 79
Figure 8-1 Diag consumer services overview 91
Figure 8-2 Call flow for initialization 92
Figure 8-3 Call flow for command request/response service 93
Figure 8-4 Call flow for logs/events services 94
Figure 9-1 Log packet generation call flow 143
Figure 9-2 Event report generation call flow 145
Figure 9-3 F3 message generation call flow 148
Figure 9-4 Diag DSM pools and watermark queues 151
Figure 9-5 Diag sleep voting call flow 153

Tables

Table 2-1 ESN request 16
Table 2-2 ESN response 16
Table 2-3 Event packet format 16
Table 2-4 Message packet format 18
Table 2-5 MSG 2.0 macros 19
Table 2-6 Bit values for DIAG_F3_TRACE_DETAIL_MASK_DEFAULT_VAL 20
Table 2-7 Log packet format 23
Table 2-8 Diag services affected by HDLC removal 26
Table 2-9 Tools that support non-HDLC encoding/decoding 26
Table 2-10 Switch mode command 29
Table 2-11 Response to the switch mode command 29
Table 3-1 Single response APSS command table 33
Table 3-2 Single response modem command table 33

QUALCOMM
2015-05-17 06:26:10 PDT
deon_zhang@asky.com.tw

Table 3-3 Delayed response APSS command table 36
Table 3-4 Delayed response modem command table 36
Table 4-1 Options available with diag_mdlog application 53
Table 4-2 New header for a QMDL2 file 56
Table 6-1 diag_socket_log command arguments 81
Table 7-1 diag_uart_log command arguments 83
Table 8-1 Log packet structure 95
Table 8-2 Initialization, command/response, and data streaming functions 95
Table 8-3 DCI error codes 96
Table 9-1 Modem loopback command structure in APSS Linux dispatch table 137
Table 9-2 Mode reset command structure in APSS Linux dispatch table 137

QUALCOMM®
2016-05-17 06:26:10 PDT
deon_zhang@askey.com.tw

1 Introduction

1.1 Purpose

This document describes the Diagnostic (Diag) module system-level architecture, its functionalities, features, and debugging, and is meant for engineers using and working with Diag module services.

NOTE: This document uses examples that are specific to Linux Android™ HLOS, but the AMSS-based concepts apply to devices using all other HLOSs.

1.2 Conventions

Function declarations, function names, type declarations, attributes, and code samples appear in a different font, for example, `#include`.

Code variables appear in angle brackets, for example, `<number>`.

Commands to be entered appear in a different font, e.g., **`copy a:*.* b:`**.

Button and key names appear in bold font, for example, click **Save** or press **Enter**.

f you are viewing this document using a color monitor, or if you print this document to a color printer, **red boldface** indicates code that is to be **added**, and ~~blue strikethrough~~ indicates code that is to be replaced or removed (if present).

Shading indicates content that has been added or changed in this revision of the document.

1.3 Technical assistance

For assistance or clarification on information in this document, submit a case to Qualcomm Technologies, Inc. (QTI) at https://createpoint.qti.qualcomm.com/.

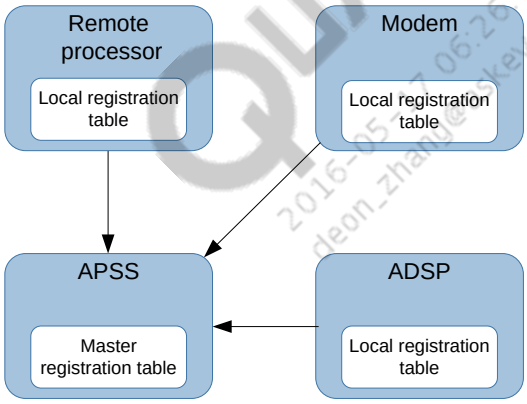
If you do not have access to the CDMATech Support website, register for access or send email to support.cdmatech@qti.qualcomm.com.

2 Diagnostic services overview

2.1 Central routing

Central routing provides a robust, scalable solution and reduces the round-trip delay found in legacy Diag systems. In the central routing environment, remote processors communicate local registrations to the master diag. This communication is performed through an SIO-SMD channel. When the master Diag receives a request, it immediately knows whether to generate an error response or a forward request to the appropriate remote processor. The master Diag uses the following routing:

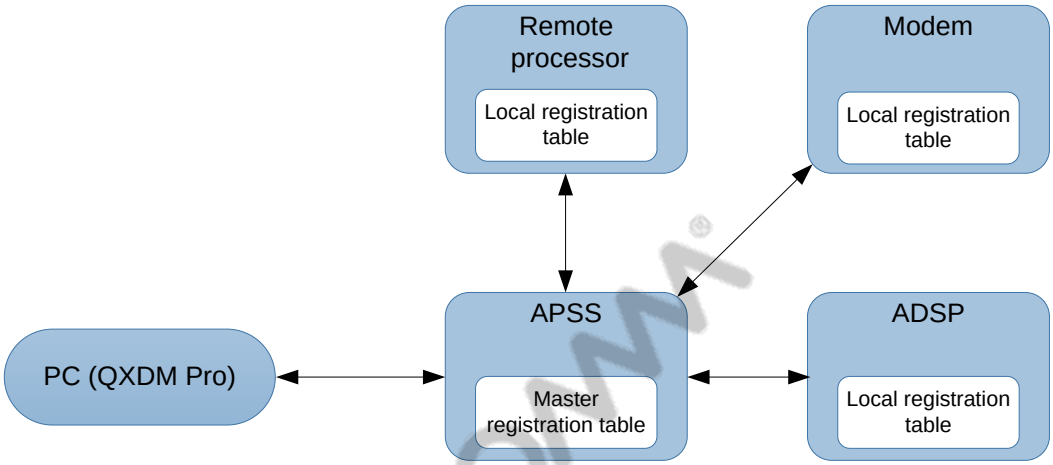
1. At initialization, the remote or slave processors create local registration tables of commands that are sent to the applications processor. The apps processor creates a master registration table of all commands. This table includes the specific processor to be used for each command.



2. When a command request is sent from a connected PC via QXDM Professional™ (QXDM Pro), the applications processor compares the request to its table, and the command is forwarded to the relevant processor.

QUALCOMM®
2016-05-17 06:26:10 PDT
deon_zhang@skey.com.tw

3. The processor handling the command, for example, modem or ADSP, sends a response routed through the applications processor, which forwards it to the PC.



QUALCOMM®
2016-05-17 06:26:10 PDT
deon_zhang@askey.com.tw

Figure 2-1 shows the central routing flow.

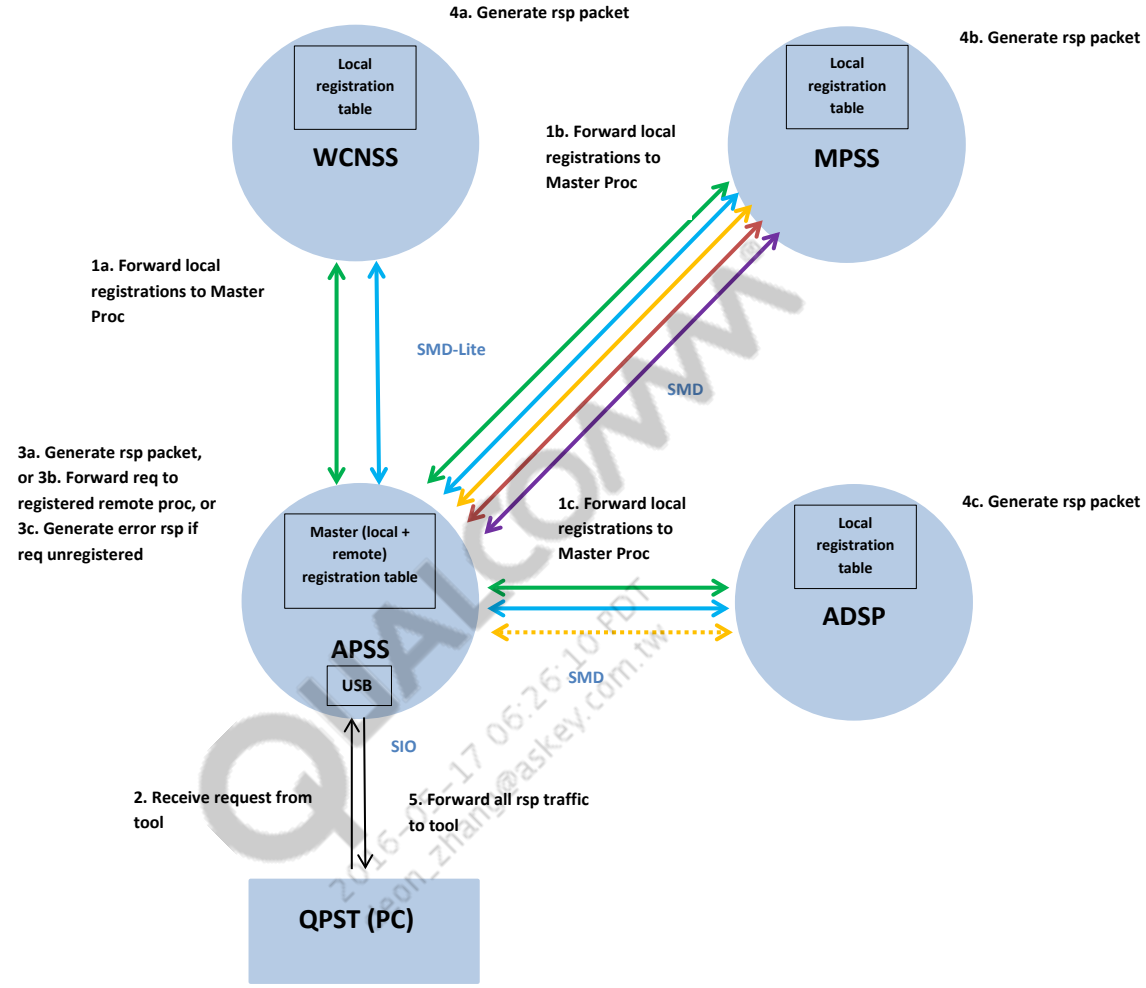


Figure 2-1 Central routing flow

NOTE: On MSM™ devices MSM8916 and MDM9x45, ADSP and MPSS run on the same processor, hence the above SMD channels between APSS and ADSP are not applicable. Common SMD channels exist for MPSS/ADSP packets to be sent to APSS.

2.1.1 Handling common requests

A set of common requests is sent by QXDM Pro to the device when QXDM Pro is connected. This set of commands configures diag by setting the F3 messages, events, and log masks. The commands must be received and processed by Diag on all processors, hence the term *common requests*.

When master Diag is notified that a slave is up, it sends a copy of the most current F3 messages/events/log masks via CTRL messages.

When the master Diag receives a set mask request from a tool, it updates its mask. After the mask is updated, the master Diag sends a copy of the updated mask to all available slaves via CTRL messages. It then responds to the runtime set mask request. The master Diag continues to forward all other common requests to the slaves.

2.2 Working with Diag services

The following methods are available for use with Diag services for processes such as sending Diag commands, accessing logs, saving logs on the device, and logging via QXDM Pro and QPST:

- Diag services API functions – Performs tasks such as defining new commands, event reports, and packets. See 80-V6135-1 for additional details.
- On-device logging – The diag_mdlog application enables and performs logging on the device and saving directly on the SD card. On-device logging offers the advantage of being able to capture log information without being connected to a PC.
- Logging a connected device on a PC via QPST and QXDM Pro:
  - Socket logging
  - UART logging
- Diag Consumer Interface (DCI) – Set of APIs used by a Diag client on the applications processor to inject command requests to supported peripherals and receive responses.
- Listeners – Tasks that register for and receive notification when the specified logs/events/F3 messages are generated.
- QPST polling – Allows users to run a trace to collect information on connection issues.
- Low Power mode – Gives non-HLOS processors the ability to buffer data for longer durations without waking the applications processor, reducing the current drain on the device.

2.3 Diag services APIs

This section describes the basic Diag services. The Request/Response service allows the clients to send commands or solicit data from DMSS/AMSS. Debug Message, Event Reporting, and Logging services are configured by the external device to deliver unsolicited data from DMSS/AMSS when triggered by system operation. Useful functions of the services are:

- Request/Response service – Enables clients to process inbound request packets from and return a response packet to the external device. The client takes appropriate action for the command, generates a response packet using the Packet Service API, and returns (or commits) the response packet. All packets must return a response packet to the external device. Refer to 80-V6135-1 for details.
- Debug Message service – Debugs messaging that is conditionally compiled into the code based on the default build mask or on a custom build mask specified in msgtgt.h. With MSG 2.0, each message is categorized first by a Subsystem ID (SSID), then by up to 32 disjoint categories associated with that subsystem. Refer to 80-V6135-1 for details.
- Event Reporting service – Reports events from the DMSS/AMSS to indicate actions, such as state changes and configuration, related to the operating standards of the system. Refer to 80-V6135-1 for details.
- Logging service – Enables clients to send information to the external device when it is available, rather than when requested from the external device. The external device can individually configure (enable/disable) any log packet in DMSS/AMSS. Refer to 80-V6135-1 for details.

2.3.1 Command service

This service enables clients to receive Diag commands from an external device. A command packet's first byte is the command code. For subsystem commands, there are additional fields:

- Command code – 1 byte
- SSID –1 byte
- Subsystem command code – 2 bytes
- Payload – Up to 4 K - 4 bytes

There are single response commands and delayed response commands for which multiple response packets are sent at certain intervals, for example, modem loopback command (75 18 41 0 1 2 3) can be broken down as:

```
Command code (1 byte): 75
Subsystem ID (1 byte): 18
Subsystem command code (2 bytes): 0 41
Payload: 1 2 3
```

Subsystem commands are discussed in subsystem ICDs, and the nonsubsystem commands are explained in 80-V1294-1.

The packet service operates by using a dispatch table for inbound packets. The master table is a list of client tables, each identified by an SSID, and a table of command code/packet handler function pointer sets.

When it is dispatched, the client must allocate a response packet. Allocation is implemented using the dedicated Diag heap. If allocation is not successful, a block and retry mechanism is employed until enough memory is available for the response packet. Clients should still handle the case of a NULL response allocation by not sending one.

When it is committed, the response is placed in a queue to be serviced by the diag task at the next Tx packet boundary. diagpkt_rsp_send() is called by diagbuf_drain() to dequeue and send a response packet. This is done at every packet boundary to ensure that packet service response packets are sent to the external device as quickly as possible. If packet service latency is greater than ~2 sec, external devices may time out and assume that the connection with Diag services is lost. Therefore, the client should respond in less than 1 sec.

APIs and implementation can be found in diagpkt.h and diagpkt.c.

A simple example using the ESN request and response is shown in Table 2-1 and Table 2-2.

Table 2-1 ESN request

Field	Length (bytes)	Description
CMD_CODE (1/0x01)	1	According to 80-V1294-1

Table 2-2 ESN response

Field	Length (bytes)	Description
CMD_CODE (1/0x01)	1	According to 80-V1294-1
ESN	4	Electronic serial number

2.3.2 Event reporting service

Static events are reported by the DMSS/AMSS to indicate actions such as state changes and configuration that are directly related to the operating standards of the system. These actions are common to all CDMA subscriber units, regardless of the hardware or firmware manufacturer. The intent is to provide a reporting mechanism that minimizes resource usage, in particular, bandwidth, and maximizes storage and transmission priority for operating information that is considered most important in software test and verification.

An event packet has the format shown in Table 2-3.

Table 2-3 Event packet format

Field	Length (bytes)	Description
Command code	1	0x60
Length	2	Length of all the subsequent fields
Event info	2	This field is further broken down as: ▪ Event ID – 12 bits ▪ Reserved – 1 bit ▪ Payload length flag – 2 bits ▪ Time length – 1 bit ▫ 0 – Full system timestamp is 8 bytes ▫ 1 – Truncated timestamp is 2 bytes
Time	Variable	Depending on Time Length

Field	Length (bytes)	Description
Payload Length	0 or 1	Depending if this event has any payload
Payload data	Payload length bytes	If this particular EVENT_ID is specified to contain extra data, it is included in this field

2.3.2.1 Operation

The event service queues events reported and packages several events to be sent in a single packet. The event service is designed to be reliable. Event packets are sent before any other unsolicited diagnostic data type. Event packets are sent when the stale timer expires or the event queue exceeds a configurable threshold size, settable in diagtune.h.

Timestamp truncation

To save bandwidth overhead, multiple events are stored in a single packet. Within a packet, timestamps may be truncated to avoid sending multiple copies of the same timestamp data. The upper 5 bytes of the 8-byte timestamp is the time context. If the time context of an event matches the time context of the previous event in the same buffer, a truncated timestamp is used. A truncated timestamp contains the next 2 bytes (6 and 7) of the timestamp. The least significant byte is omitted since the 1/32 chip granularity is a negligible loss.

Stale timer

To avoid leaving events queued for extended periods of time, a stale timer is used to trigger an event packet to be sent. If the time expires, the buffer is considered stale and is marked as ready to transmit. The time is set to EVENT_TIMER_LEN in diagtune.h. The default is 1 sec. If queueing an event results in the queue exceeding a configurable threshold, the stale timer is shortened to cause the service to trigger a new packet to be sent.

Static Event Reporting uses the diagnostics heap in lieu of the Output Buffer Management System, described below. Events are queued separately in its own queue. If more resources are needed, increase the size of DIAG_HEAP_SIZE in diagtune.h.

Signals

Diag signals are handled in diag_handle_sigs(). Use the command to check Diag signals:

```
v.v diag_tcb
```

Major signals of interest:

- DIAG_EVENT_DRAIN_SIG
  - Set whenever event_stale() is called, indicating that Diag is ready to send an event report packet for primary queue events.
  - Handled by calling event_drain() to drain events from primary event queue event_q to transport layer.
- DIAG_EVENT_DRAIN_SEC_SIG
  - Set whenever event_stale_sec() is called, indicating that Diag is ready to send an event report packet for secondary queue events.

- Handled by calling event_drain_sec() to drain events from secondary event queue event_q_sec to the transport layer.

- DIAG_EVENT_TIMER_SIG
  - Set whenever event_stale_timer expires.
  - Handled by calling event_stale(), which calls event_drain() to drain events from event_q to the transport layer.
- DIAG_EVENT_TIMER_SEC_SIG
  - Set whenever event_stale_timer_sec expires.
  - Handled by calling event_stale_sec(), which calls event_drain() to drain events from event_q_sec to the transport layer.

Event report size (Realtime mode)	128 bytes
Event report size (Non-realtime mode)	1000 bytes
Event stale timer (Realtime)	1000ms
Event stale timer (Non-realtime)	0
Event heap	16 KB
Event secondary heap	4 KB

2.3.3 Debug Message service

The Debug Message service MSG 2.0 improves efficiency, bandwidth requirements, and filter granularity. With MSG 2.0, each message is categorized first by an SSID and then by up to 32 disjointed categories associated with that subsystem.

A message packet has the format shown in Table 2-4.

Table 2-4 Message packet format

Field	Length (bytes)	Description
Command code	1	0x79
Timestamp type	1	▪ 0 – CDMA time ▪ 1 – WIN32 (used in off-target environments)
Number of arguments	1	Specifies the number of 32-bit arguments listed in the Arguments field below
Drop count	1	Total number of messages dropped between this message and the previous one
Timestamp	8	Time the message was originally generated (not transmitted)
Line number	2	Line number identifying location of the message in the file denoted by File name
Subsystem ID	2	Subsystem identifier
Subsystem mask	4	32-bit mask with each bit denoting a category assigned by the internal client denoted by SSID
Arguments	4*Number of arguments(s)	Array of 32-bit signed arguments, corresponding to the printf style Formatted String

Field	Length (bytes)	Description
Formatted String	Variable	NULL-terminated ASCII string containing a printf()-style format string, with formatting specifiers that are replaced with the formatted contents of Arguments in the order they are encountered; allowed formatting specifiers are: ▪ %x – Hexadecimal word display ▪ %u – Unsigned Decimal word display ▪ %d – Decimal word display ▪ %c – Character display ▪ %lx – Long (dword) hexadecimal display ▪ %ld – Long (dword) decimal display **Note:** Float (%f) or string (%s) types are not supported.
File name	Variable	NULL-terminated ASCII filename string identifying the location of message, such as mccdma.c

Table 2-5 shows the macros for Debug Message service MSG 2.0.

Table 2-5 MSG 2.0 macros

Macro	Description
MSG	Message with literal string; no printf(f) style arguments
MSG_N	N refers to the number (1..9) of printf() style arguments in the macro
MSG_SPRINTF_N	Message that calls sprintf() in the caller's context, passing N arguments; this is costly but allows the expansion of types such as string and float

For example, in MSG_2 (ss_id, ss_mask, fmt, arg1, arg2), ss_mask can take the values FATAL, ERROR, HIGH, MED, and LOW (msg_mask.h).

2.3.3.1 Filtering

MSG 2.0 filtering uses a 32-bit mask for each SSID. Each message is filtered using a bitwise AND against a MSG system mask. If the AND operation is nonzero, the message passes the filter. There are two types of filtering:

- Build-time filtering – Each SSID has a build mask with the naming convention, MSG_BUILD_MASK_<SSID name> (msgtgt.h). A MSG macro is conditionally compiled into the code based on this build-time mask.
- Runtime filtering – Filtering is performed based on the mask set by the user in an external tool, for example, QXDM Pro.

Filter table

The external device may query the message service for a list of SSID ranges and build/runtime masks for all SSIDs. When msg.c is compiled, it enables MSG_TGL_GEN prior to including msgcfg.h. This declares a table to be used by the Debug Message service. This table is a list of structures containing SSID ranges and pointers to build mask arrays and runtime mask arrays. The external device may query all of this information. As SSIDs are added to msgcfg.h, this table must also be updated.

Legacy macros are assigned to MSG_SSID_LEGACY (SSID 0), unless MSG_BT_SSID_LEGACY is set to a valid SSID symbol.

2.3.3.2 F3 trace

F3 trace is a runtime buffer in the device's RAM memory where F3/Debug messages can be saved and retrieved later from a RAM dump. F3 trace does not support MSG_SPRINTF macros.

Definitions needed for this feature are:

- FEATURE_SAVE_DEBUG_TRACE – Main feature to enable F3 trace saving; must be defined
- DIAG_F3_TRACE_BUFFER_SIZE – Defines the buffer size; currently set to 128 kB
- FEATURE_SAVE_TRACE_ON_BY_DEFAULT – Enables F3 trace saving by default when NV items are not set. For example:
  - If feature is defined and NV items are set, NV item values are used.
  - If feature is not defined and NV items are not set, F3 trace is disabled.
  - If feature is enabled, the following defines can be used to further control the settings:
    - DIAG_F3_TRACE_CONTROL_MASK_DEFAULT_VAL
    - 0x0 – Disables F3 message saving
    - 0x1 – Enables F3 message saving
    - DIAG_F3_TRACE_DETAIL_MASK_DEFAULT_VAL – 8-bit bitmask (where bit 0 is the Least Significant Bit) that controls the levels of detail of the saved F3 trace; if a bit is set to 1, the information associated with that bit is saved to the buffer; bitmask details are provided in Table 2-6.

Table 2-6 Bit values for DIAG_F3_TRACE_DETAIL_MASK_DEFAULT_VAL

Bit position	Information saved in buffer
7	MSG_FATAL (MSG_MASK_4)
6	MSG_ERROR (MSG_MASK_3)
5	MSG_HIGH (MSG_MASK_2)
4	MSG_MED (MSG_MASK_1)
3	MSG_LOW (MSG_MASK_0)
2	Custom MSG levels (MSG_MASK_5 to MSG_MASK_31)
1	Timestamp
0	F3 arguments

The recover_f3.cmm script is used to extract and process messages from the F3 trace buffer.

2.3.3.3 Qshrink

The main purpose of Qshrink is to reduce the memory footprint of the builds. This module scans a full target build, extracts all debug message strings, creates a hash table of these strings, and modifies the build to use the new hash values instead of the strings to pass to QXDM Pro or other tools. The hash is computed on the debug string and the file name. Prior to Qshrink 2.0, the optimization was done by maintaining a hash table for format strings and file names.

This hash table was shared with QXDM Pro and used by the T32 recovery script to reconstruct the hashed messages from the F3 trace; for example, a 4-byte hash is assigned to the following debug message with an associated source file:

```
712406761:diagdiag.c:Diagpkt_alloc returned NULL in diagdiag_peekw.
```

Qshrink 3.0 additionally hashes the SSID, subsystem mask, and runtime mask associated with the F3 message. The hash table is added to the modem ELF for runtime filtering and reconstruction of hashed messages. The reconstruction is currently being done by Diag for QXDM Pro messages (not needed for an F3 trace), so no change is needed on the QxDM Pro side.

Qshrink 4.0 is the new addition to Qshrink 3.0 to remove debug messages completely from the modem final image, saving an additional 2 MB over Qshrink 3.0 in modem builds.

- The Qshrink4 database file is generated during the build. Each image has its own Qshrink4 database file, unlike Qshrink3 and earlier versions, where the database was generated quarterly.
- The Qshrink4 database files are located with the image binaries and must be in a location that is accessible by the RFS for Diag to access it.
- The Qshrink4 database ID is generated during the build to identify individual image builds. A GUID is used as the Qshrink4 database ID.
- The tool acquires Qshrink4 database files at runtime (QXDM on connect) using a set of Diag commands. To avoid manually identifying and side-loading the database into the tools, the database can be retrieved at runtime using a series of Diag command/response packets. At a high level, the process involves the following steps:

  1. The tool retrieves available database IDs from the device using the File List command.

     The tool queries each known processor for available database IDs using the defined set of query commands found in this document.

  2. Based on the IDs retrieved, the tool decides which databases is to be transferred from the device.

     For efficiency, it is recommended that the tool keep a local cache of previously retrieved databases and only download new databases.

  3. The tool sends a series of command packets (Open, Read, and Close commands) to retrieve the necessary databases.

     In case of a failure, the tool may prompt the user to enter a build location to side-load the database.

  4. The tool uses the databases to convert Qsrhink4 message packets into string debug messages.

For more details on Qshrink4, see Chapter 4.

Figure 2-2 illustrates the first two steps for retrieving a database.

NOTE: The following graphic was added to this document revision.

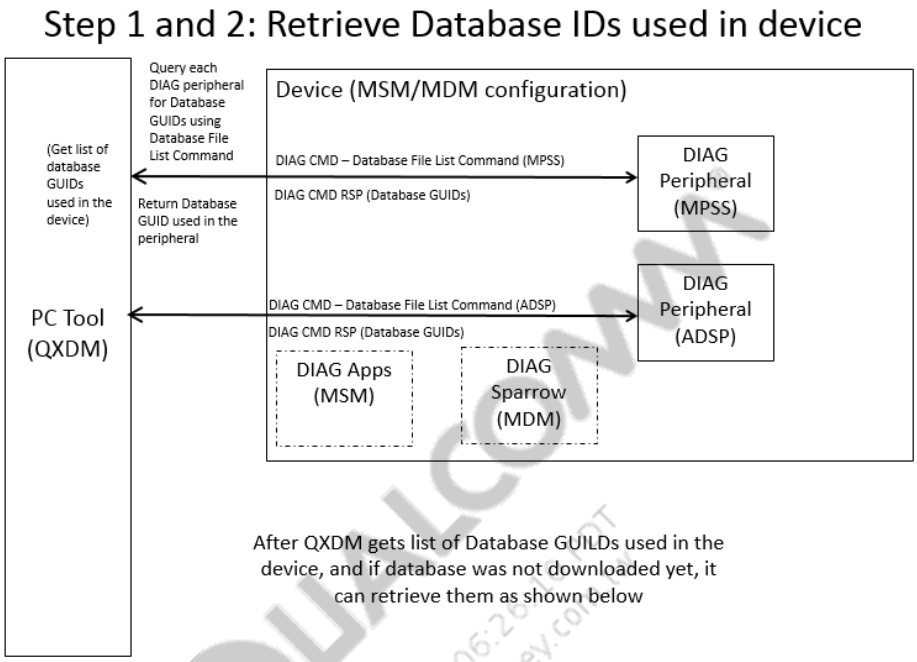


Figure 2-2 Retrieve database IDs

Figure 2-3 illustrates how to read a database using Diag commands.

NOTE: The following graphic was added to this document revision.

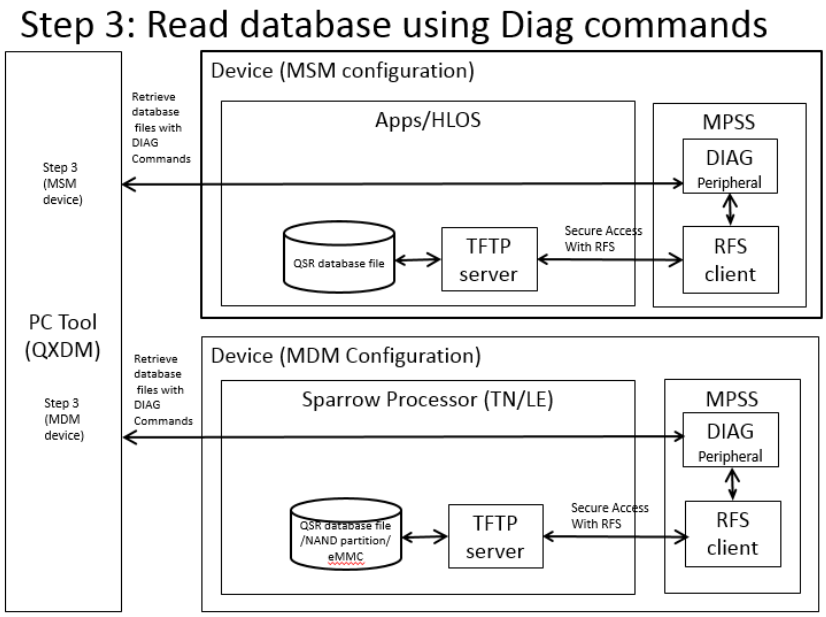


Figure 2-3 Read a database using Diag commands

2.3.4 Logging service

This service enables clients to send information to the external device when it is available, rather than when the information is requested from the external device.

The external device can individually configure (enable/disable) any log packet in the DMSS. A log packet's size is limited only by DIAGBUF_SIZE (diagtune.h) by this service. However, due to legacy reasons, some external devices cannot handle log packets larger than 1 K.

If resources are insufficient, the logging service is unreliable and logs may be dropped. Larger packets are more likely to be dropped.

Log codes are disabled during initialization. The external device uses a bitmask to configure log codes as on/off.

In log on-demand (16/0x10), the DMSS removes the oldest log item and places it in a Log Response message.

A log packet has the format shown in Table 2-7.

Table 2-7 Log packet format

Field	Length (bytes)	Description
Command code	1	0x10
More	1	More log data available indicator; indicates how many log entries (not including the one returned with this message) are queued in diag buffer
Length	2	
Length	2	Length of subsequent fields
Log code	2	Composed of: ▪ Equipment identifier – Most significant 4 bits of the log code specify the equipment ID ▪ Item identifier – Least significant 12 bits of the log code specify the log item ID within the equipment ID
Timestamp	8	System time
Payload	(Length-12)	Data specific to this log packet

QUALCOMM® 2016-05-17 05:25:10 PDT deon_zhang@asker.com.tw

2.4 ADSP Diag architecture

Diag on the ADSP supports the LSM layer, where a Userspace process can forward packets to the Diag Kernel driver (called the QDI driver) by calling write() or ioctl() over the /dev/diag file. There is a Read thread created for every process in order to perform read() from the QDI driver. The QDI driver then sets signals with the DCM thread to communicate with the SMD layer.

The LSM layer or QDI driver are not present on MPSS or WCNSS, hence only kernel space threads set signals with the DCM thread for packet forwarding.

2.4.1 LSM layer initialization

Every Userspace process must register itself with the LSM layer in order to communicate with the Kernel Diag driver. It must call Diag_LSM_Init() before using any Diag APIs.

After successful initialization, the Userspace process can call read()/write()/ioctl() over /dev/diag to send packets or receive requests from the DCM layer.

The Userspace process should call Diag_LSM_DeInit() before exit to unregister from the LSM layer.

2.4.2 Logs/F3s/Events

When a client calls an F3 macro or Log/Event submit APIs, the Diag LSM in that process space checks if the mask for that packet is set in its own copy of masks. The Diag LSM makes a write() call to the QDI driver copying the packet into the Diag buffer, then a Tx signal is set with the DCM thread in order to drain the packet to the SIO/SMD layer.

2.4.3 Command requests/responses

When the Userspace process registers a command with the LSM layer, the LSM maintains a table with all the command codes and handler functions for this process. It forwards this table along with the process ID to the QDI driver. The QDI then forwards it to the DCM thread, which can further write to the SIO/SMD layer. In this way, all commands are forwarded to the APSS side.

When a command request is received, the DCM thread sets a signal with the registered Userspace process and waits for it to read the request packet. The Userspace process calls its associated command handler function and does a QDI write() for the command response packet.

Figure 2-4 illustrates the ADSP Diag architecture.

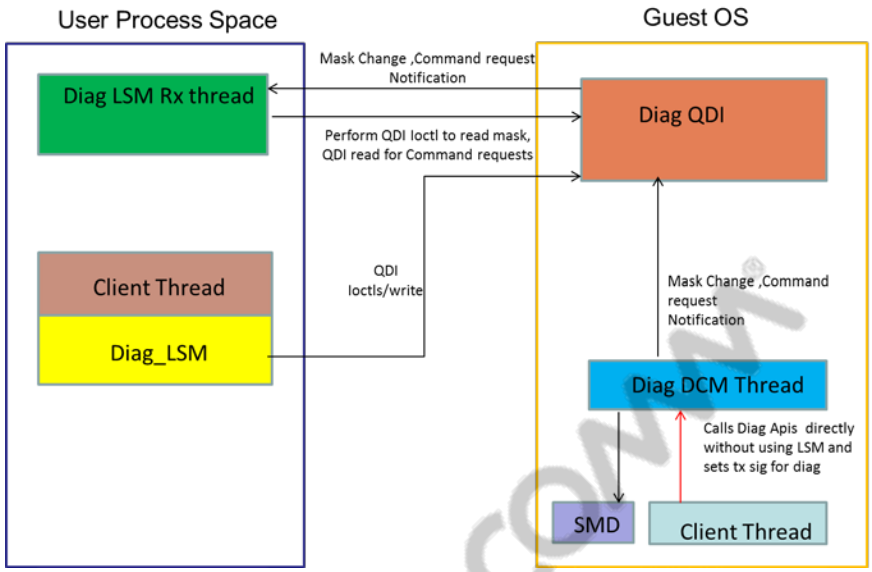


Figure 2-4 ADSP Diag architecture

2.5 Diag architecture on MSM8916 and MDM9x45

On MSM8916 and MDM9x45 targets, the ADSP and MPSS run on the same processor, hence the ADSP LSM layer is copied on the MPSS core for Audio and Sensor processes.

Figure 2-5 illustrates the Diag architecture on the MSM8916 and MDM9x45.

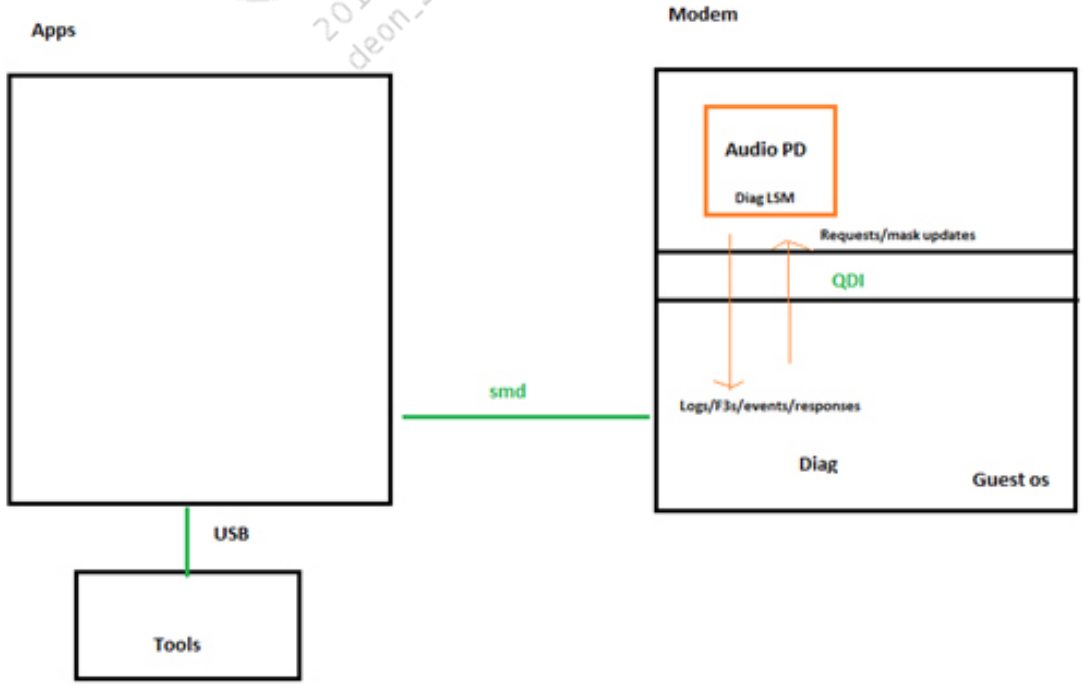


Figure 2-5 Diag architecture on MSM8916 and MDM9x45

2.6 HDLC removal

HDLC support has been removed in newer chipsets, such as MSM8952 and MSM8996. This feature is backward compatible, so it will not break any functionality. Table 2-8 lists the Diag services that are affected by this.

Table 2-8 Diag services affected by HDLC removal

Diag service	Status/tools changed
DCI	Not affected – It is not HDLC encoded
On-device logging (ODL)	Yes, QCAT/APEX changed QXDM change for CFG file
Diag over UART	Yes, QPST changed
Diag over USB	Yes, QPST changed

Table 2-9 lists the tools that contain support for the removal of HDLC.

Table 2-9 Tools that support non-HDLC encoding/decoding

Tools	Version supported	Change
QPST	QPST 2.7.430	Added support to enable/disable HDLC encoding
QCAT	QCAT.06.30.18	Non-HDLC decoding parser
APEX	APEX.06.30.18.04	Non-HDLC decoding parser
QXDM (new Windows version known as QXDM3)	QXDM 3.14.987	Concurrent generation of cfg (HDLC wrapped) and cfg2 (HDLC-stripped) files

HDLC encoding was done when HDLC over UART was used, but with reliable transports like USB, it is not needed. Since not all transports that are used to send diagnostic packets are capable of supporting packet-based delivery, some basic method to frame packets is required in the absence of HDLC encoding. This framing serves two purposes:

- Allows a receiver of diagnostic data to detect boundaries of individual packets
- Allows a basic method of resynchronization in the event of an unforeseen error

Figure 2-6 shows the previous packet format for HDLC (left table) and the new packet format for non-HDLC.

OLD : Packet Framing for HDLC

Field	Length	Description
Payload	N	Non-HDLC encoded Diagnostic packet, as defined in 80-V1294-1 section 3 and other related documents.
CRC	2	Hdlc Encoding CRC
End	1	0x7E

NEW: Packet Framing for Non-HDLC

Field	Length (bytes)	Description
START	1	Set to 0x7E
VERSION	1	Packet framing version number. The packet structure shown in this table corresponds to version 0x01
LENGTH	2	Length of PAYLOAD
PAYLOAD	LENGTH	Non-HDLC encoded Diagnostic packet, as defined in 80-V1294-1 section 3 and other related documents.
END	1	Set to 0x7E

QUALCOMM® 2016-05-17 06:26:10 PDT deon_zhang@askey.com.tw

Figure 2-6 Old vs. new packet formats

The sequence diagram in Figure 2-7 shows the negotiation protocol used by the device and the tool for the HDLC off command, which is used to tell the device to switch to Non-HDLC mode.

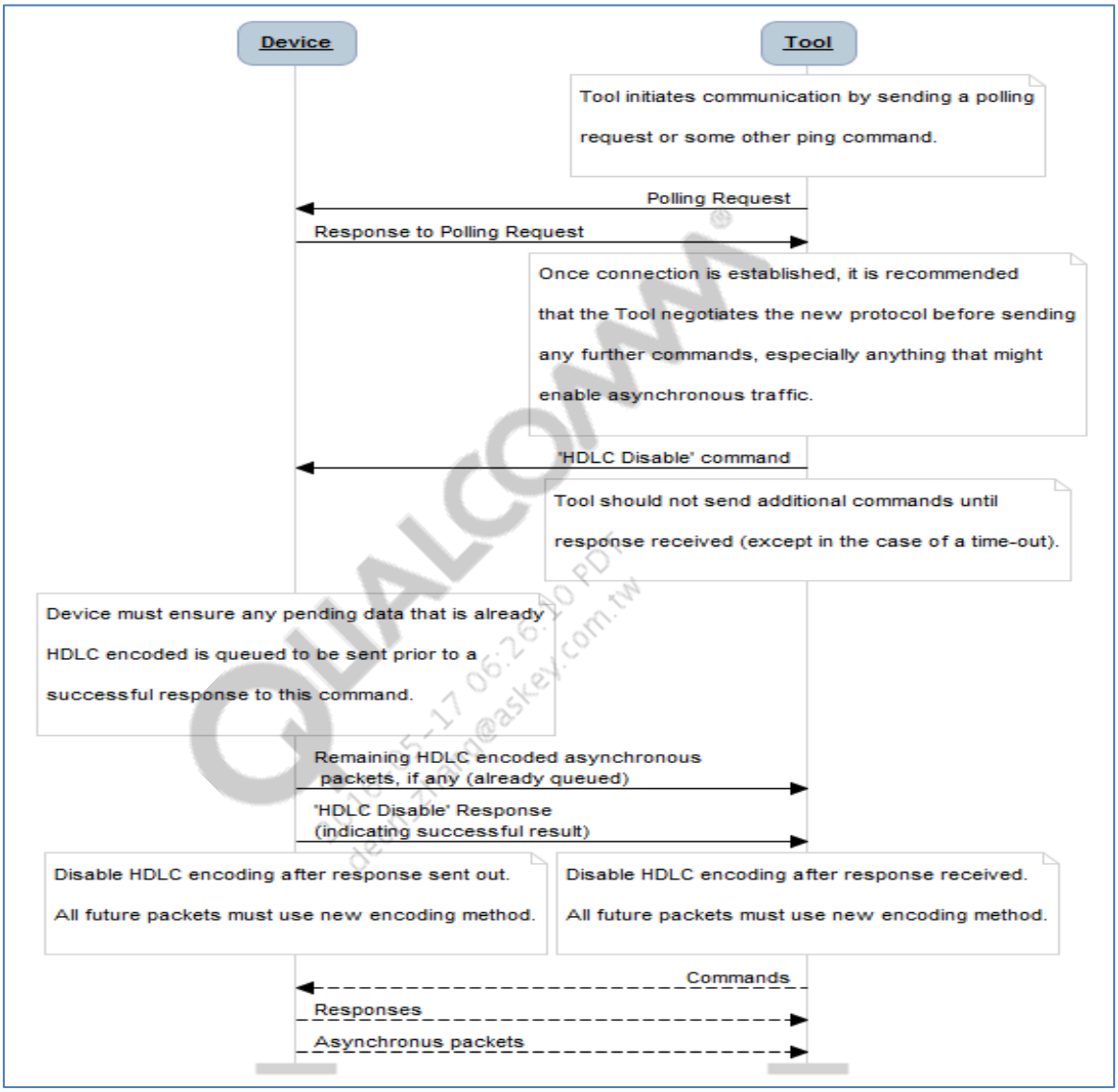


Figure 2-7 Negotiation protocol used by device and tool for HDLC off

Table 2-10 describes the command that is used to switch the mode.

Table 2-10 Switch mode command

Field	Length (bytes)	Description
CMD_CODE	1	Message ID The DM sets CMD_CODE to 75 for this message.
SUBSYS_ID	1	Subsystem ID The DM sets SUBSYS_ID to 18 for this command, indicating that this is a Diag service subsystem command.
SUBSYS_CMD_CODE	2	Subsystem command code The DM sets SUBSYS_CMD_CODE to TBD to indicate that this is an HDLC Disable command.

Table 2-11 shows the result of the switch mode command.

Table 2-11 Response to the switch mode command

Field	Length (bytes)	Description
CMD_CODE	1	Message ID The DMSS sets CMD_CODE to 75 for this message.
SUBSYS_ID	1	Subsystem ID The DMSS sets SUBSYS_ID to 18 for this command, indicating that this is a Diag service subcommand.
SUBSYS_CMD_CODE	2	Subsystem command code The DMSS sets SUBSYS_CMD_CODE to TBD to indicate that this is a response to an HDLC Disable command.
RESULT	1	Result of the HDLC Disable command request: 0 – SUCCESS: HDLC encoding has been successfully disabled. All Diag packets from the DMSS following this response will use the new framing. Values other than 0 indicate an unsuccessful attempt to disable HDLC encoding. 1 – FAILURE: General HDLC disable command failure. 2 – TRANSPORT_ERROR: The device does not support disabling HDLC encoding on the active transport. 3-255 – Reserved.

QUALCOMM CONFIDENTIAL 2015-05-17 ... PDT ... jeon_zhang ... Draft

The new version of QPST has three options used to switch or not switch to HDLC mode. See Figure 2-8.

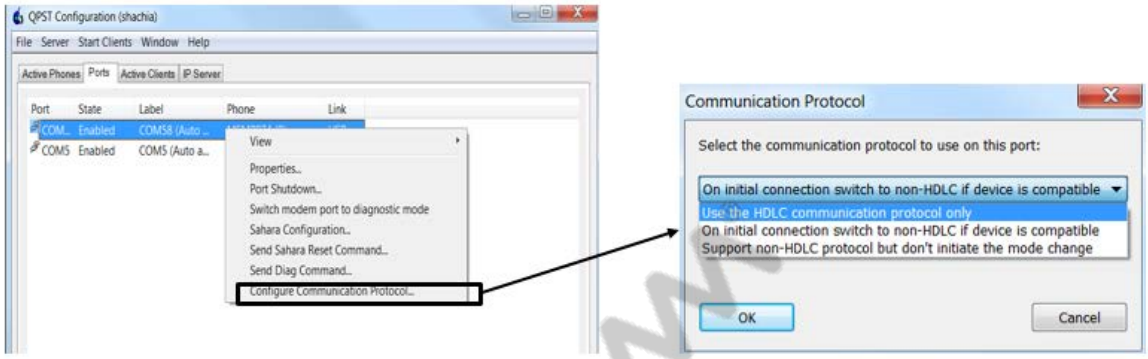


Figure 2-8 QPST HDLC mode switching options

The default option is the second option; On initial connection, switch to non-HDLC if device is compatible. With this option, regardless of the target, QPST always sends the HDLC off command and, if it succeeds, it switches to non-HDLC mode. If it fails, the old mode will continue to be used.

Once the option is changed in QPST, it is retained for that port. Closing the QPST has no effect.

QUALCOMM 2016-05-17 06:21:10 PST deon_zhang@askey.com.tw

3 Common customization examples

3.1 Defining new packets

3.1.1 Defining a new Subsystem Command (Linux, AMSS)

For a full list of command service APIs, see 80-V6135-1.

New commands can be defined on an applications processor (Linux or TN) or non-HLOS processors (modem, ADSP, or WCNSS) using the following process. The following APIs in this section are the same as well.

- Write a command handler function.
- Create a subsystem command table.
- Register the table with diag.

3.1.1.1 Single response command

Single response commands have a single immediate response.

Example – Modem loopback 75 18 41 0 1 2 3

- Write a command handler function.
  - API

```
PACK(void *) (*func_ptr) (PACK(void *) req_pkt_ptr, uint16 pkt_len);
```

  - Sample code on the modem

```
#define DIAG_SUBSYS_DIAG_SERV   18
PACK(void *) diagdiag_subsys_loopback (
  PACK(void *) req_pkt,
  uint16 pkt_len
)
{
    /* Retrieve sub-system command code from request pkt*/
    diagpkt_subsys_cmd_code_type cmd_code =
diagpkt_subsys_get_cmd_code (req);
```

QUALCOMM®
2016-07-17 06:26:10 PDT
Jeon-Hang@askey.com.tw

```
    /* Allocate memory for response pkt */
    diag_subsys_loopback_rsp_type *rsp =
(diag_subsys_loopback_rsp_type *)diagpkt_subsys_alloc
(DIAG_SUBSYS_DIAG_SERV, cmd_code, pkt_len);

    /* Fill rsp structure */

    /* Commit response pkt if it needs to be sent immediately and
return NULL */,
    diagpkt_commit((void *)rsp, pkt_len);
    return NULL;LE

  /* OR return 'rsp' pointer and Diag dispatcher will commit it*/
    return (void *) rsp;
}
```

- Create a subsystem command table. APSS commands are shown in Table 3-1 and modem commands are shown in Table 3-2.
  - API

```
  typedef struct
  {
    word cmd_code_lo; /* Minimum Command code value */
    word cmd_code_hi; /* Maximum Command code value */
    PACK(void *) (*func_ptr) (PACK(void *) req_pkt_ptr, uint16
pkt_len); /* Pointer to Function to handle the packet when command
code is in the range of cmd_code_lo, cmd_code_hi */
  }
  diagpkt_user_table_entry_type;
```

  - Sample code on the modem

```
#define DIAGDIAG_STRESS_TEST_SUBSYS_LOOPBACK_MODEM    0x0029 /* 41 */
static const diagpkt_user_table_entry_type diagdiag_subsys_tbl[] =
{
  {DIAGDIAG_STRESS_TEST_SUBSYS_LOOPBACK_MODEM,
DIAGDIAG_STRESS_TEST_SUBSYS_LOOPBACK_MODEM, diagdiag_subsys_loopback}
  /* can add more entries*/
};
```

- Register the table with Diag
  - API

```
#define DIAGPKT_DISPATCH_TABLE_REGISTER(xx_subsysid, xx_entry) \
  do { \
    static diagpkt_master_table_type xx_entry##_table = { \
     0, 0xFF, xx_subsysid, sizeof (xx_entry) / sizeof (xx_entry[0]),
DIAG_NO_PROC, 0, DIAGCOMM_CTRL_NO_PORT, xx_entry \
    }; \
     /*lint -save -e717 */ \
    diagpkt_tbl_reg_append_proc (&xx_entry##_table); \
  } while (0)
   /*lint -restore */
```

  - Sample code on the modem

```
#define DIAG_SUBSYS_DIAG_SERV   18
DIAGPKT_DISPATCH_TABLE_REGISTER (DIAG_SUBSYS_DIAG_SERV,
diagdiag_subsys_tbl);
```

Table 3-1 Single response APSS command table

cmd_code	SS_ID	client_id*	cmd_lo	cmd_hi	process_id
255	18	0	41	41	-1

*client_id:
- 0 – Modem
- 1 – ADSP/LPASS
- 2 – WCNSS
- 3 – Applications processor

Table 3-2 Single response modem command table

cmd_code	SS_ID	cmd_lo	cmd_hi	Command handler
255	18	41	41	diagdiag_subsys_loopback()

After defining a new log packet, it is necessary to rebuild the code and verify that the new log packet performs properly.

3.1.1.2 Multiple response command

Multiple response commands have an immediate response along with one or more delayed responses.

Example – Modem delayed loopback 128 18 99 0

- Write a command handler function.
  - API – Same as shown in Section 3.1.1.1
  - Sample code on the modem

```
#define DIAG_SUBSYS_DIAG_SERV   18
#define DIAGDIAG_STRESS_TEST_DELAYED_RSP       0x0063 /* 99 */
PACK(void *) diagdiag_delayed_rsp_test (
  PACK(void *) req_pkt,
  uint16 pkt_len
)
{
  uint32 delayed_rsp_id = 0;
  /* Allocates immediate response pkt for sub-system version 2 pkt
(cmd code 128)*/
  diagdiag_delayed_rsp_test_rsp_type *rsp =
(diagdiag_delayed_rsp_test_rsp_type *)diagpkt_subsys_alloc_v2
      (DIAG_SUBSYS_DIAG_SERV, DIAGDIAG_STRESS_TEST_DELAYED_RSP,
sizeof(diagdiag_delayed_rsp_test_rsp_type));
  if(rsp) {
    delayed_rsp_id = rsp->delayed_rsp_id; /*Set by Diag dispatcher to
help Tool unify all responses for this command*/
    /* Commit immediate response pkt, or return 'rsp' pointer for
Diag dispatcher to commit*/
    diagpkt_commit(rsp);
  }

  rsp = NULL;
```

```
  /* Allocates delayed response pkt for sub-system version 2 pkt (cmd
code 128)*/
  rsp = (diagdiag_delayed_rsp_test_rsp_type
*)diagpkt_subsys_alloc_v2_delay(DIAG_SUBSYS_DIAG_SERV,
           DIAGDIAG_STRESS_TEST_DELAYED_RSP, delayed_rsp_id,
sizeof(diagdiag_delayed_rsp_test_rsp_type));
  if(rsp) {
    rsp->response_cnt = 1; /* should increment for each response pkt
sent*/
    /* Must be called for the delayed response to be committed */
    diagpkt_delay_commit(rsp);
  }

  return NULL;
}
```

- Create a subsystem command table; APSS commands are shown in Table 3-3 and modem commands are shown in Table 3-4
  - API – Same as shown in Section 3.1.1.1
  - Sample code on the modem

```
#define DIAGDIAG_STRESS_TEST_DELAYED_RSP       0x0063 /* 99 */
static const diagpkt_user_table_entry_type diagdiag_delayed_rsp_tbl[]
=
{
  {DIAGDIAG_STRESS_TEST_DELAYED_RSP,
DIAGDIAG_STRESS_TEST_DELAYED_RSP, diagdiag_delayed_rsp_test}
/* can add more entries*/
};
```

- Register the table with Diag
  - API

```
  #define DIAGPKT_DISPATCH_TABLE_REGISTER_V2_DELAY(xx_cmdcode,
xx_subsysid, xx_entry) \
    do { \
      static diagpkt_master_table_type xx_entry##_table = { \
        1, xx_cmdcode, xx_subsysid, sizeof (xx_entry) / sizeof
(xx_entry[0]), DIAG_NO_PROC, 0, DIAGCOMM_CTRL_NO_PORT, xx_entry \
      }; \
       /*lint -save -e717 */ \
      diagpkt_tbl_reg_append_proc (&xx_entry##_table); \
    } while (0)
     /*lint -restore */
```

- Sample code on the modem

```
#define DIAG_SUBSYS_CMD_VER_2_F    128
#define DIAG_SUBSYS_DIAG_SERV   18
DIAGPKT_DISPATCH_TABLE_REGISTER_V2_DELAY( DIAG_SUBSYS_CMD_VER_2_F,
DIAG_SUBSYS_DIAG_SERV, diagdiag_delayed_rsp_tbl);
```

Table 3-3 Delayed response APSS command table

cmd_code	SS_ID	client_id*	cmd_lo	cmd_hi	process_id
128	18	0	99	99	-1

*client_id:
- 0 – Modem
- 1 – ADSP/LPASS
- 2 – WCNSS
- 3 – Applications processor

Table 3-4 Delayed response modem command table

cmd_code	SS_ID	cmd_lo	cmd_hi	Command handler
128	18	99	99	diagdiag_delayed_rsp_test ()

After defining a new log packet, it is necessary to rebuild the code and verify that the new log packet performs properly.

3.1.2 Defining a new Log code (Linux, AMSS)

Log packet commands are handled by the Logging Service defined in 80-V6135-1.

New log codes should be added to equipment ID 0, meant for OEMs.

In modem_proc/core/api/services/log_codes.h:

```
//Define Equipment ID 0 Base
#define LOG_OEM_BASE_C  ((uint16) 0x0000)

//Define new Log codes(upto 0x0FFF) under Equipment ID 0
#define OEM_TEST_LOG1 ( (0x001) + LOG_OEM_BASE_C )
#define OEM_TEST_LOG2 ( (0x002) + LOG_OEM_BASE_C )
...
#define LOG_OEM_LAST_C OEM_TEST_LOG2

//This macro is already defined in log_codes.h.  Change it to the last OEM
Log code
#define LOG_EQUIP_ID_0_LAST_CODE_DEFAULT LOG_OEM_LAST_C
```

In modem_proc/core/services/diag/LSM/common/src/diagtune.h:

```
#define LOG_EQUIP_ID_0_LAST_CODE LOG_OEM_LAST_C
```

The following code sample illustrates the use of the new Log code:

```
#include "log.h"
#include "log_codes.h"

typedef struct
{
    log_hdr_type xx_header; /* place holder */
    int param; /* parameter */
    int task_pri; /* task priority */
}   oem_log_type;

void foo_test (uint32 param)
{
    oem_log_type *log = NULL;
    log = (oem_log_type *) log_alloc (OEM_TEST_LOG1, sizeof
(oem_log_type));

    if (log != NULL)
    {
        log.param = param;
        log.task_pri = rex_get_pri ();
        log_commit (log);
    }
}
```

3.1.3 Defining a new Event ID (Linux, AMSS)

API files are in core\api\services\

- event.h
- event_defs.h – Defines event IDs

API functions are:

- event_report() – Reports an event when no parameters are required
- event_report_payload() – Used to report events with parameters that vary with each instance that occurs

Commands are handled by Event Service defined in 80-V6135-1. Event IDs must be defined in event_defs.h. In the following example, 0x9D8 is the last event defined in event_defs.h.

To add a new event, add it to the list of defines and update EVENT_LAST_ID to specify the last event ID. The following example shows the addition of 0x9D9:

```
EVENT_PM_UE_MODE                        = 0x9D6,
EVENT_TDSCDMA_SELF_HOSTING              = 0x9D7,
EVENT_LTE_ML1_SEARCH_IDLE               = 0x9D8,
EVENT_FOO                               = 0x9D9,
EVENT_LAST_ID                           = 0x9D9,
```

The samples in the following sections show example usage for new events.

3.1.3.1 With no parameters

```
#include "event.h"
#include "event_defs.h"

void foo (void)
{
    /* ... */

    event_report (EVENT_FOO);

    /* ... */
}
```

3.1.3.2 With parameters

```
typedef struct
{
   int foo1; /* stores which param */
   int foo2; /* stores the priority of the current task */
}foo_event_type;
typedef enum
{
   FOO_EVENT2
} event_id_enum_type; /* in event_defs.h */
void foo (int a, int b)
{
   foo_event_type payload;
   /* ... */
   payload.foo1 = a;
   payload.foo2 = b;
   event_report_payload (EVENT_FOO2, sizeof (foo_event_type), (void *)
   &payload);
}
```

After defining an event report, it is necessary to rebuild the code and verify that the new event report performs properly.

QUALCOMM 2016-05-17 06:26:10 PDT leon_zhang@askey.com.tw

3.1.4 Defining new message SSID

Commands are handled by Debug Message Service defined in 80-V6135-1. This section describes the API functions needed to define new SSIDs and use them in F3 messages.

All SSIDs are defined in msgcfg.h. Customers can define new SSIDs between 0xC000 and 0xCFFF. QTI will not use this SSID range. The definition of a new message SSID differs based on whether you defining on the AMSS or the Linux APSS side.

3.1.4.1 AMSS changes

You can either add the SSID in an existing category or create a new category.

3.1.4.1.1 Adding a new SSID to an existing category (general)

In core\api\services\msgcfg.h, define your SSID to a value 1 more than the last entry in the General category. Assign the *_LAST macro for this category to your SSID value.

```
#define MSG_SSID_GEN_FIRST
…
#define MSG_SSID_FOO 102
#define MSG_SSID_GEN_LAST     102
```

In diag\diag_mask_tbl\src\msg_arrays_i.c:

```
static const uint32 msg_bld_masks_gen[] =
{
…
   MSG_BUILD_MASK_MSG_SSID_FOO
};
```

3.1.4.1.2 Adding a new SSID to a new category

In core\api\services\msgcfg.h:

```
   #define MSG_SSID_FOO 49152
   #define MSG_SSID_FOO_LAST 49152

Increment MSG_MASK_TBL_CNT by 1
```

In diag\diag_mask_tbl\src\msg_arrays_i.c, add:

```
static const uint32 msg_bld_masks_foo[] =
{
   MSG_BUILD_MASK_MSG_SSID_FOO
};
static uint32
msg_rt_masks_preset_foo[DIAG_MAX_PRESET_ID][sizeof(msg_bld_masks_foo) /
sizeof(msg_bld_masks_foo[0])];
static uint32
msg_rt_masks_dci_foo[DIAG_MAX_DCI_ID][sizeof(msg_bld_masks_foo) /
sizeof(msg_bld_masks_foo[0])];
uint32 msg_listener_masks_foo[sizeof(msg_bld_masks_foo) /
sizeof(msg_bld_masks_foo[0])];
uint32 msg_listener_ext_masks_foo[sizeof(msg_bld_masks_foo) /
sizeof(msg_bld_masks_foo[0])];


msg_mask_tbl_type msg_mask_tbl[] = {
//snip
//add this as the last entry in this array
   ,
   {
      MSG_SSID_FOO,
      /* FIRST + array count */
      MSG_SSID_FOO+
      sizeof(msg_bld_masks_foo) / sizeof(msg_bld_masks_foo[0]) - 1,
      msg_bld_masks_foo,
      { (uint32*)msg_rt_masks_preset_foo, (uint32*)msg_rt_masks_dci_foo },
      { (uint32*)msg_rt_masks_preset_foo, (uint32*)msg_rt_masks_dci_foo }
   }
};
msg_mask_tbl_type listener_msg_mask_tbl[] = {
//snip
//add this as the last entry in this array
   ,
   {
      MSG_SSID_FOO,
      /* FIRST + array count */
      MSG_SSID_FOO+
      sizeof(msg_bld_masks_foo) / sizeof(msg_bld_masks_foo[0]) - 1,
      (uint32*)msg_listener_masks_foo,
      (uint32*)msg_listener_ext_masks_foo
   }
};
```

QUALCOMM® 2016-5-17 06:26:10 PDT zhanq@askey.com.tw

3.1.4.1.3 Adding a build mask in msgtgt.h

In core\api\services\msgtgt.h, define the mask for the new SSID. In the following example, MSG_BUILD_MASK_MSG_SSID_FOO is defined as a new mask:

```
#define MSG_BUILD_MASK_MSG_SSID_FOO        (MSG_LVL_LOW)
```

3.1.4.1.4 Test code

The following code sample illustrates the use of the new SSID:

```
#include "msg.h"
#include "msgcfg.h"

void foo_test (int parm)
{
    int param1, param2, param3 = 0;
    MSG_3 (MSG_SSID_FOO, MSG_LEGACY_HIGH,
           "MSG_3 param1 %d param2 %d param3 %d \n",
           param1, param2, param3);
}
```

After adding a new SSID, it is necessary to rebuild the code to verify that the new SSID is registered.

3.1.4.2 Linux APSS changes

As per the above AMSS changes, you could either add the new SSID to an existing category or create a new category.

3.1.4.2.1 Adding a new SSID to an existing category (general)

In file kernel\include\linux\diagchar.h, set *_LAST macro for General category to the newly added SSID.

```
#define MSG_SSID_0_LAST     102

static const uint32_t msg_bld_masks_0[] = {
   //add 1 Mask entry for you new SSID at the end of this array
    MSG_LVL_LOW
};
```

3.1.4.2.2 Adding a new SSID to a new category

In kernel\include\linux\diagchar.h, increment MSG_MASK_TBL_CNT by 1

```
   #define MSG_SSID_<X>          49152
   #define MSG_SSID_<X>_LAST          49152
```

Where <X> = MSG_MASK_TBL_CNT -1.

```
//Define new array for your new Category of SSIDs
static const uint32_t msg_bld_masks_<X>[] = {
   MSG_LVL_LOW
};
```

In kernel\drivers\char\diag\diag_masks.c:

```
static int diag_create_build_time_mask(void)
{
//snip
   //Add a new Case statement for your new Category
   case MSG_SSID_<X>:
      tbl = msg_bld_masks_<X>;
      tbl_size = sizeof(msg_bld_masks_<X>);
      break;
//snip
}
```

Where <X> = MSG_MASK_TBL_CNT -1.

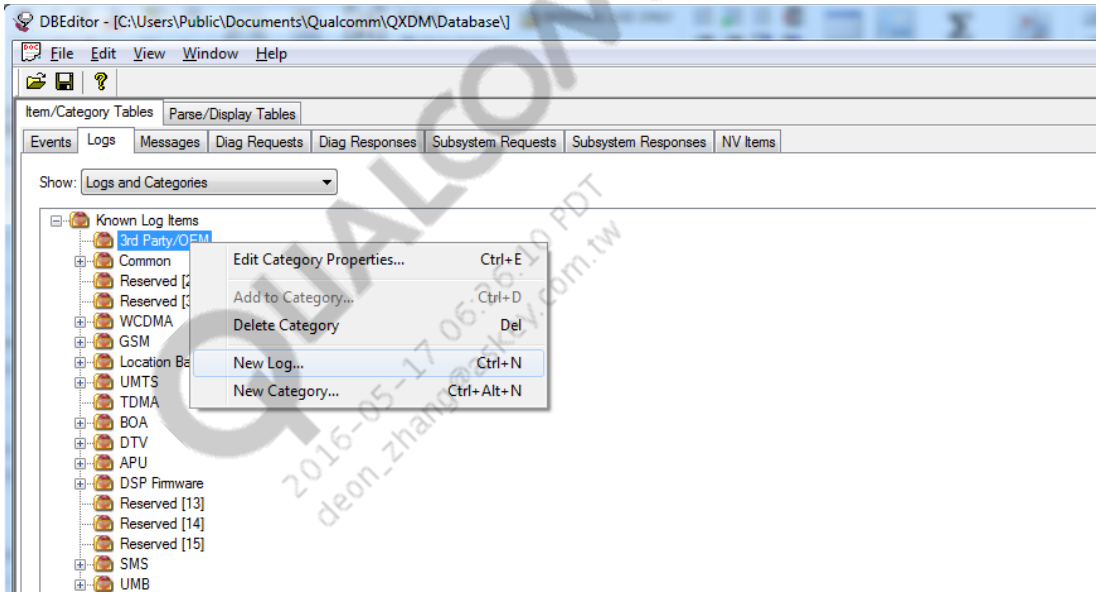
QUALCOMM®
2016-05-17 06:26:10 PDT
deon_zhang@askey.com.tw

3.2 Changing a QXDM Pro database to support newly added packets

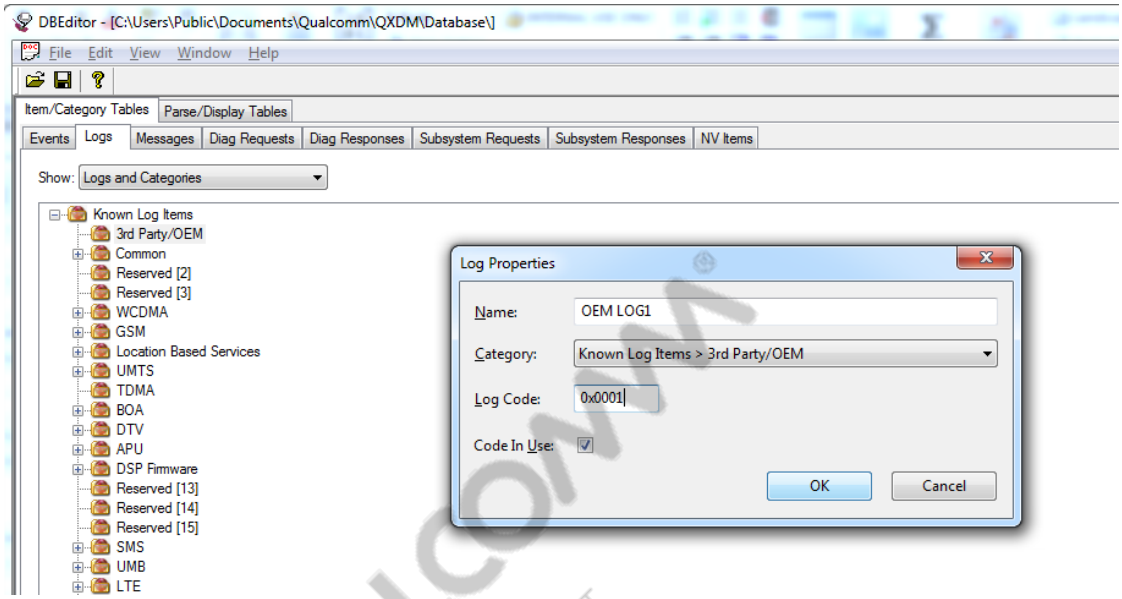
In order to see the newly added packets in QXDM Pro, the masks for the packets must be added. To add mask filters for these newly added logcodes/event IDs/message SSIDs, it is necessary to update the QXDM database. See 80-V9396-1 for more details.

To add filters for the above defined packets:

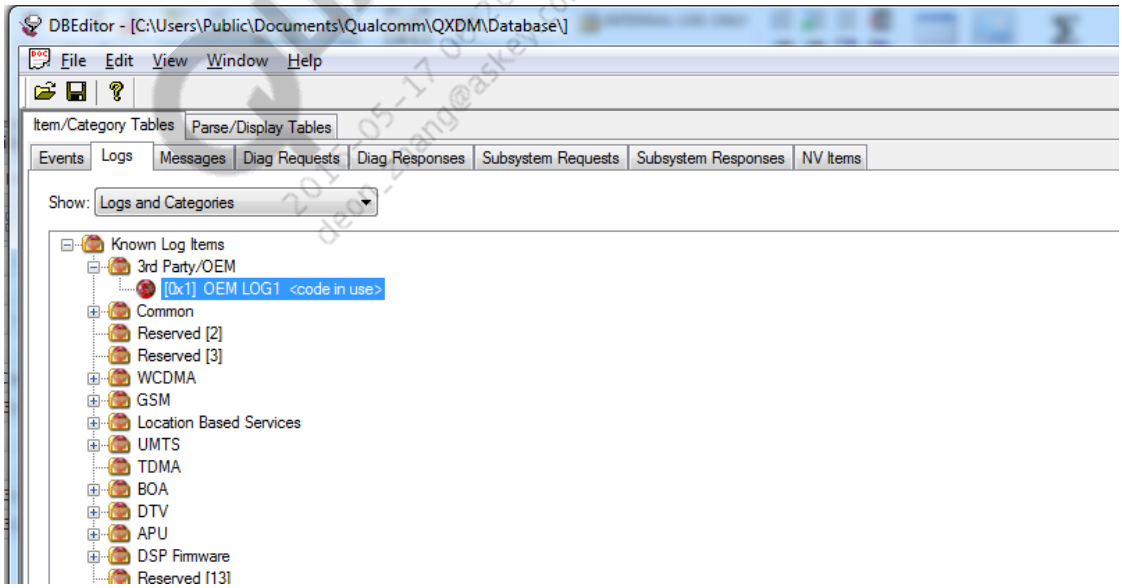
1. Launch QXDM Pro.
2. Open the Database Editor from Tools → Database Editor. The current database opens. In this example, you will add a Log code for the OEM_TEST_LOG1 (defined above).
3. In the Logs tab, right-click on 3rd Party/OEM Category and select New Log….



5. Enter the log properties:

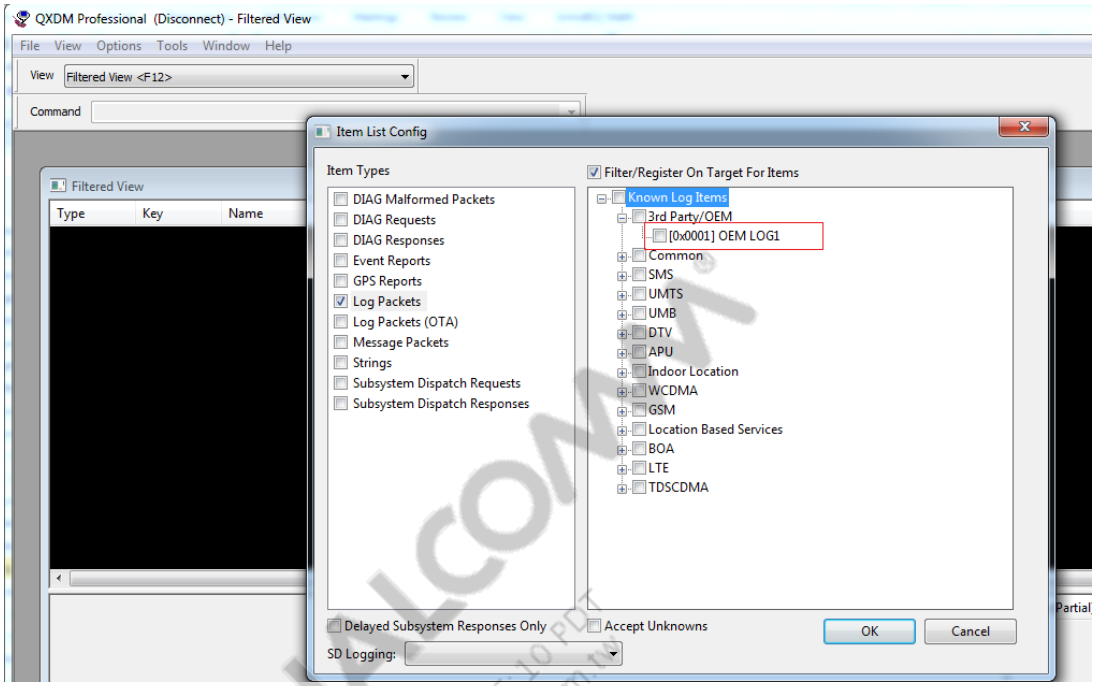


The log code gets added to the list.



6. Save the Database and close QXDM Database Editor window.

7. Open a new session. A mask option for the added log code should now be shown.



QUALCOMM
2016-05-17 06:26:10 PDT
deon_zhang@askey.com.tw

3.3 Adding listeners (AMSS only)

3.3.1 Adding log listeners

3.3.1.1 Single log listener

On an AMSS core, a task can register for and receive notification when the specified log is generated on the same core.

```
Function: diag_add_log_listener

Description
  This routine registers a function to be called when the specified log
  is generated by DMSS/AMSS.
Input parameters
   log_code: Log code for which the listener want to be registered for.
   listener: Callback function to be called when the log is generated for a
             registered log code.
   param: User input that will be passed unmodified to the registered
          function when called.

Returns
  TRUE – listener was successfully added to the listener table.
  FALSE – Listener was not added due to a lack of resources.

============================================================================
  boolean diag_add_log_listener (const unsigned int log_code,
                const diag_log_listener listener,
                void *param);

Function: diag_remove_log_listener

Description
  This routine unregisters a function from the log listener table.
Input parameters
   listener will no longer be called when the log code is unregistered.
   log_code, listener, and param are used to match a registered
   listener

Returns
  Boolean indicating the success of the operation.  If FALSE, the entry was
  not found in the listener table.

============================================================================
*/
  boolean diag_remove_log_listener (const unsigned int log_code,
                    const diag_log_listener listener,
                    void *param);
```

3.3.1.2 Extended log listener

A task can register for multiple log codes as well using the extended log listener. This function should be used if we want to register for multiple listeners instead of using diag_add_listener function multiple times.

```
Function: diag_add_log_listener_ext

Description
  This routine registers a function to be called by the Diag task when any
  one of the logs specified by the 'logs' array is generated by DMSS/AMSS.
Input parameters
  logs: Specifies a logs array.
  num_logs: The number of logs present in the logs array.
  listener: Listener.
  param: This will be passed unmodified to the registered function when
         called.

Returns
  TRUE – Listener was successfully added to the listener table.
  FALSE – Listener was not added due to lack of resources.

============================================================================
*/
  boolean diag_add_log_listener_ext (const uint16* logs, const unsigned int
num_logs,
                        const diag_log_listener_ext listener, void *param);

Function: diag_remove_log_listener_ext

Description
  This routine unregisters a function from the log listener table.
Input parameters
  listener: Used to match a registered listener. listener will no longer be
            called when any of the logs registered earlier by calling
            diag_add_log_listener_ext() is generated.
            If duplicate entries exist, only one is removed.

Returns
  Boolean indicating the success of the operation. If FALSE, the entry was
  not found in the listener table.

============================================================================
*/
  boolean diag_remove_log_listener_ext (const diag_log_listener_ext
listener);
```

3.3.2 Adding event listeners

3.3.2.1 Single event listener

On an AMSS core, a task can register for and receive notification when the specified event is generated on the same core.

```
Function: diag_add_event_listener

Description
  This routine registers a function to be called by the Diag task when the
  specified event is generated by DMSS/AMSS.
Input parameters
  event_id: Event ID.
  listener: Listener.
  param: This will be passed unmodified to the registered function when
         called.
  Duplicate entries are allowed.

Returns
  TRUE – Listener was successfully added to the listener table.
  FALSE – Listener was not added due to lack of resources.
============================================================================*/

  boolean diag_add_event_listener (const unsigned int event_id,
                   const diag_event_listener listener,
                   void *param);

Function: diag_remove_event_listener

Description
  This routine unregisters a function from the event listener table.
  Parameters event_id, listener, and param are used to match a registered
  listener.
Input parameters
  event_id: Event ID.
  listener: This will no longer be called if the event ID is unregistered.
  If duplicate entries exist, only one is removed.

Returns
  Boolean indicating the success of the operation. If FALSE, the entry was
  not found in the listener table.

============================================================================*/
  boolean diag_remove_event_listener (const unsigned int event_id,
                      const diag_event_listener listener,
                      void *param);
```

3.3.2.2 Extended event listener

A task can register for multiple event IDs as well using the extended event listener:

```
Function: diag_add_event_listener_ext

Description
  This routine registers a function to be called by the Diag task when any
  one of the events specified by the events array is generated by
  DMSS/AMSS.
Input parameters
  events: Events array.
  num_events: The number of events present in the events array.
  listener: Listener.
  param: This will be passed unmodified to the registered function when
called.

Returns
  TRUE – Listener was successfully added to the listener table.
  FALSE – Listener was not added due to lack of resources.
============================================================================
*/

  boolean diag_add_event_listener_ext (const uint16 *events, const unsigned
int num_events,
                        const diag_event_listener_ext listener, void
*param);

Function: diag_remove_event_listener_ext

Description
  This routine unregisters a function from the event listener table.
Input parameters
  listener: Used to match a registered listener.listener will no longer be
            called when any of the events registered earlier by calling
            diag_add_event_listener_ext() is generated
  If duplicate entries exist, only one is removed.

Returns
  Boolean indicating the success of the operation. If FALSE, the entry was
  not found in the listener table.

============================================================================
*/
  boolean diag_remove_event_listener_ext (const diag_event_listener_ext
listener);
```

3.3.3 Adding F3 listeners

3.3.3.1 Single F3 listener

On an AMSS core, a task can register for and receive notification when the specified F3 message is generated on the same core.

```
Function: diag_add_f3_listener

Description
  This routine registers a function to be called when the specified F3
  with matching mask is generated by DMSS/AMSS.
Input parameters
  ssid: Subsystem ID.
  ss_mask: Subsystem mask.
  listener: Listener
  param: Will be passed unmodified to the registered function when called.
  Duplicate entries are allowed.

Returns
  TRUE – Listener was successfully added to the listener table.
  FALSE – Listener was not added due to lack of resources.

============================================================================
*/
  boolean diag_add_f3_listener (const unsigned int ssid,
                                const unsigned int ss_mask,
                                const diag_f3_listener listener,
                                void *param);

Function: diag_remove_f3_listener

Description
  This routine unregisters a function from the F3 listener table.
Input parameters
  listener: Will no longer be called when the F3 SSID is unregistered.
  ssid, ss_mask, listener, and param are used to match a registered
  listener.
  If duplicate entries exist, only one is removed.

Returns
  Boolean indicating the success of the operation. If FALSE, the entry was
  not found in the listener table.

============================================================================
*/
  boolean diag_remove_f3_listener (const unsigned int ssid,
                                   const unsigned int ss_mask,
                                   const diag_f3_listener listener,
                                   void *param);
```

3.3.3.2 Extended F3 listener

A task can register for multiple F3 messages as well using the extended F3 listener:

```
Function: diag_add_f3_listener_ext

Description
  This routine registers a function to be called by the DIAG Task when any
  of the SSID+Mask pairs in the keys array matches with the F3 generated by
  DMSS/AMSS.
Input parameters
  keys: Keys array.
  numkeys: Number of keys.
  listener: Listener.
  param: Will be passed unmodified to the registered function when called.
        Duplicate entries are allowed.

Returns
  TRUE – Listener was successfully added to the listener table.
  FALSE – Listener was not added due to lack of resources.

============================================================================
*/
boolean diag_add_f3_listener_ext (f3_listener_ext_key_type *keys,

const unsigned int num_keys,

const diag_f3_listener_ext listener,

void *param);

Function: diag_remove_f3_listener_ext

Description
  This routine unregisters a function from the F3 listener table.
Input parameters
  listener: Will no longer be called for any F3 generated by DMSS/AMSS.
  param: Parameter.
  If duplicate entries exist, all will be removed.

Returns
  Boolean indicating the success of the operation. If FALSE, the entry was
  not found in the listener table.

============================================================================
*/
  boolean diag_remove_f3_listener_ext (const diag_f3_listener_ext listener,
void* param);
```

QUALCOMM®
2015-05-17 06:36:10 PDT
deon_zhang@asky.com.tw

4 On-device logging

On-device logging allows diagnostic traffic to be saved to a memory device; that is, SD card or eMMC. It can be used in field testing when connecting the device to a workstation or laptop is undesirable. Later on, host PC tools can be used to process the logged items files.

On-device logging uses circular logging, which allows logging to continue even if the device runs out of memory. When the device runs out of memory, the application deletes the oldest log file to create free space and continues logging. With each deletion of a file, the application prints the log filename and size of the log file (in kB) that was deleted to create space. The deleted logs are not recoverable.

Figure 4-1 illustrates how the mdlog application works.

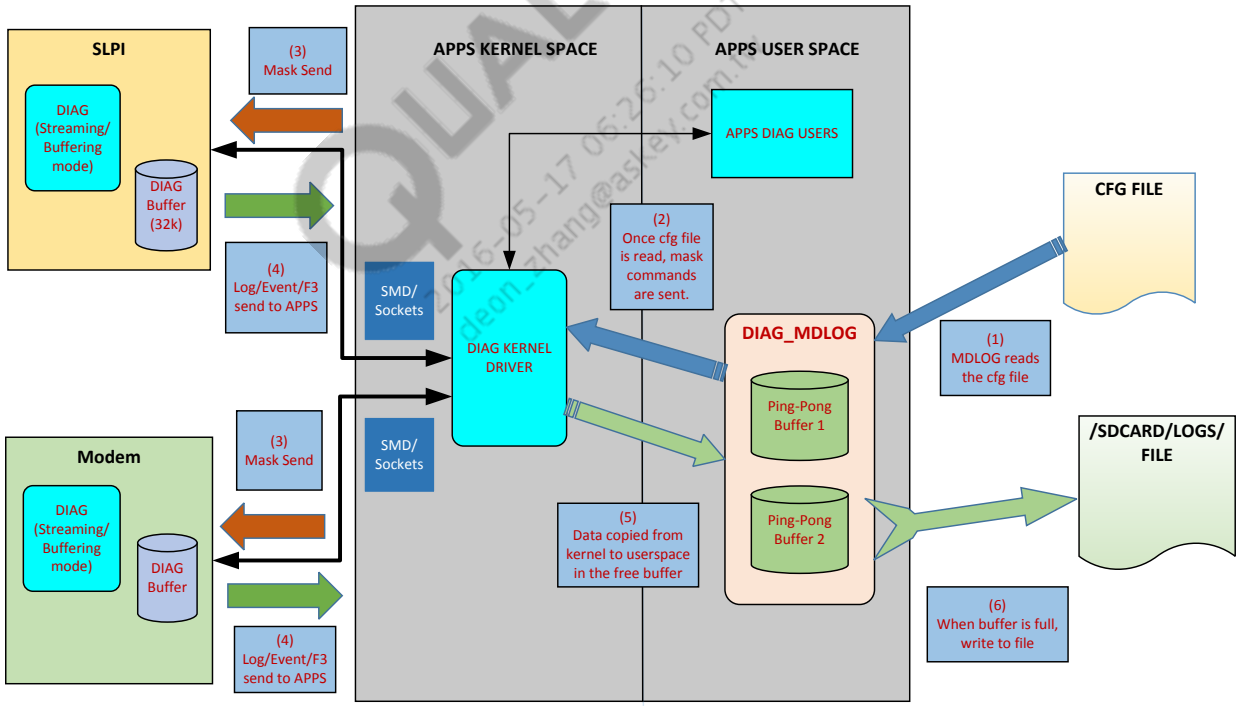


Figure 4-1 Block diagram of the mdlog application operation

4.1 Using diag_mdlog

Below is an example on how to use diag_mdlog. This enables the mdlog application.

diag_mdlog -f <mask_file name> -o <output dir> -s <size in MB> -c

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4.2 Command line options avaialable

Table 4-1 shows the available options with the diag_mdlog application.

Table 4-1 Options available with diag_mdlog application

Argument	Function
-f	Path and mask filename for MSM™ and APQ targets
-m	Path and mask filename for MDM targets
-l	Name of file containing the list of mask files
-o	Name of the output directory
-s	Maximum log file size in MB
-w	Waiting for directory
-n	Maximum log file number
-k	Kill existing instance of diag_mdlog
-c	Send mask cleanup to modem at exit. This should be given while starting diag_mdlog, and not while killing it as 'diag_mdlog –k'
-d	Disable console messages
-e	Run using wakeup source to keep apps processor on
-b	Have peripherals buffer data and send data in nonreal-time
-h	Usage help; prints a list of these arguments
-a	Disable HDLC encoding

The -s, -n, and -o options are used to manage the files created by MDLOG:

- The -s option is used to limit the size of each file. When the specified file size is reached, a new file is created.
- The -o option is used to specify the output file directory.
- The -n option is used to limit the number of files created. If the number of files is not specified, MDLOG will keep creating new files until the storage space is filled. When the number of files is specified, the oldest file is deleted when the file limit is reached.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Figure 4-2 illustrates the flow using the diag_mdlog command line options.

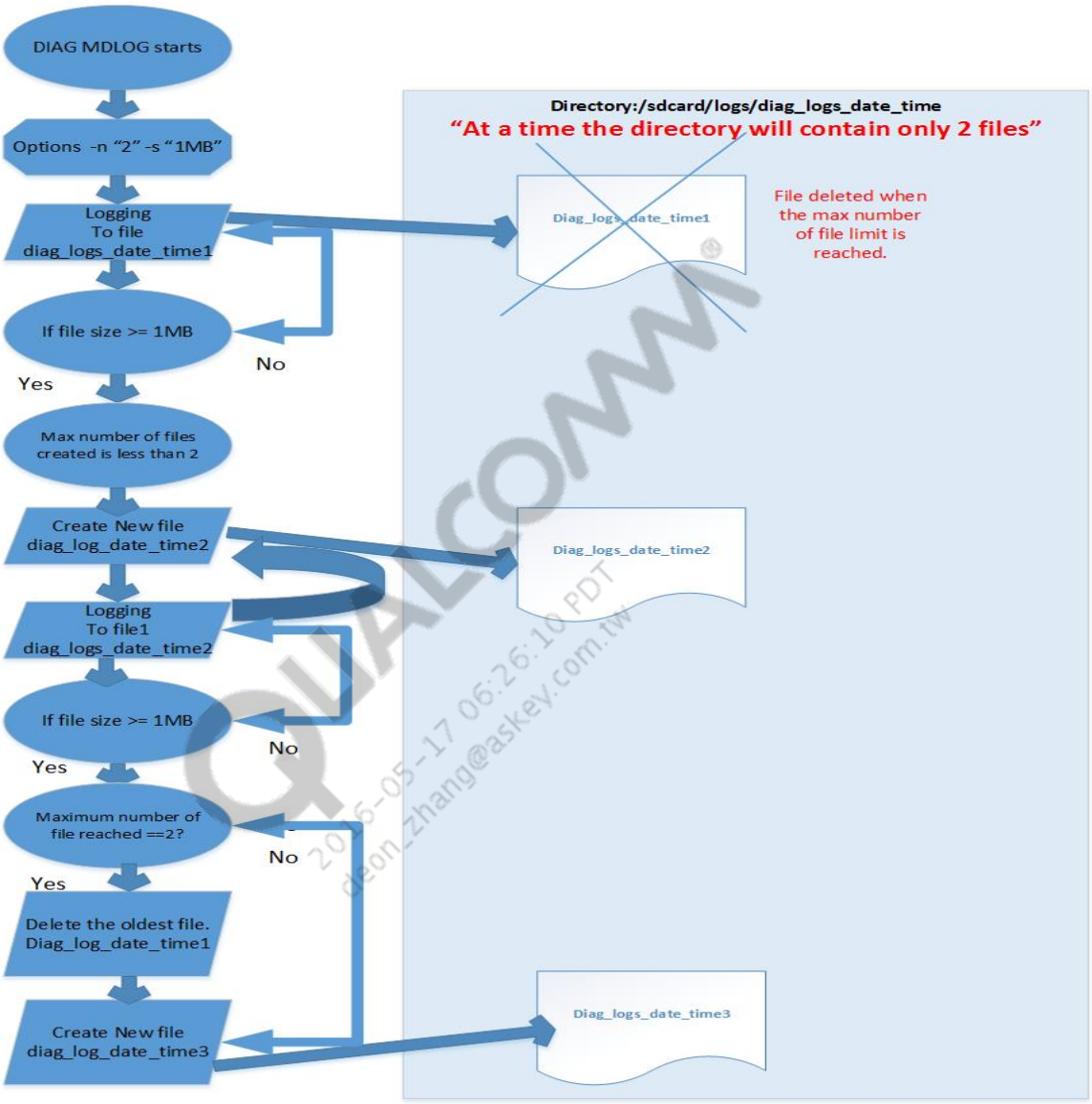


Figure 4-2 Flow diagram for using the diag_mdlog command line options

MDLOG creates the directory and files in the following format:

```
diag_logs_date_time
diag_logs_date_time.qmdl
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4.3 Running on-device logging (diag_mdlog)

Two methods can be used to run the on-device logging (diag_mdlog) application:

- From an ADB shell
- Using the Qualcomm Settings menu on device
- From a user space application

4.3.1 Running diag_mdlog using ADB

1. Push the mask configuration file (.cfg) to the device (see Section 4.4 to learn how to create a .cfg file).

   ```
   adb push <local path to Diag.cfg> /data
   chmod 777 /data/Diag.cfg
   ```

2. Create an output directory

   ```
   mkdir /data/diag_logs
   chmod 777 /data/diag_logs
   ```

3. Run the logging application.

   ```
   /system/bin/diag_mdlog –f /data/Diag.cfg –o /data/diag_logs/test1 –c
   /system/bin/diag_mdlog –f /data/Diag.cfg –o /data/diag_logs/test1 –c -b
   ```

The directory `/data/diag_logs/test1` is created by the application. Log files show up at this location.

If an SD card has been mounted, logs can be saved in the SD card directory /sdcard/diag_logs.

4.3.2 Running diag_mdlog on the device

1. In the phone UI, navigate to **Settings→Qualcomm Settings**.
2. Select the option to enable on-device logging to route traffic to the SD card on the phone. Clearing this option switches the traffic back to USB logging.

4.3.3 Starting mdlog from another application

NOTE: This section was added to this document revision.

To start mdlog from another C application, do a system call and invoke mdlog just like it is done from the command line:

```
system(“/system/bin/diag_mdlog & “);
```

NOTE: The application should have root access and other read/write access to the SD card.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4.4 Troubleshooting SD card issues

If the SD card is not present when the app is launched, the app displays “file open error.” In addition, the following issues can occur when the SD card is removed:

- If the SD card is removed during logging, the Android OS kills the logging application because it has an open file handle to the SD card that no longer exists. In such cases, the current open file may or may not get corrupted.
- If the SD card is removed and reinserted, the device does not always detect the SD card because polling for the SD card is disabled in the default configuration to save power. Always ensure that the card is properly mounted before launching the diag_mdlog application.

4.5 Diag_mdlog output file

This section describes the output file that is generated by diag_mdlog and how that file can be parsed. Diag_mdlog generates the output file in *.qmdl format. This must be convertet it into a .isf file to view it in QXDM.

4.5.1 Converting .qmdl/.qmdl2 files to .isf for use in QXDM Pro

With the introduction of the HDLC off feature (see Section 2.6), whenever the -a option is used with diag_mdlog, use the .cfg2 file. The output will then be a qmdl2 file.

All Diag traffic is written to diag_log_<num>.qmdl/qmdl2 files in the output folder. The .qmdl files can be read back using QCAT and converted for use in QXDM Pro.

To convert a .qmdl/qmdl2 file to an .isf file for viewing in QXDM Pro:

1. Load the .qmdl file in QCAT and save it as a .dlf file.
2. Use Tools→DLF File Converter (bundled with QXDM Pro) to convert from .dlf to .isf.

The QMDL file in the hex editor shows that a new header has been added to the QMDL file to help parse the data. Table 4-2 describes the new header for a QMDL2 file.

Table 4-2 New header for a QMDL2 file

Field	Length (bytes)	Description
HeaderLength	4	Number of bytes reserved for header.Data in the QMDL2; starts after the header.
Version	1	Version value to be set.
HDLCDataType	1	0 – Indicates that HDLC encoding is disabled 1 – Indicates that HDLC encoding is enabled
GuidListEntryCount	4	Number of GUIDs available to read.
GUID (GuidListEntryCount)	16*(GuidListEntrtCount	Array of available GUIDs. This can expand up to the maximum number of GUID list entries reserved in Diag.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Figure 4-3 shows the difference between the QMDL (top) and the QMDL2 (bottom) formats.

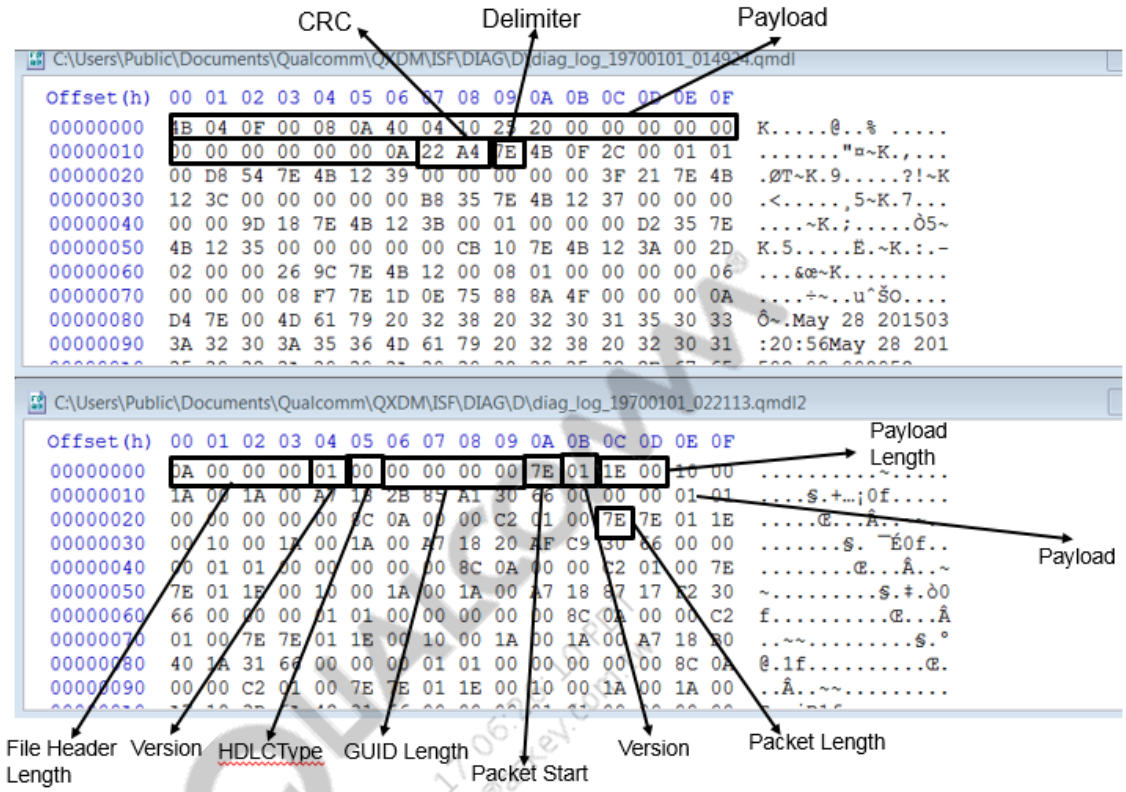


Figure 4-3 Difference between the QMDL and QMDL2 formats

4.6 Diag masks

With on-device logging, to enable Diag packet masks on each processor:

1. Generate a file containing all mask information from QXDM Pro.
2. Place this file in the diag_logs folder in the output location.

When working with diag masks in QXDM Pro, Filtered View must be used. However, multiple instances of Filtered View, or other views, may remain open at the same time. Saving diag masks in QXDM Pro does not require a device connection.

4.6.1 Creating a mask config (.cfg/.cfg2) file

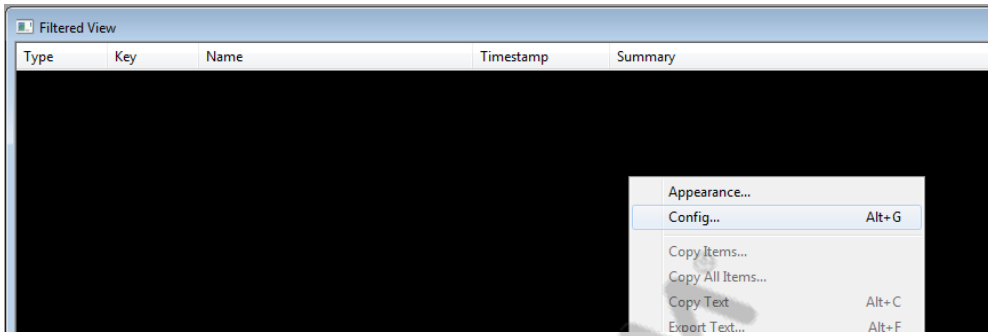
With the introduction of the HDLC off feature (see Section 2.6), a new configuration file (.cfg2) is now used. The file is generated along with the .cfg file using the following steps:

1. Launch QXDM Pro.
2. Masks are loaded only from Filtered View. Use one of the following options to select Filtered View:
   - Press **F12**.

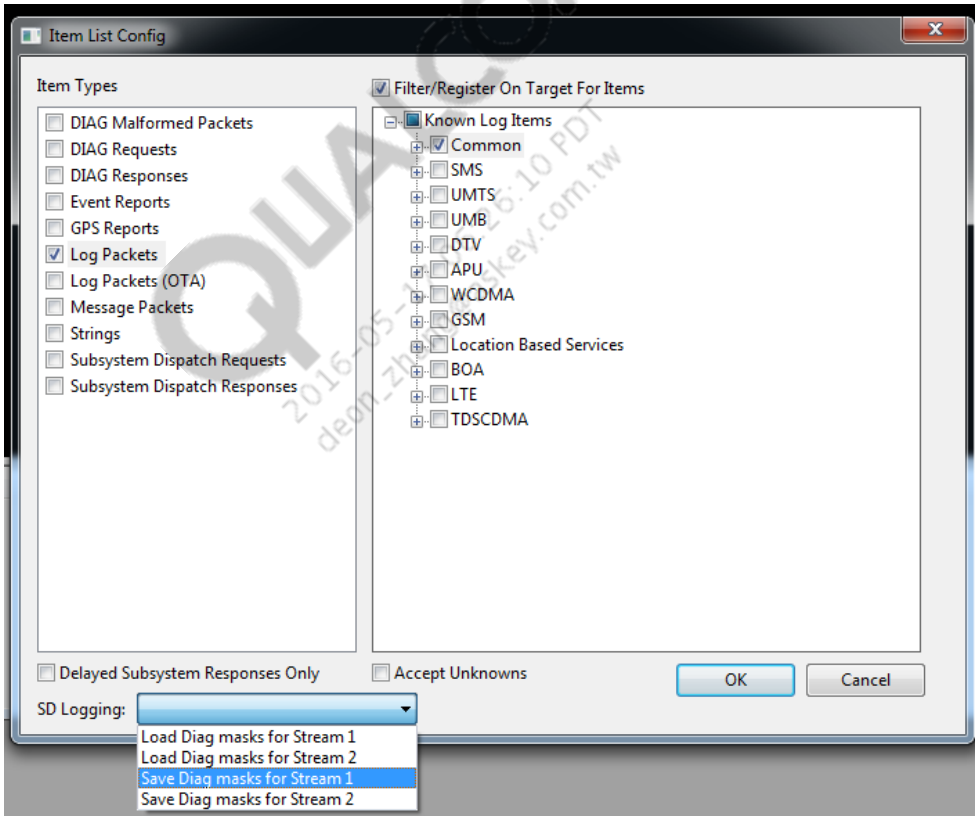
     or
   - Select View→New→Common→Filtered View.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3. Right-click the Filtered View pane and select **Config**.



4. Select the intended Log/Event/Message (F3) masks.
5. In the SD Logging drop-down list, select Save Diag masks for Stream 1 and click **OK**.



A dialog appears for saving the .cfg file.

NOTE: Stream 2 is intended for an obsolete feature and should not be used.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

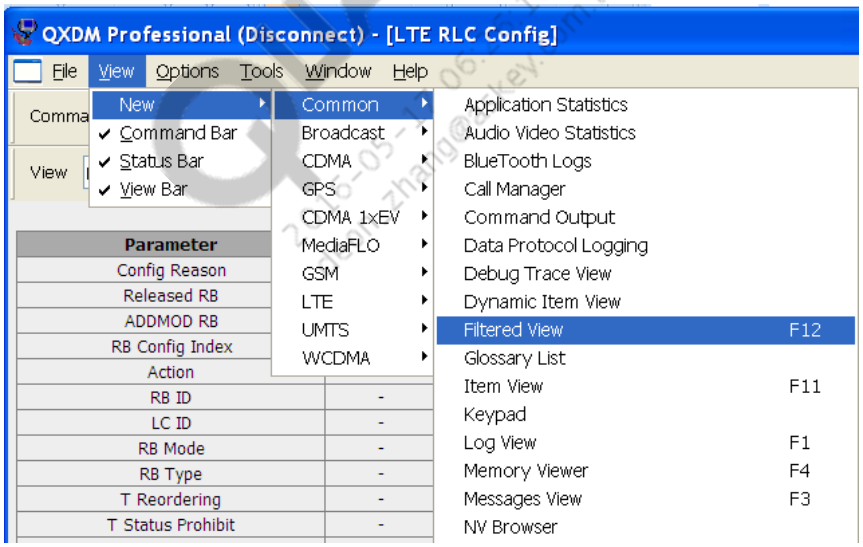
6. Enter a filename or keep the system-generated one and click **Save**. A diag.cfg/.cfg2 file, for example, DIAG_test1.cfg, is generated in the C:\Users\Public\Documents\Qualcomm\QXDM\ISF\DIAG directory.

NOTE: It is not necessary to close QXDM Pro before generating another config file, although there is no harm in doing so. Closing QXDM Pro after generating a config file does not result in deletion of a saved file.

4.6.2 Loading a mask config (.cfg/.cfg2) file in QXDM Pro

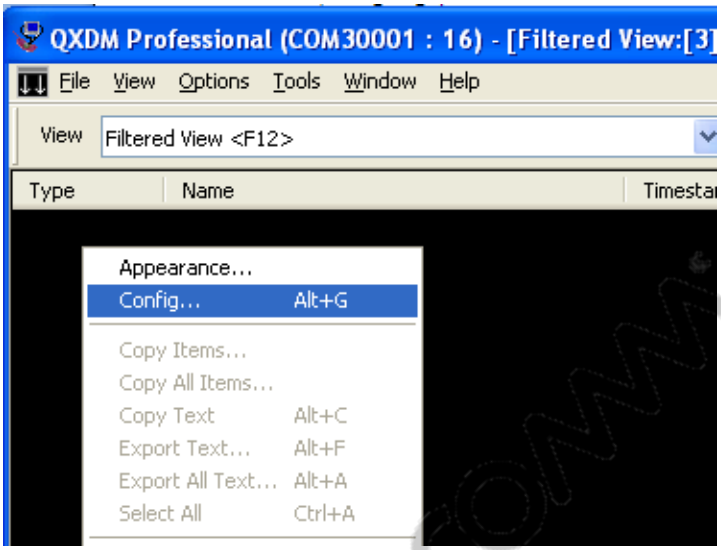
1. Launch QXDM Pro.
2. It is necessary to disconnect the device before proceeding. Check the device and disconnect it from the PC, if necessary.
3. Masks are loaded only from Filtered View. Use one of the following options to select Filtered View:
   - Press **F12**.

     or
   - Select View→New→Common→Filtered View.

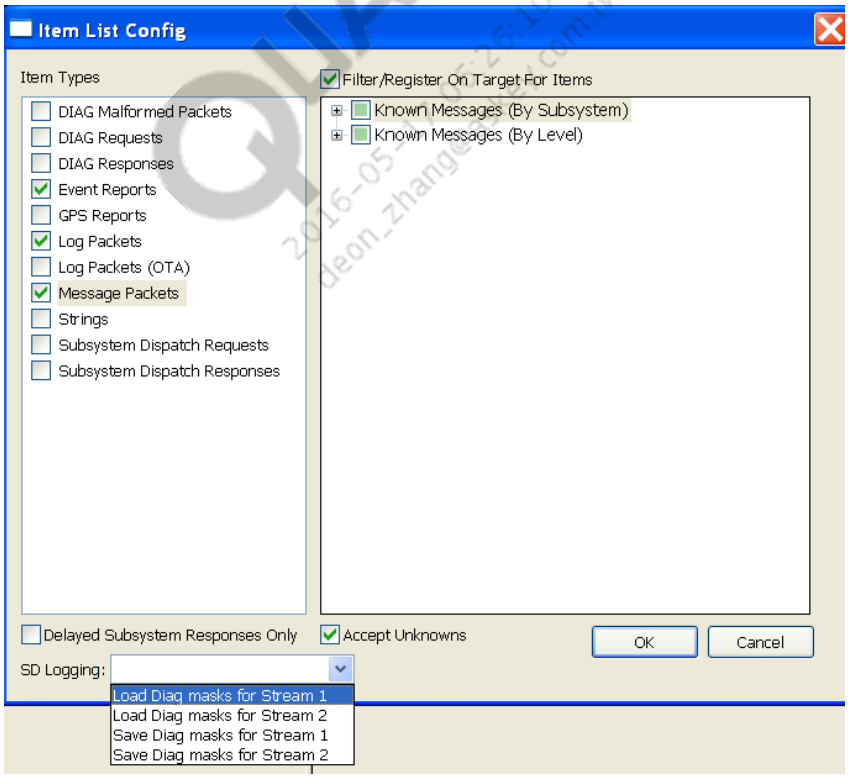


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4. Right-click the Filtered View pane and select **Config**.



5. In the SD Logging drop-down list, select Load Diag masks for Stream 1.

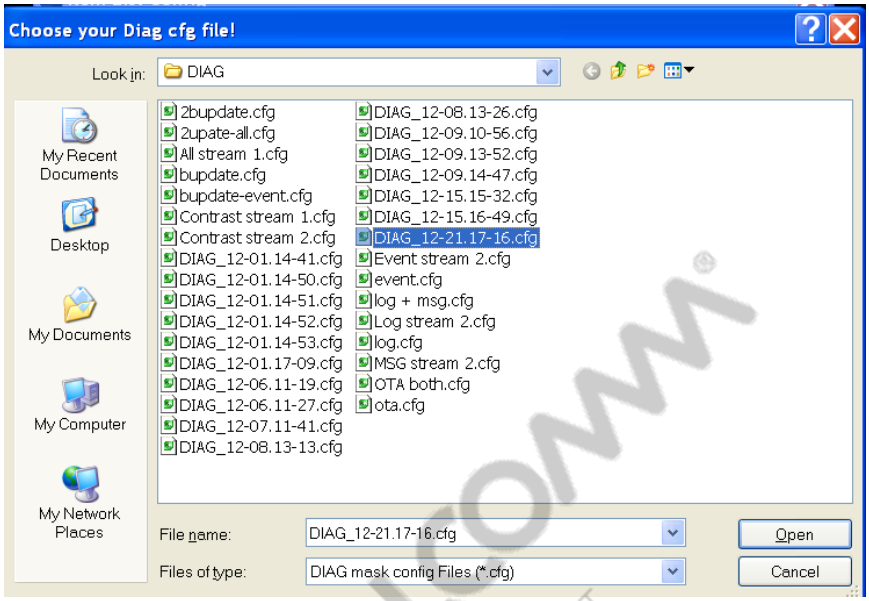


A dialog appears for loading the .cfg file.

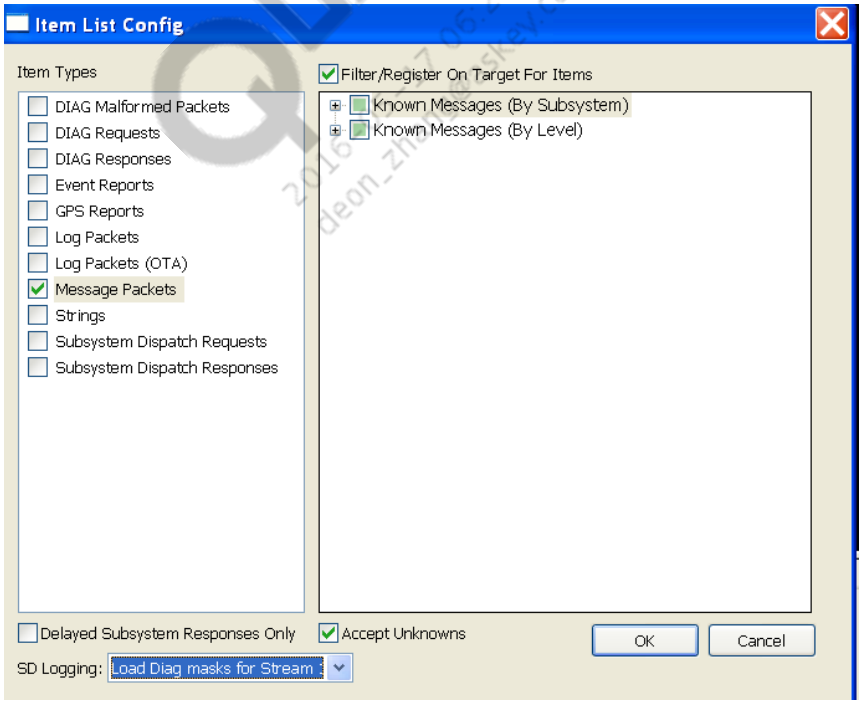
NOTE: Stream 2 is intended for an obsolete feature and should not be used.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6. From the file dialog that appears, select an existing diag.cfg file and click **Open**.



The Diag masks from the chosen stream are loaded and displayed.



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4.6.3 Saving Diag stress test command requests

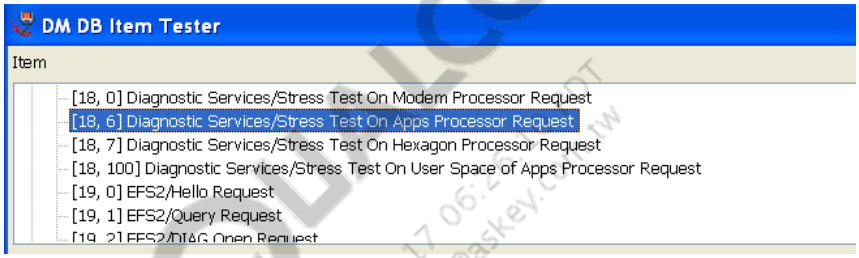
Diag stress testing allows checking the health of each service, such as F3 messages, logs, and events on each processor. QXDM Pro can be used to append such a command request to a .cfg file.

For example, to check the health of the F3 message service on the modem processor, the command is:

```
75 18 0 0 1 0 0 0 1 1 1 0 0 1 0 0 0 0 0 0 0 0 0 0
```

To save a diag stress test command into a .cfg file:

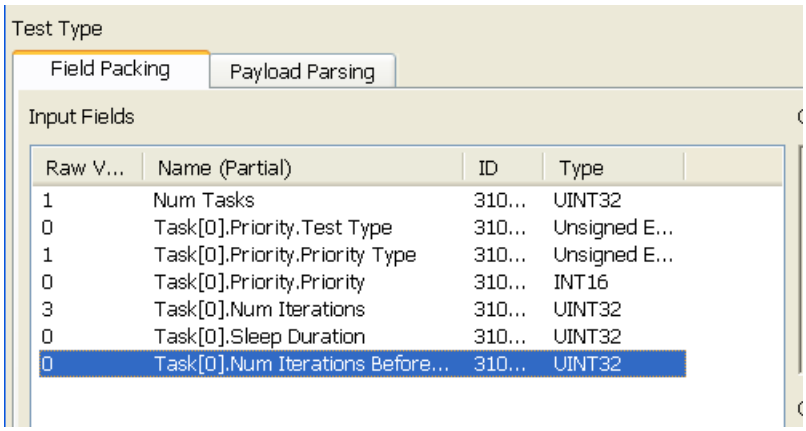
1. Launch Item Tester bundled with QXDM Pro.
2. Expand Subsystem Dispatch Request in the Item pane and scroll down to subsystem [18] Diagnostic Services.



The name of the diag stress test request displays in the QXDM Pro Command Example status bar.



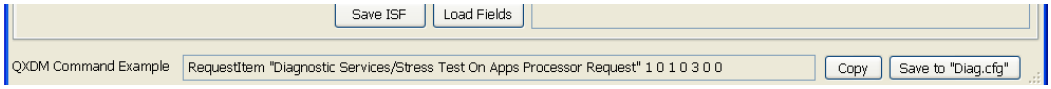
3. On the Field Packing tab, tune the command parameters.



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

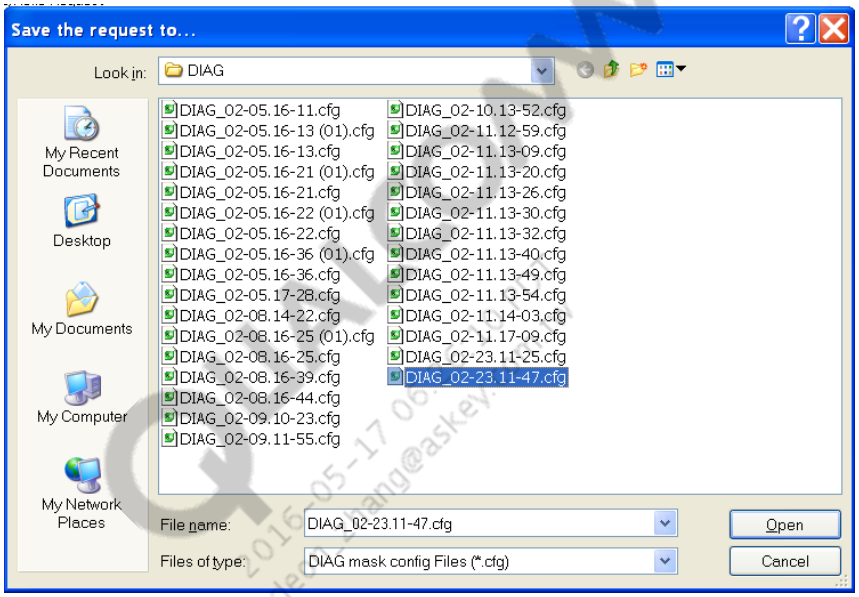
4. To append the command to an existing diag.cfg file:

   a. Click **Save to "Diag.cfg"**.



   b. Select the applicable .cfg file and click **Open**. The subsystem command automatically appends to the end of the selected file.

      Note that this dialog is displayed only if a Diag stress test subsystem was selected.



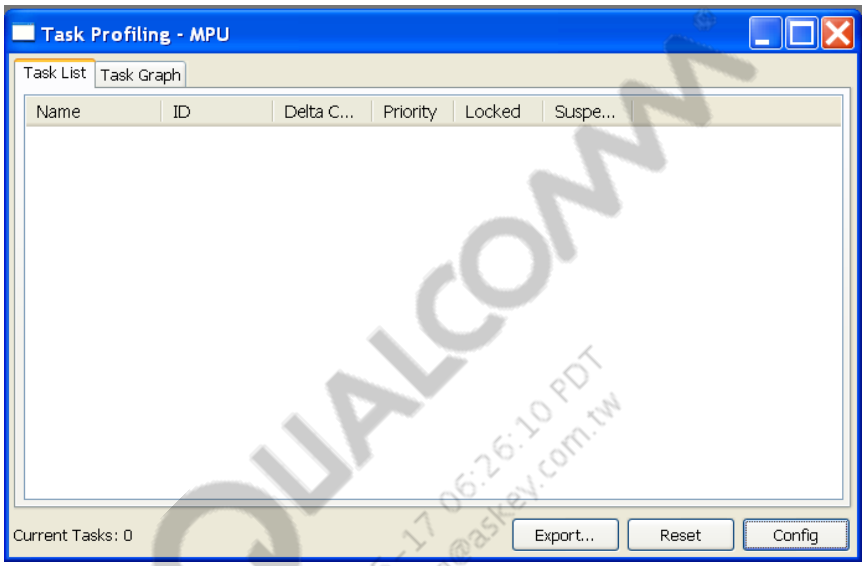
NOTE: If there are no .cfg files in the folder, see Section 4.4.1 for instructions on creating a .cfg file.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4.6.4 Saving Diag task profile requests

Diag task profiling requests allow system-level tasks and per-task profiling logs to be viewed. See 80-V1241-21 for information on configuring this feature. To save task profile requests into the diag.cfg file:

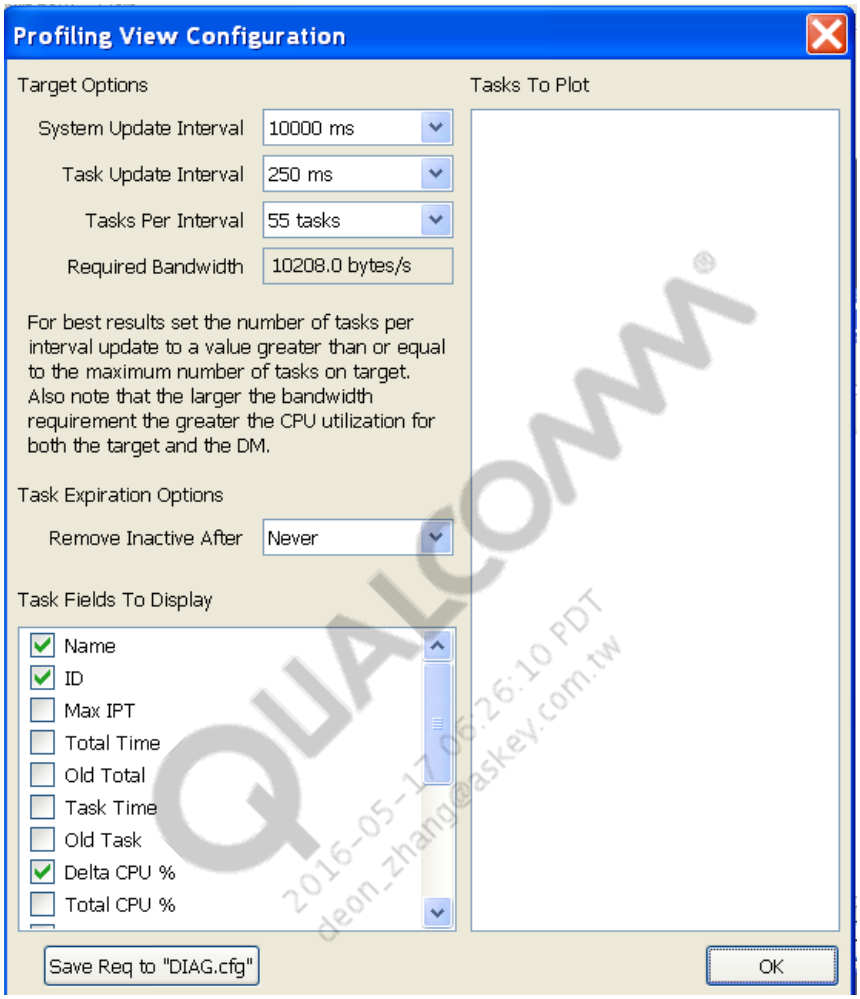
1. Launch QXDM Pro.
2. Open either the MPU or APU task profiling display. From the View drop-down, go to Common→Task Profiling→Task Profiling MPU or Task Profiling APU and click **Config**.



QUALCOMM
2016-05-17 06:26:10 PDT
deon_zhang@askey.com.tw

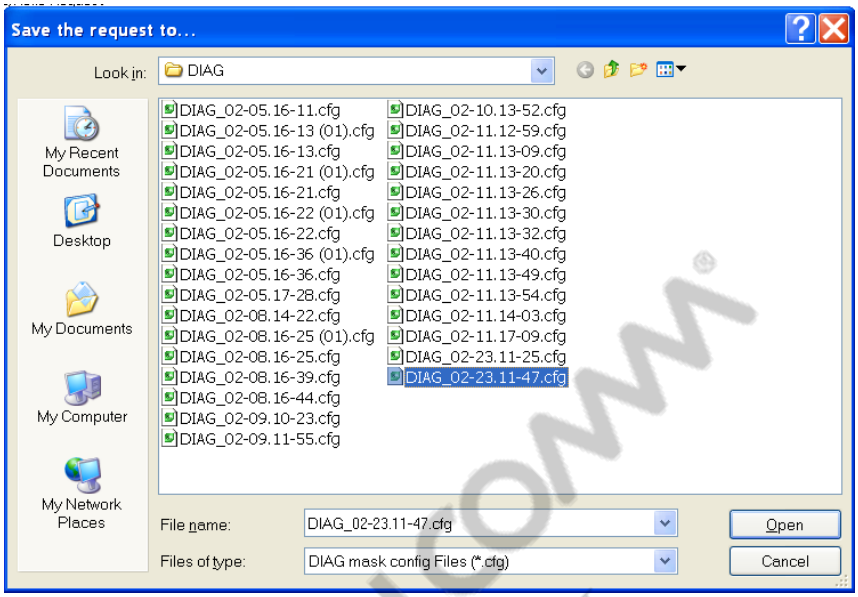
MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3. Select Intervals and click **Save Req to "DIAG.cfg"**.

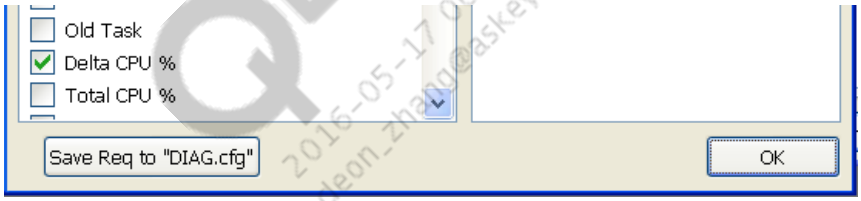


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4. When the “Save the request to...” dialog appears, select a diag.cfg file and click **Open**:



5. Click **OK** in the Config dialog. The Task Profile subsystem requests are automatically saved in the selected Diag configuration file.



NOTE: If there is no diag.cfg file in the folder, see Section 4.4.1 for instructions on creating a .cfg file.

4.6.5 Reading the CFG file

Open the CFG file in a hex editor. The format of the file is the exact same as in the ICD.

For HDLC-encoded config

Refer to 80-V1294-1, Section 2.1 Async-HDLC frame format.

```
Field            Length (in bits)   Description
-------          ----------------   -----------
Information      Variable           ICD Packet or Message
Frame Check      16                 CRC-CCITT standard 16-bit CRC
Ending Flag      8                  Ending character "0x7E"
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Here is a part of hexdump of a DIAG.CFG file saved from QXDM's Filtered View dialog:

```
1D 1C 3B 7E 00 78 F0 7E 7C 93 49 7E 1C 95 2A 7E 0C 14 3A 7E 63 E5 A1 7E 4B
0F 00 00 BB 60 7E 4B 09 00 00 62 B6 7E 4B 08 00 00 BE EC 7E 4B 08 01 00 66
F5 7E 4B 04 00 00 1D 49 7E 4B 04 0F 00 D5 CA 7E
```

Commands are separated using “7e” delimiters. The two bytes just before “7e” is CRC. The rest of the bytes in between the two “7e” is data.

```
   4b 0f 0a 00 00 // Info
   bb 60          // CRC
   7e             // DELIMITER
```

Using the end flags to split the messages, we can decode the messages according to the ICDs:

```
1d 1c 3b 7e  // 0x1D == Time Stamp Request (80-V1294-1)
00 78 f0 7e  // 0x00 == Version Request (80-V1294-1)
7c 93 49 7e  // 0x7C == Extended Build ID Request (80-V1294-1)
1c 95 2a 7e  // 0x1C == DIAG Version Request (80-V1294-1)
0c 14 3a 7e  // 0x0c == Status Request (80-V1294-1)
63 e5 a1 7e  // 0x6C == Phone State Request (80-V1294-1)
4b 0f 1a 00 00 bb 60 7e // 0x4B 0x0F 0x001A == Call Manager Subsystem Sys
Select (80-V1294-7)
4B 09 00 00 62 B6 7E // 0x4B 0x09 0x0000 == UMTS Subsystem Version Request
(80-V2708-1
4b 08 00 00 be ec 7e    // 0x4B 0x08 0x0000 == GSM Subsystem Version
Request (80-V5295-1)
4b 08 01 00 66 f5 7e    // 0x4B 0x0F 0x0001 == GSM Subsystem Status Request
(80-V5295-1)
4b 04 00 00 1d 49 7e    // 0x4B 0x04 0x0000 == WCDMA Subsystem Version
Request (80-V2708-1)
4b 04 0f 00 d5 ca 7e    // 0x4B 0x04 0x000F == WCDMA Subsystem Additional
Status Request (80-V2708-1)
```

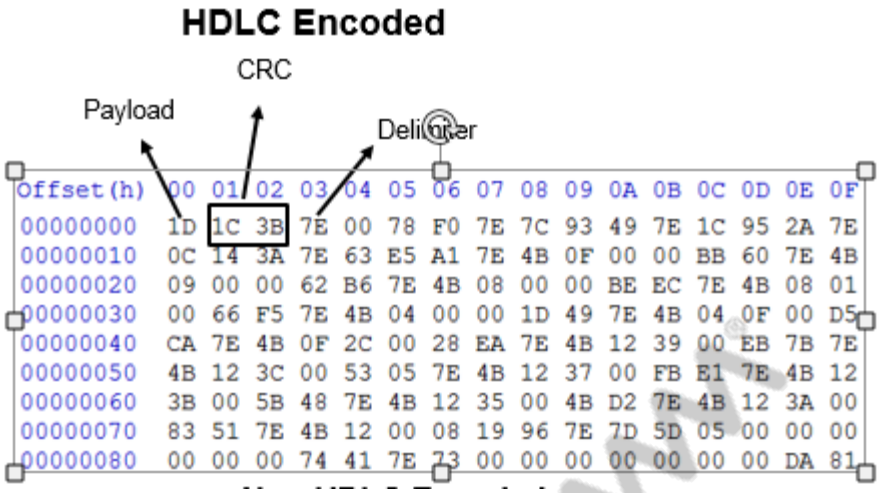
The subsystem CMD_CODE(75) is defined in Subsystem Dispatch (75 / 0x4B) of 80-V1294-1.

```
   4b    // CMD_CODE 75 DIAG command identifier
   0f    // Subsystem ID for Call Manager commands, refer to 80-V1297-7
```

Once the subsystem ID is known, use that subsystem ICD to decode the subsystem message.

```
   1a 00 // Subsystem CMD_CODE 0x001a (26d) for System Selection Preference
of Current Subscription
   00    // 0 = No active subscription ID
```

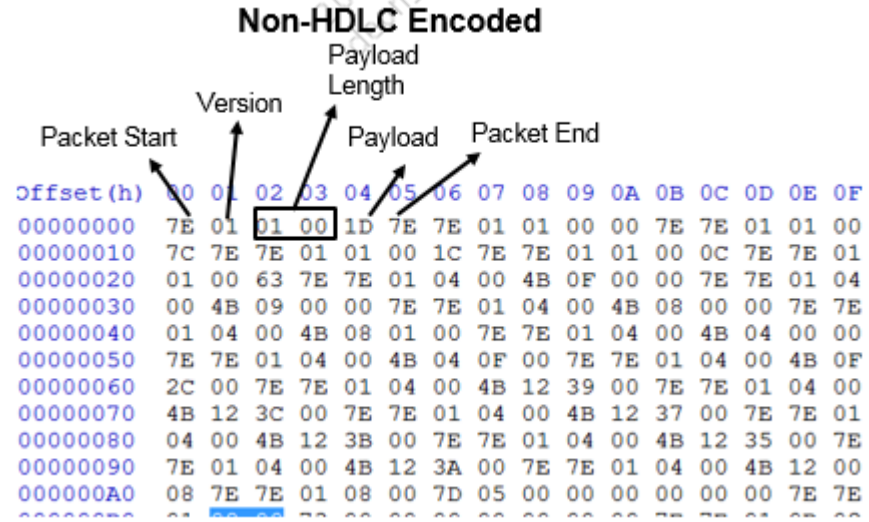

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION



For non-HDLC encoded config

When .cfg2 is used, which is non-HDLC encoded, the format of the file is as follows:

Field	Length (in bits)	Description
Start	8	This is the start of packet, 0x7E
Version	8	Version
Payload Length	16	Payload length
Payload	Variable	This is the actual data
Packet End	8	Ending character "0x7E"



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4.7 Qshrink4 support

NOTE: This section was added to this document revision.

With Qshrink4, the Qshrink database is generated in each build, and either the tools query and download the database or the user must provide the database file manually.

Each on-device logging run creates the directory <currenttimestamp>, in which qmdl files are saved as diag_log_<currenttimestamp>.qmdl. A file can be created under the same directory with the same timestamp value and saved as diag_qsr4_guid_list_<timestamp>.txt. QCAT can parse the list of GUIDs in this text file and read the database files for the current on-device logging run. All QMDL files generated under this directory require the database files associated with the GUIDs in the text file to decode the F3 messages.

The database file can be retrieved as follows:

- The on-device logging application using the DIAG LSM library sends command requests to get the GUIDs of all available peripherals
- If the GUID response is successful, it sends commands to retrieve the database files from DIAG peripherals, just like QXDM.

NOTE: The request is sent only if the database was not retrieved previously for the GUID with which it was associated.

Figure 4-4 shows the files that are needed for successful Qshrink decoding.

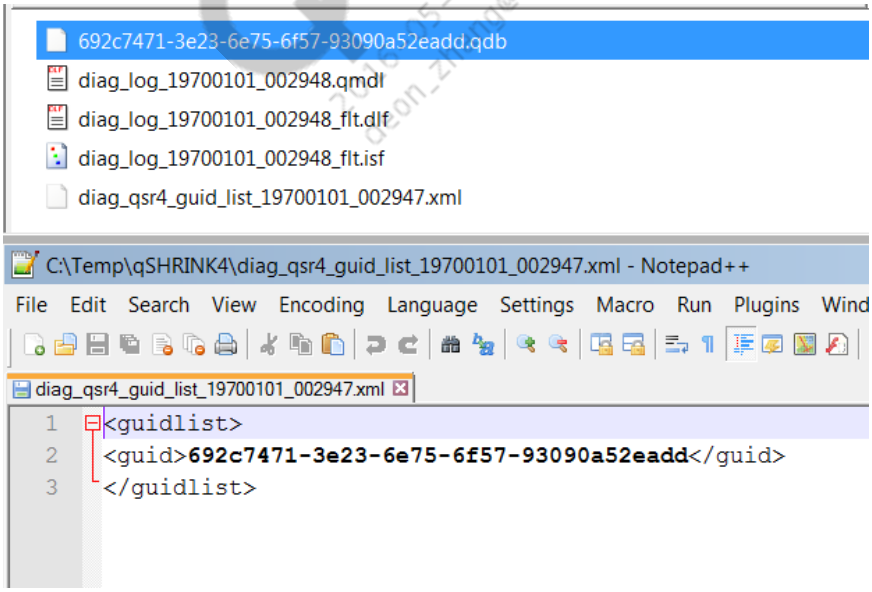


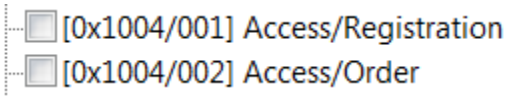
Figure 4-4 Files needed for successful Qshrink decoding.

NOTE: The GUID in the .xml file must be the same as the .qdb file GUID to ensure that the database matches.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4.8 OTA logs support in the CFG file

OTA logs are the log types that have type information along with log codes.



1004 == OTA LOG CODE

001/002 == OTA LOG TYPE

QXDM is able to capture the OTA logs properly when enabled since it enables the main log codes and receives all the log types. It then internally filters the enabled log types and displays only the enabled log types.

In the case of CFG, the Diag command that carries the OTA does not support OTA types, it only supports LOG_CODEs by themselves. So it seems that some of the OTA logs are missing.

In the case of a CFG file, this can be seen if an OTA Log and Type is selected that also has a valid OTA LOG_CODE alone. For example, 0x1078/002/084 is valid as well as 0x1078 alone. If a .CFG with 0x1078/002/084 is saved and then loaded, the checkbox for 0x1078 alone will be marked, and when **OK** is pressed, the generated Logging Config Request Command (115) will still contain LOG_CODE 0x1078.

If an OTA log without a valid root is selected, the .CFG file will still contain the LOG_CODE, but it is less obvious in the QXDM Config dialog. For example, loading a .CFG file with 0x10D8/015 will NOT show the OTA messages that have a subtype in the QXDM database enabled at all, but if **OK** is pressed after loading the .CFG file into the dialog, it is right-clicked and CONFIG is chosen again (yes, a second time), and “Only Show Supported Items” is right-clicked and unchecked, then LOG_CODE 0x10D8 itself will show as a “Discovered Item” and it is checked. This means it is saved in the .CFG file, although the type selections are not present (and they do not need to be for sending a Logging Configuration Message to the device).

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Figure 4-5 illustrates the Item List Config window in QXDM Pro.

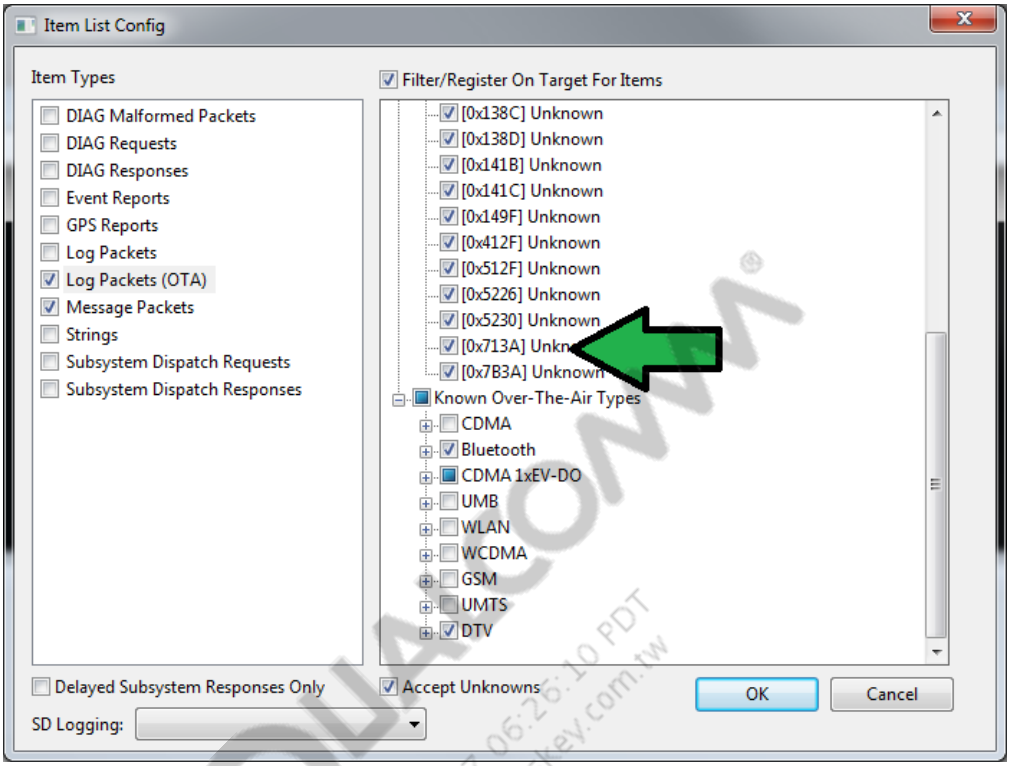


Figure 4-5 Item List Config window in QXDM Pro

Converting the CFG file to a DMC

Currently there are no solutions to convert a CFG file to DMC or vice-versa, and there are no plans to develop one. However, if the intended configuration is in the Filtered View, it is possible to load a DMC in QXDM, open the Filtered View's config dialog by right-clicking and selecting "Config...", and then saving it to a CFG file by selecting either "Save Diag masks for Stream 1" or "Save Diag masks for Stream 2" in the SD Logging drop-down menu. Stream 1 is for single SIM, and Stream 2 is for dual-SIM. Note that the CFG file will not be saved until after the **OK** button is pressed in the Item List Config. If the DMC file does not have a Filtered View already saved and configured, this method will not work. Saving a CFG file is based on Filtered View only and not based on the aggregate settings of open views in QXDM.

4.9 Low power modes

In the default design, non-HLOS processors frequently drain packets to the communication layer and hence to the applications processor. This results in too many wake-ups for the applications processor, causing high current drain for the device. Non-HLOS processors have the ability to buffer data for longer durations, reducing the current drain on the device.

When there is more than one client in the system, a client opting for real-time data is not suppressed by a client opting for NRT data. The clients vote for their intended mode and the applications processor Diag makes a decision on the final mode, considering all the active Diag clients (DCI and different logging modes) and the state of the USB connection.

The APIs for voting for RT and NRT/Buffering modes are provided for Diag and DCI clients.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4.9.1 NRT mode

In NRT mode, there are very few applications processor wake-ups. If there are no other active processes running, it might go to power collapse, and hence the Diag driver would drop packets to be sent to user space.

To prevent the system from power collapse, the Diag driver notifies the Power Management core that a wake-up event is being processed by acquiring a wake-up source object. The Diag driver acquires the wake-up source in the SMD-notify context and holds the wake-up source until the data is copied to the user space library. Once the data reaches the user space, the Diag library signals the client to release the acquired wake-up object. It is then the client's responsibility to acquire and release a new wake-up source(s) within their application to process the data.

When NRT or Buffering mode is selected on a processor, the Diag drain timer is set to a larger value along with the buffer threshold adjusted to hold more data before forwarding it to the applications processor. When a USB is connected, power consumption is already high and there is no significant savings in power. APIs are provided for Diag and DCI clients to achieve these power savings, giving similar results to running on-device logging with a –b (buffering) argument:

```
int diag_vote_md_real_time(int real_time);
```

This function is used for the Diag client. It accepts parameters MODE_REALTIME or MODE_NONREALTIME and returns 0 for success.

4.9.2 Real-time mode

This function is used for the Diag client. It accepts a pointer to store the real-time value and returns 0 for success.

```
void diag_get_real_time_status(int *real_time);
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4.10 Initializing diag_mdlog at boot time

The diag_mdlog on-device logging application can also be started at boot time, without ADB shell command intervention. To start diag_mdlog at boot time, the following code changes must be made and the code recompiled.

4.10.1 Required code changes for initializing diag_mdlog at boot time

If this document is being viewed using a color monitor, or if this document is printed on a color printer, **red boldface** indicates code that is to be **added**.

MSM8996

File: /system/core/rootdir/init.rc

At the end of the file, add the code shown below.

```
service diag_mdlog /system/bin/diag_mdlog –f /data/Diag.cfg –o
/data/diag_logs/test1 &
   class main
   user shell
   group system diag sdcard_rw sdcard_r media_rw
   oneshot    #for running only once
```

NOTE: The SD card is accessible from here, but not the data folder, since it requires more privileges.

MSM8960

File: /system/core/rootdir/init.rc

At the end of the file, add the code shown below.

```
service diag_mdlog /system/bin/diag_mdlog –f /data/Diag.cfg –o
/data/diag_logs/test1 &
   class main
   user system
   group log system sdcard_rw
   oneshot    #for running only once
```

At least three levels of directories are needed for the -o argument. The first two directories should already be present and the third is created by the application.

APQ8064+MDM9x15

Make the same changes as on MSM8960 above.

If logs from MDM are missing, try the following change in diag_mdlog.c:

```
/* Main Function. This initializes Diag_LSM, sets up On Devic ec Logging,
then exits. */
int (main(int argc, char *argv[])
{
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
boolean bInit_Success = FALSE;
. . .

/* Read mask file to tell On Device Logging what you are interested in */
sleep(10);
diag_read_mask_file();
. . .
}
```

MDM9x15

Add a diag_mdlog bash script, such as the following.

```
#! /bin/sh

set -e

case "$1" in
  start)
    echo -n "Starting diag_mdlog: "
    start-stop-daemon -S -b -a /usr/bin/diag_mdlog -- -f /data/all_logs.cfg
-o /data/diag_logs
    echo "done"
    ;;
  stop)
    echo -n "Stopping diag_mdlog: "
    start-stop-daemon -K -n diag_mdlog
    echo "done"
    ;;
  restart)
    $0 stop
    $0 start
    ;;
  *)
    echo "Usage diag_mdlog { start | stop | restart }" >&2
    exit 1
    ;;
esac

exit 0
```

Linux services can be started, stopped, and reloaded with the use of scripts stocked in /etc/init.d/. However, during startup or when changing run level, those scripts are searched as follows in /etc/rcX.d/, where X is the run level number.

1. Push the above script in /etc/init.d/.
2. Add symlinks to this script in /etc/rc?.d/.

   ```
   #update-rc.d diag_mdlog defaults 100
   ```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

5 Buffering mode

The peripheral buffering feature adds the capability of buffering the Diag data on the peripherals to reduce applications wakeups. The peripheral buffering feature supports Threshold Buffering mode and Circular Buffering mode

5.1 Buffering mode functions

The subsequent subsections contain functions that can be used to configure the peripherals in various modes.

5.1.1 diag_configure_peripheral_buffering_tx_mode

Allows the Diag client AP infrastructure to configure the peripheral Diag's Tx mode to Streaming, Circular, or Threshold Buffering mode, and sets high and low watermark threshold limits.

Streaming mode is a default Tx mode for a QTI peripheral Diag.

```
int diag_configure_peripheral_buffering_tx_mode(
uint8 peripheral_id,
uint8 tx_mode ,
uint8 lowwmqval,
uint8 highwmqval)
```

Input parameters

peripheral_id – Peripheral ID to identify a peripheral that is to receive the request:

- 0 – Modem
- 1 – ADSP
- 2 – WCNSS
- 3 – SLPI

tx_mode – Mode to which to set a peripheral:

- 0 – STREAMING_MODE
- 1 – THRESHOLD_BUFFERING_MODE
- 2 – CIRCULAR_BUFFERING_MODE

NOTE: Switching to Threshold or Circular Buffering mode puts the peripheral Diag to NRT mode. Switching to Streaming mode puts the peripheral to RT mode. In NRT mode, Diag sleep voting is disabled and drain timers and thresholds are increased to delay the draining of data to reduce applications wakeups.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

HighWMValue – High watermark value in percentage with respect to the peripheral buffer size. This value can range from 0 to 100. 0 keeps the current values on the peripherals. This value cannot be lower than or equal to LowWMValue.

LowWMValue – Low watermark value in percentage with respect to the peripheral buffer size. This value can range from 0 to100. 0 keeps the current values on the peripherals.This value cannot be higher than or equal to HighWMValue.

Returns

Success – A control packet was sent to the peripheral diag to change the Tx mode

Errorcode – Otherwise

Sample usage

diag_configure_peripheral_buffering_tx_mode(1,1,10,80) – This configures the ADSP to Threshold Buffering mode with a low water mark value of 10 percent of the peripheral buffer size and a high water mark value of 80 percent of the peripheral buffer size .

diag_configure_peripheral_buffering_tx_mode(1,2,10,80) – This configures the ADSP to Circular Buffering mode with a low water mark value of 10 percent of the peripheral buffer size and a high water mark value of 80 percent of the peripheral buffer size .

5.1.2 diag_peripheral_buffering_drain_immediately

Allows the Diag client AP infrastructure to request the peripheral to drain its Tx buffering pool immediately. If peripheral Diag receives this request in

- Streaming mode – No action is taken because Diag is already streaming
- Threshold or Circular Buffering modes – Diag drains its Tx buffering pool until the low watermark threshold is reached, and then resumes buffering in the Tx mode to which it was set.

int diag_peripheral_buffering_drain_immediately(uint8 peripheral_id)

peripheral_id – Peripheral ID to identify a peripheral that is to receive the request:

- 0 – Modem
- 1 – ADSP
- 2 – WCNSS
- 3 – SLPI

Returns

Success – A control packet was sent to a peripheral Diag to request it to drain its Tx buffering pool immediately.

Errorcode – Otherwise

Sample Usage

diag_peripheral_buffering_drain_immediately(0); – Modem

diag_peripheral_buffering_drain_immediately(1); – ADSP

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Prerequisite for API usage

APIs are available in diag_lsm.h. To use these Diag APIs, the user application must first initialize the Diag services. To achieve this, ensure that the client is calling Diag_LSM_Init() before using the Diag APIs. The client must call Diag_LSM_Deinit() on the application's exit.

5.2 Sample buffering application

NOTE: This section was added to this document revision.

A sample buffering application is available for testing the buffering modes. See the examples below on how to use the command.

Diag_Buffering_Test application

Include the application in the build and run it from the adb shell. Once the modes change takes place, depending on if QXDM or on-device logging is being used, the LOGS/F3/EVENTS are saved accordingly.

Enable threshold mode:

aDSP: `diag_buffering_test -t -p 1 -m 1 -l 20 -h 70`

SLPI: `diag_buffering_test -t -p 3 -m 1 -l 20 -h 70`

Enable Circular mode:

aDSP: `diag_buffering_test -t -p 1 -m 2 -l 20 -h 70`

SLPI: `diag_buffering_test -t -p 3 -m 2 -l 20 -h 70`

Enable Streaming mode:

aDSP: `diag_buffering_test -t -p 1 -m 0 -l 20 -h 70`

SLPI: `diag_buffering_test -t -p 3 -m 0 -l 20 -h 70`

Drain:

aDSP: `diag_buffering_test -d -p 1`

SLPI: `diag_buffering_test -d -p 3`

QXDM commands

These are the QXDM commands to use to change buffering modes and drain.

Enable Threshold mode:

aDSP: `send_data 75 18 12 14 0 1 1`

SLPI: `send_data 75 18 0x19 0x1A 0 1 1`

Enable Circular mode:

aDSP: `send_data 75 18 12 14 0 1 2`

SLPI: `send_data 75 18 0x19 0x1A 0 1 2`

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Enable Streaming mode:

aDSP: `send_data 75 18 12 14 0 1 0`

SLPI: `send_data 75 18 0x19 0x1A 0 1 0`

Drain:

aDSP: `send_data 75 18 12 14 6 1`

SLPI: `send_data 75 18 0x19 0x1A 6 1`

QUALCOMM®
2016-05-17 06:26:10 PDT
deon_zhang@askey.com.tw

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6 Socket logging

The socket logging feature provides diagnostic traffic over Ethernet and Wi-Fi. If the device has Ethernet/Wi-Fi enabled, it can be connected to QPST and QXDM Pro to fetch Diag packets similar to other logging methods. The architecture for socket logging is shown in Figure 6-1.

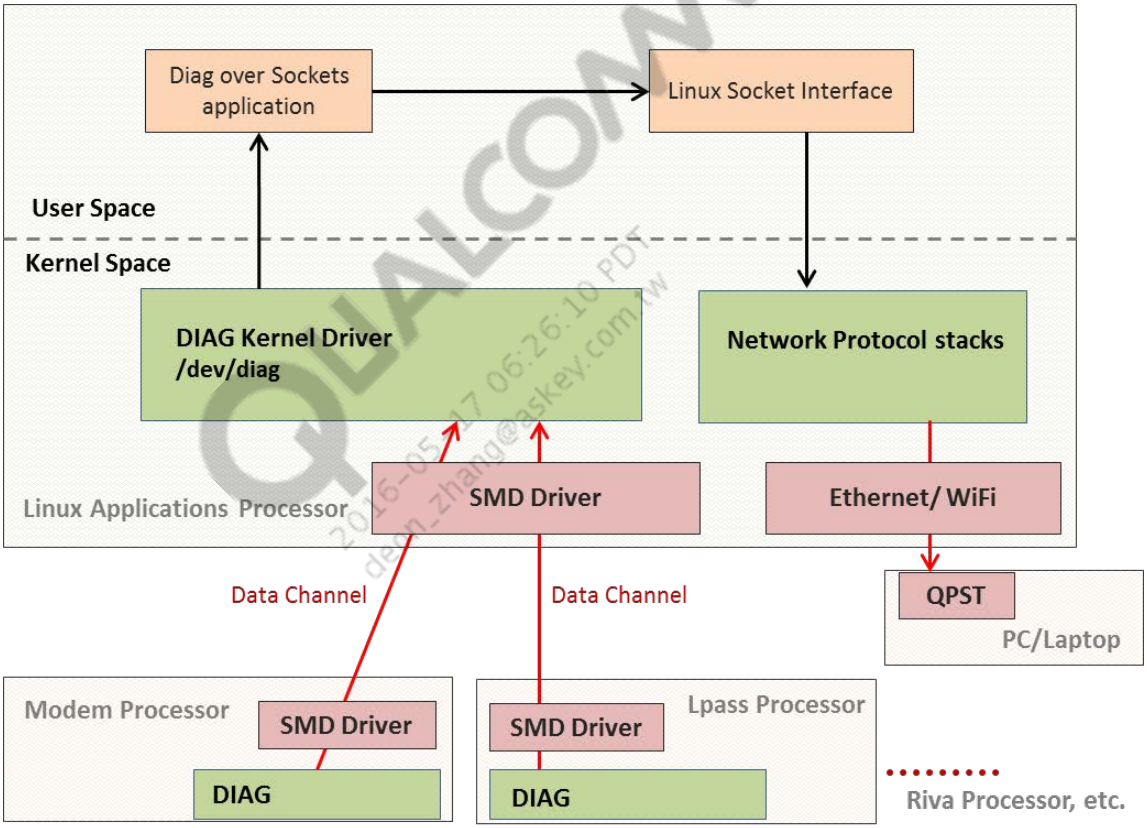


Figure 6-1 Architecture for socket logging

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6.1 Initial setup

The Android device must be set up to Rx and Tx data to/from the network. This behavior can be verified by using the ping command with the PC and Android device IP addresses as described in Section 6.1.1 and Section 6.1.2.

6.1.1 Ping device from PC

Use the netcfg command to get the Android device's Wi-Fi/Ethernet device status and IP address.




```
C:\Users\johndoe>ping 192.168.1.7
Pinging 192.168.1.7 with 32 bytes of data:
Reply from 10.73.82.37: bytes=32 time=76ms TTL=55
Reply from 10.73.82.37: bytes=32 time=100ms TTL=55
Reply from 10.73.82.37: bytes=32 time=122ms TTL=55
…
```

6.1.2 Ping PC from device

Use the ipconfig command to get the Windows machine Wi-Fi/Ethernet device IP address.

```
root@android:/ # ping 10.73.46.115
ping 10.73.46.115
PING 10.73.46.115 (10.73.46.115) 56(84) bytes of data.
64 bytes from 10.73.46.115: icmp_seq=1 ttl=63 time=84.9 ms
64 bytes from 10.73.46.115: icmp_seq=2 ttl=63 time=97.8 ms
64 bytes from 10.73.46.115: icmp_seq=3 ttl=63 time=108 ms
…
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6.2 Application usage

6.2.1 Running the diag_socket_log application

1. Plug in the device via USB.
2. Open an ADB command shell as root.
3. Run the following command:

```
diag_socket_log –a <IP address of PC> -p <port number> -r <number of
retries to connect> &
```

Table 6-1 lists the diag_socket_log command arguments.

Table 6-1 diag_socket_log command arguments

Argument	Function
–a	IP address
–p	Port number; defaults to 2500
–r	Number of retries for connect; defaults to 10000, with a 2 sec wait between tries
–k	Kill existing instance of diag_socket_log
–h	Usage help; prints a list of these arguments

NOTE: -a is required; -p and -r are optional.

Note that appending an & to the command causes the application to run in the background, which allows the application to keep running when the USB cable is disconnected.

6.2.2 Stopping the diag_socket_log application

1. Determine the process ID of diag_socket_log.

```
# ps
```

2. Kill the process with the following command:

```
# kill -9 <diag_socket_log process id>
```

When the diag_socket_log application exits, Diag functionality returns to the default USB configuration.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6.3 QPST configuration

If QPST/QXDM Pro/Qualcomm Tools are being used, QPST must be set up to accept client connections. In the QPST configuration:

1. Click the **IP Server** tab.
2. On the IP Server screen, select the **Accept client connections** checkbox. This screen also displays the IP address of the machine in use.

When running Diag over sockets, the QPST Configuration Ports screen should display the following:

- The USB port currently connected to should change to “No Phone.”
- A port with a number in the 30000 range is created. This is the port that QPST uses for the network connection. The display should change from “No Phone” to the string associated with the device. If QXDM Pro is being used, connect to this port.
- After stopping Diag over sockets, the 30000 port should change back to “No Phone” and the USB port that was used should change from “No Phone” back to the string associated with the device.

QUALCOMM®
2016-05-17 06:26:10 PDT
deon_zhang@askey.com.tw

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

7 UART logging

UART logging refers to getting Diag packets over a UART device. The device can be connected to QPST/QXDM Pro via a UART, and traffic can be fetched similar to other logging methods.

7.1 Application usage

The diag_uart_log application routes Diag traffic to and from a UART port. The application reads Diag packets from a Diag driver and writes the packets to the UART port. It reads packets from QPST/QXDM Pro over the UART and sends the packets to the Diag driver.

To use Diag over UART, neither the kernel nor any application should use or perform an operation on the UART port of the device.

7.1.1 Running the diag_uart_log application

1. Ensure that the code changes described in Section 7.2 have been done and that the code is recompiled.
2. Open an ADB command shell.
3. Run the following command:

```
diag_uart_log –d <device node> -o <Out baud rate> -i <In baud rate>
```

Table 7-1 lists the diag_uart_log command arguments.

Table 7-1 diag_uart_log command arguments

Argument	Function
-d	UART device node; default is /dev/ttyHSL0
-o	Output baud rate; default is 115200
-i	Input baud rate; default is 115200
-p	Logging processor type; kills previous instance of application running on previous processor type and starts routing Diag traffic from the given processor type to QXDM Pro; acceptable values are: ▪ MSM (default) – Both MSM and APQ targets take this value ▪ MDM ▪ QSC
-h	Usage help; prints a list of these arguments

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

7.1.2 Fused target enhancements

The following enhancements were made for QTI fused architecture targets:

- Separation of traffic from local (MSM) and remote (MDM) processors
- Runtime selection of processor for Diag traffic; that is, a user can select a processor for diag→diag traffic and the processor can be switched at runtime as indicated in Table 7-1.

7.1.3 Examples

- The user can start the application with MSM, MDM, or QSC traffic as follows:

  ```
  /system/bin/diag_uart_log -p MSM  <Starting UART log for MSM traffic>
  ```

  This instance of the UART application kills the previous instance of the UART application running with a processor type as MDM/QSC and starts routing Diag traffic from MSM→QXDM Pro.

- To switch to the remote (MDM, MDM2, etc.) processor, start the UART application in another shell with the processor option as MDM:

  ```
  /system/bin/diag_uart_log -p MDM  <Starting UART log for MDM traffic>
  ```

  This instance of the UART application kills the previous instance of the application running with an MSM/QSC processor type and starts routing Diag traffic from MDM→QXDM Pro.

- To switch to the remote QSC processor, start the UART application in another shell with QSC as a processor option:

  ```
  /system/bin/diag_uart_log -p QSC  <Starting UART log for QSC traffic>
  ```

  This instance of the UART application kills the previous instance of the application running with an MSM/MDM processor type and starts routing Diag traffic from QSC→QXDM Pro.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

7.2 Required code changes for enabling UART logging

The applications processor serves as the master in the Diag central routing architecture. All slave processors (modem, ADSP, and WCNSS) forward their Diag traffic to the applications processor, which then pushes the traffic over the USB or UART channels.

The modem processor can have a standalone Diag configuration as well; that is, only modem Diag traffic is pushed out of the device. This can be done by configuring a UART device on the modem to support Diag services. Hence, a UART device can be configured either on the applications processor or modem processor for Diag services.

Make sure that the GSBI or BLSP device pins being used on the configured processor are not being used on any other processor or by some client other than Diag services.

7.2.1 TN builds – MDM9x15, MDM9x25, MDM9x35

All of the following configurations set the UART1 device for Diag services. Check the UART configuration files for the physical GSBI (custsio_9x15.h) or BLSP (uart/build/SConscript) device mapped to UART 1. Diag uses low speed UART with a baud rate of 115200 bps.

7.2.1.1 Configuring a UART on the applications processor

If this document is being viewed using a color monitor, or if this document is printed on a color printer, **red boldface** indicates code that is to be **added**, and ~~blue strikethrough~~ indicates code that is to be replaced or removed (if present).

Changes required in apps build

In cust<build_flavor>.h (\apps_proc\build\ms):

```
define FEATURE_FIRST_UART

define FEATURE_DMOV_HS_UART_ON_APPS

define FEATURE_RDEVMAP_DIAG_DEFAULT_TO_USB
define RDM_USB_SER1_INIT_SRVC    RDM_DIAG_SRVC
define RDM_UART1_INIT_SRVC  RDM_DIAG_SRVC

define FEATURE_DIAG_MP_MASTER_APPS
define FEATURE_DIAG_MP_MASTER_MODEM
define FEATURE_DIAG_MP_MODEM_ONLY

define DIAG_RUNTIME_DEVMAP
```

Changes required in modem build

None

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

7.2.1.2 Configuring a UART on the modem processor

Changes required in the applications build

In cust<build_flavor>.h (\apps_proc\build\ms):

```
define FEATURE_RDEVMAP_DIAG_DEFAULT_TO_USB
define RDM_USB_SER1_INIT_SRVC    RDM_DIAG_SRVC

define FEATURE_DIAG_MP_MASTER_APPS
define FEATURE_DIAG_MP_MASTER_MODEM

define FEATURE_DIAG_MP_MODEM_ONLY
define DIAG_RUNTIME_DEVMAP
```

Changes required in the modem build

In cust<build_flavor>.h (\modem_proc\build\ms):

```
define FEATURE_FIRST_UART

define FEATURE_DIAG_MODEM_ONLY

define RDM_UART1_INIT_SRVC  RDM_DIAG_SRVC
define DIAG_RUNTIME_DEVMAP
```

7.2.2 LE/LA builds

In the reference design, the UART might be getting used by the applications boot loader (little kernel) or the kernel for debugging over a serial console. Disable this use of the UART before using it for Diag logging.

7.2.2.1 Disable the console in LK (LE)

In bootable/bootloader/lk/project/mdm9615.mk, comment out:

```
#DEFINES += WITH_DEBUG_UART=1
```

In bootable/bootloader/lk/platform/mdm9x15/platform.c, comment out:

```
 //uart_dm_init(gsbi_id, GSBI_BASE(gsbi_id),
GSBI_UART_DM_BASE(gsbi_id));
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

7.2.2.2 Disable the console in the kernel (LE)

recipes/sysvinit/sysvinit-inittab_2.88dsf.bb

Reviewed & nex

⇐Commit Message ⇧Up to change

Patch Set Base 1 | Patch Set 1

... skipped 22 common lines ...

```
do_install() {
      install -d ${D}${sysconfdir}
   install -m 0644 ${WORKDIR}/inittab ${D}${sysconfdir}/inittab
   if [ ! -z "${SERIAL_CONSOLE}" ]; then
       echo "S:2345:respawn:${base_sbindir}/getty -L ttyHSL0 ${SERIAL_CONSOLE}" >> ${D}${sysconfdir}/inittab
   fi

   idx=0
   tmp="${SERIAL_CONSOLES}"
```

```
do_install() {
      install -d ${D}${sysconfdir}
   install -m 0644 ${WORKDIR}/inittab ${D}${sysconfdir}/inittab
   #if [ ! -z "${SERIAL_CONSOLE}" ]; then
   #    echo "S:2345:respawn:${base_sbindir}/getty -L ttyHSL0 ${SERIAL_CONSOLE}" >> ${D}${sysconfdir}/inittab
   #fi

   idx=0
   tmp="${SERIAL_CONSOLES}"
```

recipes/images/9615-cdp-bootimg.inc

Reviewed & next

⇐Commit Message ⇧Up to change

Patch Set Base 1 2 | Patch Set 1 2

... skipped 21 common lines ...

```
      ${STAGING_DIR_NATIVE}/usr/bin/mkbootimg --kernel ${STAGING_DIR_TARGET}/boot/Image-${ver} \
          --ramdisk /dev/null \
          --cmdline "noinitrd root=/dev/mtdblock14 rw rootfstype=yaffs2 console=ttyHSL0,115200,n8 androidboot.hardware=qcom" \
          --base 0x40800000 \
          --ramdisk_offset $kernelsize \
          --output ${DEPLOY_DIR_IMAGE}/boot-oe-msm9615.img
```

```
      ${STAGING_DIR_NATIVE}/usr/bin/mkbootimg --kernel ${STAGING_DIR_TARGET}/boot/Image-${ver} \
          --ramdisk /dev/null \
          --cmdline "noinitrd root=/dev/mtdblock14 rw rootfstype=yaffs2 androidboot.hardware=qcom" \
          --base 0x40800000 \
          --ramdisk_offset $kernelsize \
          --output ${DEPLOY_DIR_IMAGE}/boot-oe-msm9615.img
```

In kernel/arch/arm/configs/msm9615_defconfig

```
CONFIG_SERIAL_MSM_HSL_CONSOLE=n
```

In kernel/arch/arm/mach-msm/clock-9615.c (make this change only if using the UART on another processor)

```
// CLK_LOOKUP("core_clk", gsbi4_uart_clk.c, "msm_serial_hsl.0"),
// CLK_LOOKUP("iface_clk", gsbi4_p_clk.c, "msm_serial_hsl.0"),
```

QUALCOMM® 2016-05-17 08:26:10 PDT deon_zhang@asky.com.tw

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

7.2.2.3 Configuring a UART on APSS

Changes required in the APSS build

By default, diag_uart_log uses /dev/ttyHSL0. If it is being used by the console, disable the console as explained above.

Changes required in the MPSS build

None

7.2.2.4 Configuring a UART on MPSS

Changes required in the APSS build

If the UART device is being used by APSS, disable the client's usage.

Changes required in the MPSS build

In cust<build_flavor>.h (\modem_proc\build\ms):

```
define FEATURE_FIRST_UART

define FEATURE_DIAG_MODEM_ONLY

define RDM_UART1_INIT_SRVC  RDM_DIAG_SRVC
define DIAG_RUNTIME_DEVMAP
```

7.3 Tools tested

The UART configurations have been tested to work with the following tools:

- QXDM Pro
- QPST NV Backup/Restore
- EFS Explorer
- QSPR

QUALCOMM 2015-09-17 06:26:10 PDT deon_zhang@askey.com.tw

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

7.4 Limitations

Due to UART throughput, QPST may lose the port connection in heavy traffic. QPST only waits 400 ms for any command/response, including polling. Because of a throughput bottleneck, the response is delayed and reaches QPST after 2 to 3 sec.

If a port disconnection issue is observed during Diag over UART logging:

- Update QPST register settings
- Configure QPST and QXDM Pro for the new settings

The default QPST port parameter settings are:

- Polling timeout duration – 400 ms
- Polling timer interval – 600 ms
- Polling maximum retries – 3

When experiencing a port disconnect, these values can be increased:

- Polling timeout duration – 3000 ms
- Polling timer interval – 3000 ms
- Polling maximum retries – 5

If a port disconnection issue is observed during Diag over UART logging:

1. Ensure that both QPST and QXDM Pro are closed.
2. Two options are available for making the required registry changes to resolve the port disconnection issue.
   - Create and run a Windows Registry file (.reg) with the following contents:

```
Windows Registry Editor Version 5.00

[HKEY_LOCAL_MACHINE\Software\Qualcomm\QPST\PARAMS\PORT_SERVER]
"polling_timeout_ms"=dword:00000bb8
"polling_timer_interval_ms"=dword:00000bb8
"polling_max_tries"=dword:00000005

[HKEY_CURRENT_USER\Software\Qualcomm\QPST\PARAMS\PORT_SERVER]
"polling_timeout_ms"=dword:00000bb8
"polling_timer_interval_ms"=dword:00000bb8
"polling_max_tries"=dword:00000005
```

   or

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

- Manually change the registry setting for QPST; that is, the timeout, polling interval, and number of retries.
  - Go to regedit, to the path [HKEY_LOCAL_MACHINE\Software\Qualcomm\QPST\PARAMS\PORT_SERVER], and update the configuration values.
    - polling_timeout_ms – 3000 ms
    - polling_timer_interval_ms – 3000 ms
    - polling_max_tries – 5
  - Go to regedit, to the path [HKEY_CURRENT_USER\Software\Qualcomm\QPST\PARAMS\PORT_SERVER], and update the configuration values.
    - polling_timeout_ms – 3000 ms
    - polling_timer_interval_ms – 3000 ms
    - polling_max_tries – 5

To configure QPST and QXDM Pro:

1. Start QPST.
2. Right-click the COM port in QPST, select the baud rate option, and update the baud rate to 230400.
3. If a target was previously connected over USB and connected to QXDM Pro, disable the logs. This ensures that if many masks were enabled with USB, after the UART application restarts, data is pumped to the UART, disabling the unwanted masks.
4. Start Diag over the UART application with the following command:

   ```
   ./diag_uart_log –p <proc name> -i 230400 –o 230400
   ```

5. Start QXDM Pro and set the mask per the requirement. However, this should be kept minimal and use only required logs. Wait for the target to be enumerated in QPST over the UART port. This may take additional time because the QPST parameters were changed.
6. Connect QXDM Pro to the device over the COM port.
7. In this configuration, runtime switching does not work because QPST needs time for the port to be re-enumerated due to the change in parameters. To change the logging processor:
   a. Disconnect QXDM Pro.
   b. Kill the diag_over_uart app. Wait for the target to be enumerated on USB (all ports) and to disappear from the UART port. This takes time due to the change in the QPST port parameter.
   c. Start the UART application with the intended processor by repeating steps 4 through 6.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8 Diag consumer interface

8.1 Overview

The DCI allows a client application to inject command requests, set logs and events masks on any of the supported cores, and receive back command responses, log packets, and event reports on the APSS. The existing Diag communication with PC Tools has not changed. Therefore, this interface can be used simultaneously with the existing use of QXDM Pro/on-device logging.

A new SMD data channel handles the DCI traffic on MSM and MDM devices and a new High-Speed Inter-chip interface (HSIC)/Peripheral Component Interconnect express (PCIe) channel on APQ+MDM devices. The APSS Diag kernel driver manages the traffic between the remote cores and individual clients. Figure 8-1 is an architectural diagram of DCI.

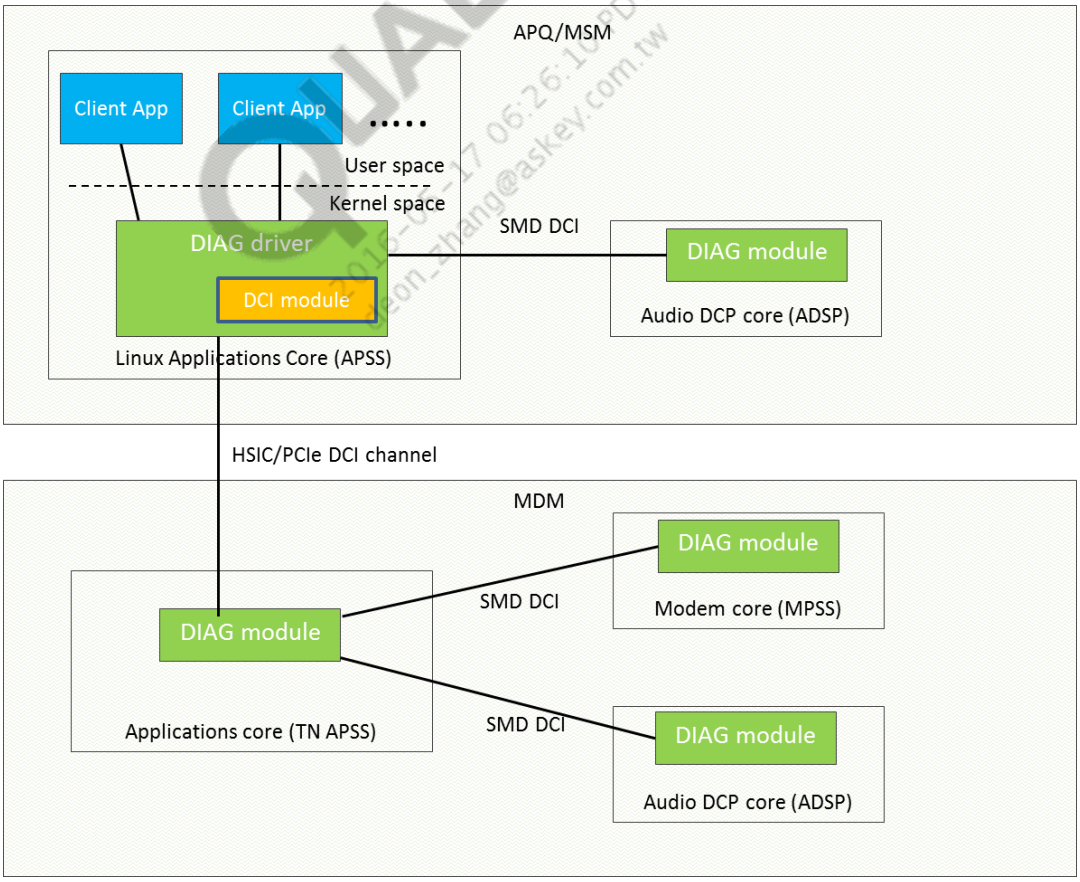


Figure 8-1 Diag consumer services overview

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.2 Initialization

A client application must call Diag_LSM_Init() to access the Diag Userspace Library/Legacy Service Mapping (LSM) API. This call creates a read thread for the client application. While this read thread performs all read operations with the kernel driver and is internal to the Diag library, the main client application thread is responsible for write operations, such as sending command packets and masks.

The client must register with the DCI via the LSM layer to receive a handle for using any of the DCI services. In APQ+MDM configurations, the APQ and MDM devices maintain their own instances of Diag, and the client must indicate which instance it will communicate with during initialization.

As part of the initialization process, a client may also register a signal callback and indicate the remote cores for which it must receive state change notifications. These notification can be triggered by the opening or closing of a remote core's DCI channel (as shown in Figure 8-2), which can be caused by events such as a subsystem restart (SSR). The kernel driver keeps track of all DCI clients with the corresponding core mask, signal information, and log/event masks for each client.

Figure 8-2 shows the call flow.

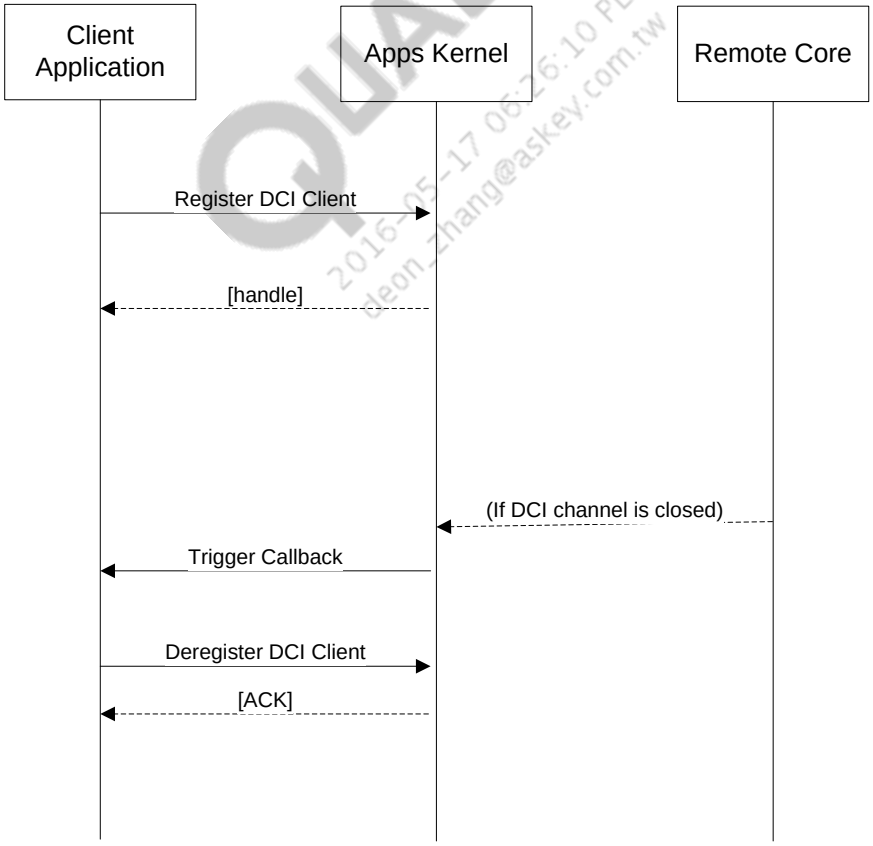


Figure 8-2 Call flow for initialization

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.3 Command/response packets

The command request/response service allows a client application to originate a Diag command request and receive a response. The Diag kernel driver handles the request packets and routes them to the remote core that registered with Diag to receive that command. If that core does not have a DCI channel open or does not support the DCI, an error code is returned. Likewise, if the command is not registered with Diag, another error code is returned.

On fused ASICs such as APQ+MDM, APQ Diag does not maintain command registrations from MDM cores. Therefore, it does not know in advance whether the command is supported. The DCI API returns SUCCESS to the client application and then forwards the command to the MDM device over its DCI channel. If the MDM does not find a matching entry for this command, it returns an error response packet while the DCI API returns SUCCESS to the client application. In this case, the client must check the response packet for validation.

The client application must send a command request in raw byte format using the async command send function described in 80-N8469-2.

The format of the command packets and their corresponding responses are described in 80-V1294-1. Command/response packets are documented in various Interface Specific Documents (ICDs), for example, 80-VP457-1.

Unlike command/response packets sent over the primary Diag data stream, those sent over the DCI channel are not High-Level Data Link Control (HDLC)-encoded.

Figure 8-3 shows the call flow.

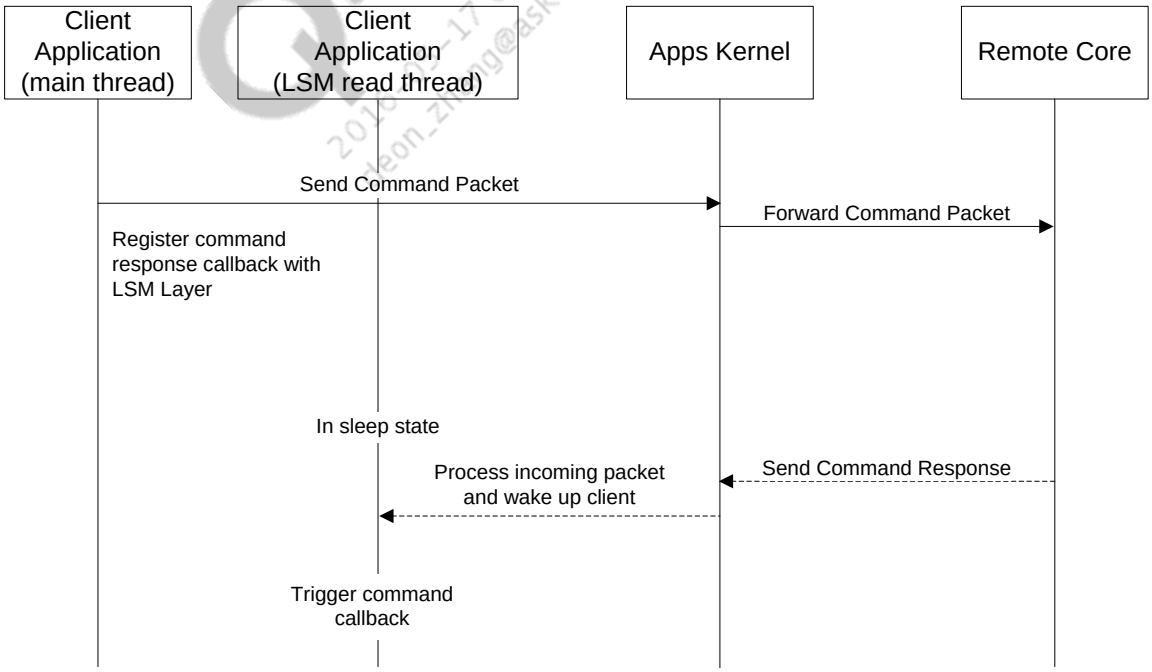


Figure 8-3 Call flow for command request/response service

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.4 Logs and events

A DCI client can configure masks for logs and events, and register corresponding callbacks to be called upon their receipt. The masks must be set individually per client (APQ or MDM). A mask set by an MDM client does not mean that the same mask is set on the APQ.

The client must not include complex processing in callback functions, or packets will be dropped on remote cores. The client can buffer the received packets for later processing.

The Diag kernel router maintains mask information for every DCI client so it can route a packet coming from the remote core to the matching DCI clients. In Figure 8-4, LSM refers to the read thread for a client application that is created with the Diag_LSM_Init() call. While the main client thread is used for writing data, the LSM read thread is used for reading data from the kernel driver.

The individual log records and event reports are formatted as outlined in 80-V1294-1. Logs are encapsulated as a Log Response message as described in Section 3.4.13.2 of 80-V1294-1. Events are encapsulated in event reports as described in Section 3.4.94.1 of 80-V1294-1. Unlike log and event packets sent over the primary diag data stream, packets sent using the DCI are not HDLC-encoded.

Figure 8-4 shows the call flow.

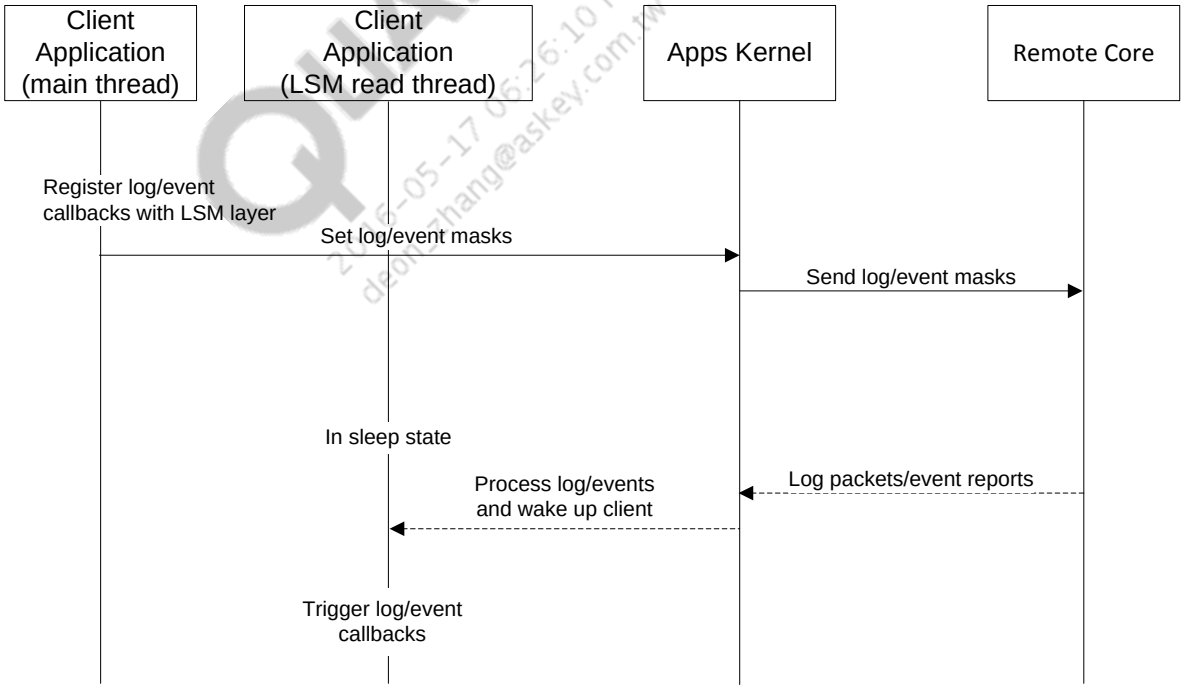


Figure 8-4 Call flow for logs/events services

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Table 8-1 lists the complete log packet format described in the CDMA DMSS ICD 80-V1294-1.

Table 8-1 Log packet structure

Field	Length (bytes)	Description
CMD_CODE (16/0x10)	1	Set to 16 for all log packets.
MORE	1	More log data are available. This field indicates how many log entries (not including the one returned with this message) are queued in the DMSS.
LENGTH	2	Length of the log record. This is the entire record including LENGTH, LOG_CODE, and TIMESTAMP.
LOG_CODE	2	16-bit logging code and consists of the following fields: ▪ Equipment identifier – 4 most significant bits of the log code specify the equipment ID ▪ Item identifier – 12 least significant bits of the log code specify the log item ID within the equipment ID
TIMESTAMP	8	DMSS timestamp; this is the same format as in the Time Stamp (29/0x1D).
DATA	LENGTH-12	Data specific to log type.

8.5 Limitations

Currently, DCI does not support Diag debug/F3 messages.

8.6 DCI functions

Table 8-2 lists functions available for DCI client applications.

NOTE: The functions in Table 8-2 work in the context of the calling DCI client. They do not affect other clients' masks maintained by the DCI server.

Table 8-2 Initialization, command/response, and data streaming functions

Function	Description
diag_register_dci_client()	Registers a client with the Diag router to receive traffic from other remote cores for the specified Diag instance, and for any other channel-related notifications.
diag_release_dci_client()	Closes the DCI connection for this client.
diag_get_dci_support_list()	DEPRECATED. Use diag_get_dci_support_list_proc().
diag_get_dci_support_list_proc()	Provides a list of remote cores supporting the DCI for the specified Diag instance, in the form of a bitmask.
diag_send_dci_async_request()	Nonblocking function that sends a command request. A callback function passed as an argument to this function is called when a command response becomes available.
diag_register_dci_stream()	DEPRECATED. Use diag_register_dci_stream_proc()

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Function	Description
diag_register_dci_stream_proc()	Initializes DCI streaming for logs and events. Callback functions for a log and event packet are passed as arguments to this function.
diag_log_stream_config()*	Configures the log codes for the client's DCI data stream.
diag_event_stream_config()*	Configures the event IDs for the client's DCI data stream.
diag_get_health_stats()	DEPRECATED. Use diag_get_health_stats_proc().
diag_get_health_stats_proc()	Gets the current DCI statistics for the logs and events, such as a count of dropped packets. This information covers only packets received or dropped by the DCI server on the APSS and does not include Diag statistical information from the individual remote cores.
diag_get_log_status()	Queries a specific log code to check whether it is enabled.
diag_get_event_status()	Queries a specific event ID to check whether it is enabled.
diag_disable_all_logs()	Disables all the log codes for a specified client.
diag_disable_all_events()	Disables all the event IDs for a specified client.
diag_dci_vote_real_time()	Votes for RT or NRT mode for a DCI client.
diag_dci_get_real_time_status()	Gets the current mode (RT or NRT) in which Diag operates.
diag_register_dci_signal_data()	Registers a signal to be triggered on receiving DCI data from non-HLOS remote cores.
diag_deregister_dci_signal_data()	Deregisters the signal for receiving DCI data from non-HLOS remote cores.

A DCI client must call Diag_LSM_Init() to register itself with the LSM layer and to use the functions listed in Table 8-2. Diag_LSM_Init() creates a read thread for the client application.

Eventually, a client application calls Diag_LSM_DeInit() to deregister itself and release all the buffers.

NOTE: `Android.mk` for a client application must include the Diag library `libdiag.so`.

Table 8-3 lists some of the error codes for the DCI functions.

Table 8-3 DCI error codes

Name	ID	Description
`DIAG_DCI_NO_ERROR`	1001	No error.
`DIAG_DCI_NO_REG`	1002	Could not register.
`DIAG_DCI_NO_MEM`	1003	Failed memory allocation.
`DIAG_DCI_NOT_SUPPORTED`	1004	Client is not supported.
`DIAG_DCI_HUGE_PACKET`	1005	Request/response packet is too big.
`DIAG_DCI_SEND_DATA_FAIL`	1006	Writing to the kernel or remote core/ASIC failed.
`DIAG_DCI_ERR_DEREG`	1007	Error dealing with registration tables.
`DIAG_DCI_PARAM_FAIL`	1008	Incorrect parameter.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

From APQ8084+MDM9625 onwards, the DCI is supported on MDMs. A DCI connection can only communicate with a single Diag instance (either APQ or MDM devices), so certain API functions include an additional parameter to make the distinction. For details, see the individual function descriptions.

If a client must use the DCI to communicate with both APQ and MDM devices, then it must register twice with the DCI and a separate handle is assigned for each connection.

8.6.1 diag_register_dci_client()

Initialization function required for DCI functions.

As part of registration, the client can pass in a bitmask of remote cores from which it wants to receive status notifications, as well as a signal to be used when the channel status changes, for example a channel opens or closes.

Parameters

```
int diag_register_dci_client
(
     int *client_id,
     diag_dci_peripherals *list,
     int proc,
     void *os_params
)
```

→	client_id	Pointer to an integer that holds the client ID.
→	List	Pointer to a bitmask that holds the remote core information.
→	proc	This specifies the instance of Diag with which the client is to communicate. Any other value return an error. Possible values are DIAG_PROC_MSM or DIAG_PROC_MDM.
→	os_params	OS-specific parameters. The parameters can be a pointer to an event object, or a signal mask and a Task Control Block (TCB).

Returns

DIAG_DCI_NO_ERROR – Registered successfully

DIAG_DCI_NO_REG – Could not register

DIAG_DCI_NO_MEM – Memory allocation failed

Sample code

```
/* Registering with DCI */
int client_id = 0, signal_type = SIGCONT, channel = 0, err =0;
diag_dci_peripherals list = DIAG_CON_MPSS;
err = diag_register_dci_client(&client_id, &list, channel, &signal_type);
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.6.1.1 DCI connection notification

When there is a change in DCI channel status (for example, if the DCI channel to a remote core opens or closes), a DCI client can be notified of the change. This is done using the signal passed in as part of the diag_register_dci_client() registration process.

To receive the actual notifications, the client must register a signal callback with the kernel driver. An example of how to register a signal callback is shown below. This example does not involve any of the DCI API functions directly. The signal used must match the signal passed into diag_register_dci_client().

```
/* Signal handling for DCI status notification */
int signal_type = SIGCONT;  /* Must match the signal passed into
                               diag_register_dci_client() */
struct sigaction notify_action;
void notify_handler(int signal, siginfo_t *info, void *unused);

sigemptyset(&notify_action.sa_mask);
notify_action.sa_sigaction = notify_handler;
/* Use SA_SIGINFO to denote we are expecting data with the signal */
notify_action.sa_flags = SA_SIGINFO;
sigaction(signal_type, &notify_action, NULL);
```

When the notification is called, the client uses diag_get_dci_support_list_proc() to determine the current status of the DCI remote cores.

On fused ASICs such as an APQ+MDM device, the MDM's APSS notifies the APQ Diag when a core's channel status changes. The APQ device then signals the DCI clients that have registered for channel change notification.

If the DCI channel between the MDM and APQ devices is open, the DCI API functions send the mask changes and command requests to the MDM device and return success (they return failure only when the DCI channel is disconnected).

When the DCI channel between the MDM and APQ devices goes down, the APQ device notifies the client that the APSS channel was closed (this is the channel to the MDM APSS). In single ASICs, notification that the APSS channel is closed is never sent (a closed APSS channel means the device has crashed).

NOTE: In the function diag_process_dci_transcation, for a "kk" build, an error is returned when the SMD channel is not UP, but no error is returned when the channel is not UP on L builds.

Earlier, NUM_DCI_CHANNELS was set to a predefinied number whose value was equal to the actual number of peripherals that had DCI support. Now, it is scaled to peripherals that exist in the system. This is not actually an error case – the channel will be open if DCI support is available, and is NULL otherwise.

The client must query the support for DCI before registering the client using the int diag_get_dci_support_list_proc(int proc, diag_dci_peripherals *list) API.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

In the case of SSR or a DCI kill switch (when the channel goes down), the client is given a signal. The client should then query the support again when this signal is given. The client can register for the signal during DCI registration.

int diag_register_dci_client(int *client_id, diag_dci_peripherals *list, int proc, void *signal);

This is the recommended way of knowing whether the peripheral supports DCI.

8.6.2 diag_release_dci_client()

Closes the DCI connection for this client. The client must pass a pointer to the client ID that was generated earlier.

If a client terminates prematurely without calling this function, some performance degradation or battery drain can occur because the set log/event masks are not cleared. Log/event packets and delayed command responses continue to be generated on the remote core and forwarded to the APSS. The client buffers are not released, limiting the number of DCI clients that can be created.

Parameters

```
int diag_release_dci_client
   (
        int *client_id
   )
```

→	client_id	Pointer to an integer that holds the client ID.

Returns

DIAG_DCI_NO_ERROR – Deregistered successfully

DIAG_DCI_ERR_DEREG – Error while deregistering

Sample code

```
int err =0;
err = diag_release_dci_client(&client_id);
```

8.6.3 diag_get_dci_support_list()

NOTE: This function is deprecated. It is the same as calling diag_get_dci_support_list_proc(DIAG_PROC_MSM,…). Use diag_get_dci_support_list_proc() instead.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.6.4 diag_get_dci_support_list_proc()

Provides information on which remote cores support the DCI and currently have their DCI channel open.

DCI clients might call this function when they receive notification that the status of a remote core has changed. The status change is signaled using the os_params and list fields passed in when calling diag_register_dci_client(). DCI clients might also call this function at any time to check the status of the remote cores in the system. The user must pass a pointer to the bitmask described in diag_register_dci_client() and the ID of the ASIC that requires logging.

Parameters

```
int diag_get_dci_support_list_proc
   (
        int proc,
        diag_dci_peripherals *list
   )
```

→	proc	Value can be DIAG_PROC_MSM or DIAG_PROC_MDM.
→	list	Pointer to a bitmask that holds remote core information. When DIAG_PROC_MSM is used, the DIAG_CON_APSS bit refers to the local applications processor. When DIAG_PROC_MDM is used, the DIAG_CON_APSS bit refers to the remote applications processor on the MDM.

Returns

DIAG_DCI_NO_ERROR – No error

DIAG_DCI_PARAM_FAIL – Invalid parameter

DIAG_DCI_NO_MEM – Memory allocation failed

Sample code

```
diag_get_dci_support_list_proc(DIAG_PROC_MDM, &list);
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.6.5 diag_send_dci_async_request()

Submits a request packet to the DCI.

In addition to providing the byte sequence for the request packet (as defined in 80-V1294-1), the client must also include a pointer to a buffer to hold the response packet when it is received. The client ensures that the response buffer is large enough to hold the response.

Parameters

```
int diag_send_dci_async_request
   (
        int client_id,
        unsigned char buf[],
        int bytes,
        unsigned char *rsp_ptr,
        int rsp_len,
        void (*func_ptr)(unsigned char *ptr, int len, void *data_ptr),
        void *data_ptr
   )
```

→	client_id	Client ID.
→	buf	Request buffer.
→	bytes	Request buffer length.
→	rsp_ptr	Pointer to the response buffer.
→	rsp_len	Response buffer length.
→	func_ptr	Callback function pointer.
→	data_ptr	Data pointer.

Return values

DIAG_DCI_NO_ERROR – No error

DIAG_DCI_NOT_SUPPORTED – Client is not supported

DIAG_DCI_HUGE_PACKET – Request/response packet is too large

DIAG_DCI_SEND_DATA_FAIL – Writing to the kernel or remote core/ASIC failed.

Sample code

```
unsigned char  *dci_rsp_pkt = malloc(100);
/* Diag Log stress test command - one Log packet comes out every second */
unsigned char buf[24] = {75, 18, 0, 0, 1, 0, 0, 0, 16, 1, 1, 0, 0, 1, 0, 0,
232, 3, 0, 0, 1, 0, 0, 0};
/* Handler for response; prints the response packet to the file */
void process_response(unsigned char *ptr, int len, void *data_ptr);
err = diag_send_dci_async_req(data, buf, 24, dci_rsp_pkt,
   100, &process_response, NULL);
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.6.6 diag_register_dci_stream()

NOTE: This function is deprecated. It is the same as calling diag_register_dci_stream_proc() for a client that registered a DCI connection with an MSM device. If this function is called for a client registered for an MDM, it fails. Use diag_register_dci_stream_proc() instead.

8.6.7 diag_register_dci_stream_proc()

Initialization function required for a client to receive logs and events from the DCI. The client must provide callbacks to be used when log and event packets are received.

Parameters

```
int diag_register_dci_stream_proc
(
     int client_id,
     void (*func_ptr_logs)(unsigned char *ptr, int len),
     void (*func_ptr_events)(unsigned char *ptr, int len)
)
```

→	client_id	ID from client registration with DCI.
→	func_ptr_logs	Callback function for logs.
→	func_ptr_events	Callback function for events.

Returns

DIAG_DCI_NO_ERROR – Registered successfully.

Sample code

```
/* Handler for logs; prints the log packet to the file */
void process_dci_log_stream(unsigned char *ptr, int len);
/* Handler for events; prints the event packet to the file */
void process_dci_event_stream(unsigned char *ptr, int len);
err = diag_register_dci_stream_proc(client_id, process_dci_log_stream,
process_dci_event_stream);
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.6.8 diag_log_stream_config()

Configures log streaming to the client.

This function sends an array of log codes that are requested by the client. The individual logs listed by the client are either enabled or disabled in the internal log mask used to route data to the client, depending on the value of set_mask.

Parameters

```
int diag_log_stream_config
(
     int client_id,
     int set_mask,
     uint16 log_codes_array[],
     int num_codes
)
```

→	client_id	Client ID.
→	set_mask	Enables or disables the log specified later.
→	log_codes_array	Array of log codes.
→	num_codes	Number of log codes specified in log_code_arrays.

Returns

DIAG_DCI_NO_ERROR – No error

DIAG_DCI_NOT_SUPPORTED – Client is not supported

DIAG_DCI_SEND_DATA_FAIL – Writing to the kernel or remote core/ASIC failed

Sample code

```
uint16  log_codes_array[7] = {0x5072, 0x115F, 0x12E8, 0x119B, 0x11AF,
0x14C8, 0x1375};
err = diag_log_stream_config(client_id, ENABLE, log_codes_array, 7);
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.6.9 diag_event_stream_config()

Configures event streaming to the client.

This function sends an array of event IDs requested by the client. The individual events listed by the client are either enabled or disabled in the internal event mask used to route data to the client, depending on the value of set_mask.

Parameters

```
int diag_event_stream_config
   (
        int client_id,
        int set_mask,
        int event_id_array[],
        int num_codes
   )
```

→	client_id	Client ID.
→	set_mask	Enables or disables the event specified later.
→	event_id_array	Array of event IDs.
→	num_codes	Number of event IDs specified in event_id_array.

Returns

DIAG_DCI_NO_ERROR – No error

DIAG_DCI_NOT_SUPPORTED – Client is not supported

DIAG_DCI_SEND_DATA_FAIL – Writing to the kernel or remote core/ASIC failed

8.6.10 diag_get_health_stats()

NOTE: This function is deprecated. It is the same as calling diag_get_health_stats_proc() for a client that has registered a DCI connection with an MSM device. If this function is called for a client registered for an MDM, it fails. Use diag_get_health_stats_proc() instead.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.6.11 diag_get_health_stats_proc()

Gets the current DCI statistics for logs and events on a specific core.

Parameters

```
int diag_get_health_stats_proc
(
     int client_id,
     struct diag_dci_health_stats *dci_health,
     int proc
)
```

→	client_id	ID received while registering with DCI interface

→	dci_health	Structure for storing all the statistics:
		`struct diag_dci_health_stats`
		`{`
		`  int dropped_logs;`
		`  int dropped_events;`
		`  int received_logs;`
		`  int received_events;`
		`  int reset_status;`
		`};`

→	proc	Specifies the core for which health stats are needed. Values: ▪ DIAG_ALL_PROC ▪ DIAG_MODEM_PROC ▪ DIAG_LPASS_PROC ▪ DIAG_WCNSS_PROC ▪ DIAG_APPS_PROC When a MDM DCI client uses diag_get_health_stats_proc(), the proc parameter is ignored and the only result is always cumulative of all the MDM cores' data statistics (the equivalent of passing in DIAG_ALL_PROC).

Returns

DIAG_DCI_NO_ERROR – No error

DIAG_DCI_NO_MEM – Memory allocation failed

Sample code

```
/* Display Health Statistics */
struct diag_dci_health_stats *dci_health_stats = malloc(sizeof(struct
diag_dci_health_stats));
err = diag_get_health_stats_proc(client_id, dci_health_stats,
DIAG_MODEM_PROC);
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.6.12 diag_get_log_status()

Checks whether a client has enabled a specific log code.

Parameters

```
int diag_get_log_status
(
     int client_id,
     uint16_t log_code,
     boolean *value
)
```

→	client_id	Client ID.
→	log_code	Log code to check.
←	value	Boolean indication of whether the log is enabled.

Returns

DIAG_DCI_NO_ERROR – No error

DIAG_DCI_NOT_SUPPORTED – Client is not supported

DIAG_DCI_NO_MEM – Memory allocation failed

DIAG_DCI_SEND_DATA_FAIL – Writing to the kernel failed

8.6.13 diag_get_event_status()

Checks whether a client has enabled a specific event ID.

Parameters

```
int diag_get_event_status
(
     int client_id,
     uint16_t event_id,
     boolean *value
)
```

→	client_id	Client ID.
→	event_id	Event ID to check.
←	value	Boolean indication of whether the event is enabled.

Returns

DIAG_DCI_NO_ERROR – No error

DIAG_DCI_NOT_SUPPORTED – Client is not supported

DIAG_DCI_NO_MEM – Memory allocation failed

DIAG_DCI_SEND_DATA_FAIL – Writing to the kernel failed

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.6.14 diag_disable_all_logs()

Disables all the log codes for a specified client.

Parameters

```
int diag_disable_all_logs
(
     int client_id
)
```

→	client_id	Client ID.

Returns

DIAG_DCI_NO_ERROR – No error

DIAG_DCI_NOT_SUPPORTED – Client is not supported

DIAG_DCI_NO_MEM – Memory allocation failed

DIAG_DCI_SEND_DATA_FAIL – Writing to the kernel or remote core/ASIC failed

8.6.15 diag_disable_all_events()

Disables all the event IDs for a specified client.

Parameters

```
int diag_disable_all_events
(
     int client_id
)
```

→	client_id	Client ID.

Returns

DIAG_DCI_NO_ERROR – No error

DIAG_DCI_NOT_SUPPORTED – Client is not supported

DIAG_DCI_NO_MEM – Memory allocation failed

DIAG_DCI_SEND_DATA_FAIL – Writing to the kernel or remote core/ASIC failed

QUALCOMM® 2016-07-17 06:25:10 PDT deon_zhang@askey.com.tw

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.6.16 diag_dci_vote_real_time()

Requests real time or non-real time mode for Diag on non-HLOS cores.

In non-real time mode, remote cores aggregate Diag packets in their local buffers for a longer period of time before transmitting them to the APSS. This helps prevent frequent wakeups of the APSS because of Diag activity, but the effectiveness depends on how much data is requested (see Section 8.8).

If a DCI client for an MDM device requests NRT mode (and all other conditions for NRT mode are satisfied, such as a USB disconnect when diag_mdlog is not up), the NRT mode control packet is sent to the MDM device. But MSM cores might still run in RT mode if there are any DCI clients requesting RT mode.

```
int diag_dci_vote_real_time
(
     int clienreal-timeint real_time
)
```

→	client_id	Client ID.
→	real_time	Requested mode: 0: MODE_NONREALTIME 1: MODE_REALTIME

Returns

DIAG_DCI_NO_ERROR – No error

DIAG_DCI_NOT_SUPPORTED – Client is not supported

DIAG_DCI_PARAM_FAIL – Invalid parameter

DIAG_DCI_SEND_DATA_FAIL – Writing to the kernel or remote core/ASIC failed

8.6.17 diag_dci_get_real_time_status()

Checks the current mode of operation (real time or non-real time) for Diag.

Parameters

```
int diag_dci_get_reareal-timeatus
(
     int *real_time
)
```

→	real_time	Pointer to an integer to hold the mode value

Returns

DIAG_DCI_NO_ERROR – No error

DIAG_DCI_PARAM_FAIL – Invalid parameter

DIAG_DCI_SEND_DATA_FAIL – Writing to the kernel or remote core/ASIC failed

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.6.18 diag_register_dci_signal_data()

Registers a signal to be triggered by the Diag user space library upon receiving DCI data from the remote cores (see Section 9.3.6).

Parameters

```
int diag_register_dci_signal_data
(
     int client_id,
     int signal_type
)
```

→	client_id	Client ID.
→	signal_type	Signal number, which is to be an uncommon Linux signal.

Returns

DIAG_DCI_NO_ERROR – No error

DIAG_DCI_PARAM_FAIL – Invalid parameter

DIAG_DCI_NO_MEM – Memory allocation failed

DIAG_DCI_SEND_DATA_FAIL – Writing to the kernel or remote core/ASIC failed

Sample code

```
#define DIAG_SAMPLE_SIGNAL  SIGRTMIN + 15
int err = diag_register_dci_signal_data(client_id, DIAG_SAMPLE_SIGNAL);
if (err != DIAG_DCI_NO_ERROR) {
DIAG_LOGE(" Could not register signal for DCI Data, err: %d, errno: %d\n",
err, errno);
   free(dci_rsp_pkt);
   return -1;
}
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.6.19 diag_deregister_dci_signal_data()

This function disables the signal registered with the diag_register_dci_signal_data() function. .

Parameters

```
int diag_deregister_dci_signal_data
(
     int client_id,
     int signal_type
)
```

→	client_id	Client ID.
→	signal_type	Signal number, which is to be an uncommon Linux signal

Returns

DIAG_DCI_NO_ERROR – No error

DIAG_DCI_NOT_SUPPORTED – Client is not supported

DIAG_DCI_NO_MEM – Memory allocation failed

DIAG_DCI_PARAM_FAIL – Invalid parameter

8.7 Security

Several security methods exist to help protect unauthorized applications from using or abusing the DCI. The following sections provide a brief overview of these methods, referring to other detailed documentation where appropriate.

8.7.1 APSS protections

8.7.1.1 Access permissions

The DCI client application needs access to the /dev/diag port to function properly. /dev/diag has access permissions set to 660: only the owner and group have read and write permissions.

Therefore, the DCI client application must run as one of the following:

- User root
- User system
- Within the qcom_diag group

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.7.2 MPSS protections

8.7.2.1 DCI override feature

NOTE: For full details on the DCI override feature, refer to 80-NE668-1. This section simply provides a brief overview of what the override feature does.

The DCI authentication feature locks down access to the DCI while the device is in the factory by provisioning an authentication cookie to the modem. When DCI access is locked out, the modem does not open its DCI SMD channel to the APSS, preventing any DCI access. To unlock DCI access, an override cookie must be provided over the standard Diag interface. This is referred to as the DCI override feature.

8.7.2.2 Secure services module

NOTE: A full description of the Secure Service Module (SSM) can be found in 80-NH283-1.

The SSM is a service used on the MPSS that provides fine-grain controls over the Diag commands that can execute on the modem. Each incoming Diag command received on the modem (either through the DCI channel or the standard Diag channel) is checked against the modem's SSM permissions files that the OEM has provisioned to the device. If allowed by the SSM, the command executes. If disallowed by the SSM, an error response returns to the originator of the command.

8.8 NRT mode

Real time Diag packets generated on remote cores are forwarded to the APSS with minimal buffering to help ensure the packets are received in a timely manner. However, this behavior can lead to high power consumption because of frequent wakeups of the APSS.

The DCI can configure the remote cores to aggregate Diag packets for a longer period of time before sending them to the APSS, which reduces the frequency of APSS wakeups.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.8.1 NRT/Buffering mode

When NRT mode is enabled on a core, the Diag drain timer is set to a larger value, and the buffer threshold is adjusted to hold more data before forwarding it to the APSS.

8.8.1.1 DCI

A DCI client can request NRT mode by using the following API functions:

1. To vote for real time or non-real time mode:

```
int diag_dci_vote_real_time(int clireal-timent real_time);
```

   - Parameters – MODE_REALTIME or MODE_NONREALTIME
   - Return values

DIAG_DCI_NOT_SUPPORTED	Client is not supported.
DIAG_DCI_PARAM_FAIL	Incorrect parameter.
DIAG_DCI_NO_ERROR	No error.
DIAG_DCI_SEND_DATA_FAIL	Writing to the kernel or remote core/ASIC failed.

2. To retrieve the current Diag mode:

```
int diag_dci_get_real_time_status(int *real_time);
```

   - Parameters – pointer to store the mode value
   - Return values

DIAG_DCI_SEND_DATA_FAIL	Writing to the kernel or remote core/ASIC failed.
DIAG_DCI_PARAM_FAIL	Incorrect parameter.
DIAG_DCI_NO_ERROR	No error.

When there is more than one client in the system, a client voting for real time data takes precedence over a client voting for non-real time data. Real time clients vote for their preferred mode and the APSS Diag makes a decision on the final mode considering all the active Diag clients (which includes not just DCI clients but also other Diag logging modes) and the state of the USB connection.

On fused ASICs such as an APQ+MDM devices, the HSIC channel between the MDM and APQ devices is suspended whenever there is no data traffic for approximately 200 ms. In NRT mode, if less data is generated, the HSIC DCI channel is suspended on both sides. When the MDM device has data to drain, it resumes the channel and data is drained to the APQ device. Similarly, when the APQ device must send a mask change or command request to the MDM device, it resumes the channel and sends data to the MDM device.

If heavy logging occurs, DCI clients must run in NRT mode because HSIC wakeups on fused devices consume significant power.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8.8.2 Power management on the APSS

In NRT mode, the number of APSS wakeups can be reduced by buffering more data on the remote cores. If there are no active processes on the APSS, it might go into power collapse and, therefore, the kernel driver cannot transfer data to a user space application. This might lead to packets being dropped on remote cores because of transport layer blockage.

If registered with the diag_register_dci_signal_data() function described in 80-N8469-2, the Diag user space library signals the client that there is data for it to process. In this signal handler, the client must use its own power management scheme to keep the core awake until it processes the data. It must acquire the wake lock (or other suitable power related object) in this signal handler and release it when processing completes.

QUALCOMM
2016-05-17 06:26:10 PDT
deon_zhang@askey.com.tw

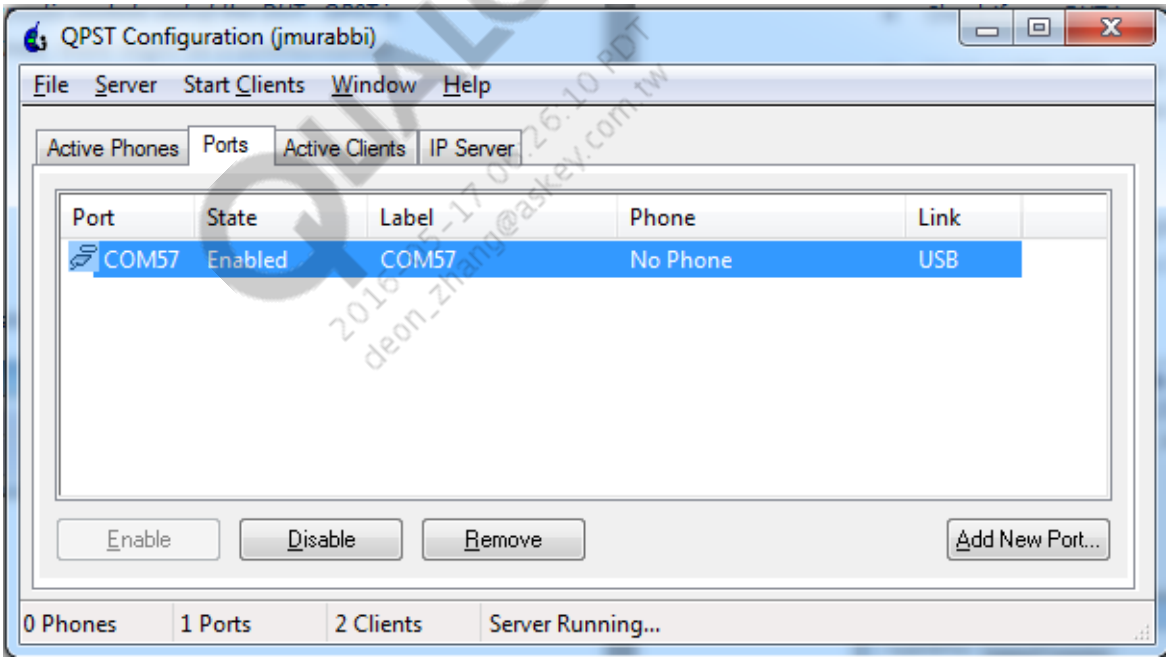
MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

9 Debugging

This chapter provides information on troubleshooting QPST port connection issues, collecting QPST port traces, and various techniques and scripts available for analyzing other port connection status issues. In addition, techniques are described for debugging commands on the APSS and modem processor.

9.1 QPST disconnect/No Phone status

Diag communicates with an external tool to send diagnostic packets out of the device. QPST is one such server tool listening to the device's Diag port. Occasionally, there might be a communication problem between QPST and diag running on the device.

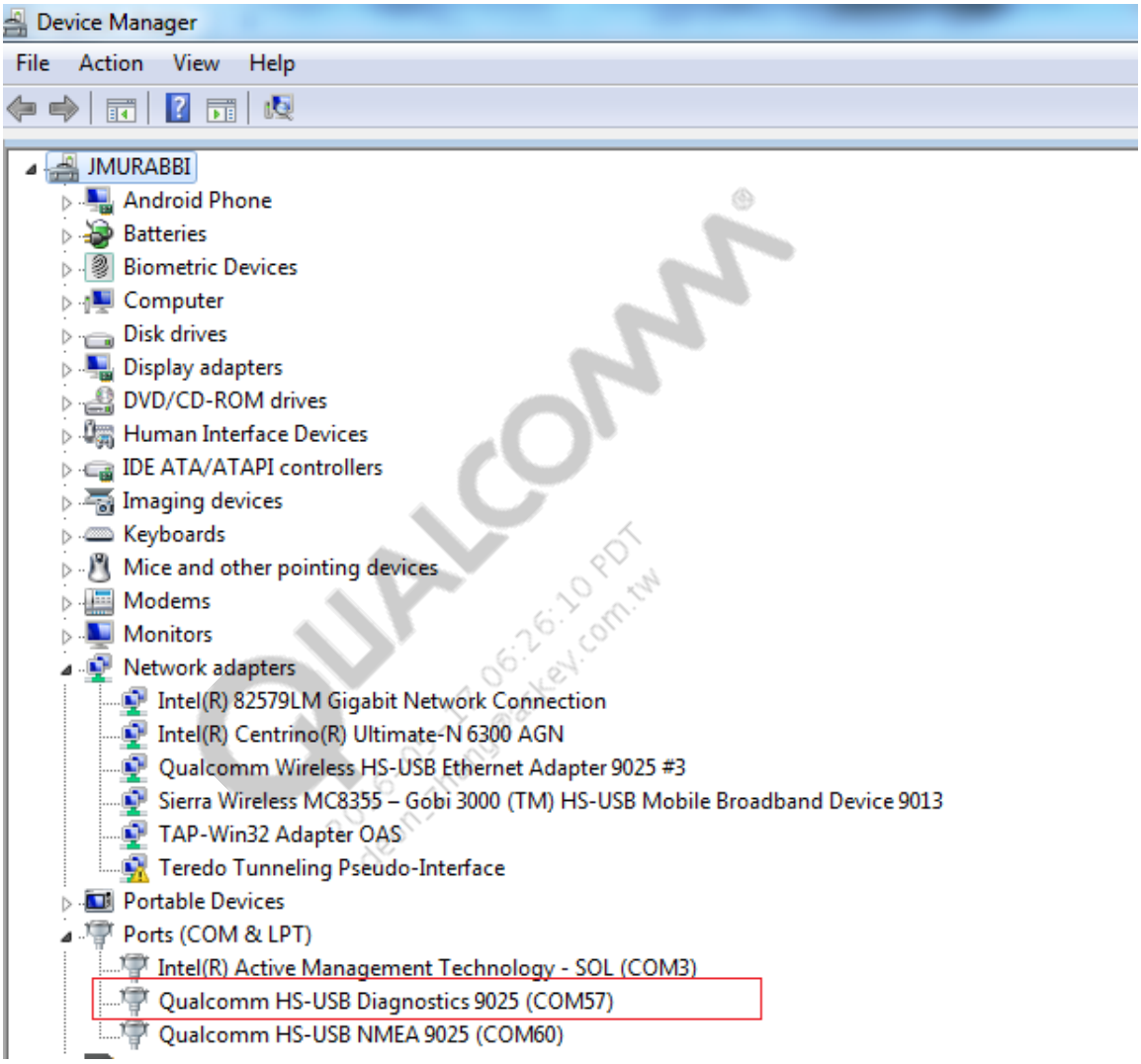


QPST string	What it means
No Phone	Communication error with device Diag port
?(0)	QPST does not have a matching string for this chipset, should upgrade QPST Or MPSS is not up, hence APSS is responding
MSMxxxx, MDMxxxx, APQxxxx	Ideal condition; device responding properly to polling commands

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

9.1.1 Check for host driver issues

In Windows Device Manager → Ports, check if Diagnostics USB port is present.



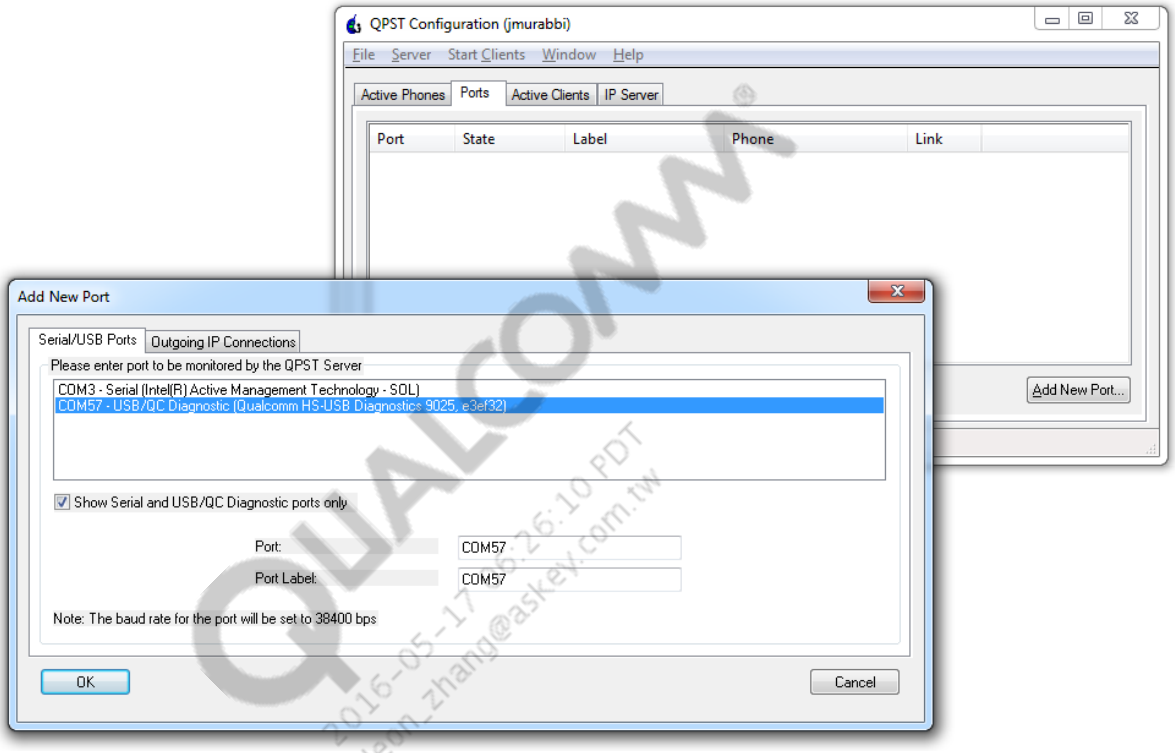
Troubleshooting tips:

- Check if the device is powered properly
- Unplug the USB cable, wait for approximately 30 sec and plug it back in. The Diagnostic USB port should go away and come back.
- If the port is not present, check if host USB drivers are installed properly. If the host USB drivers are installed properly, create a case with host USB tools/drivers.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

9.1.2 Check for tools issues

- Close existing sessions of QPST and/or QXDM and start new session.
- If you do not see the device's Diag port in the QPST Configuration view, click **Add New Port** to select the intended port.

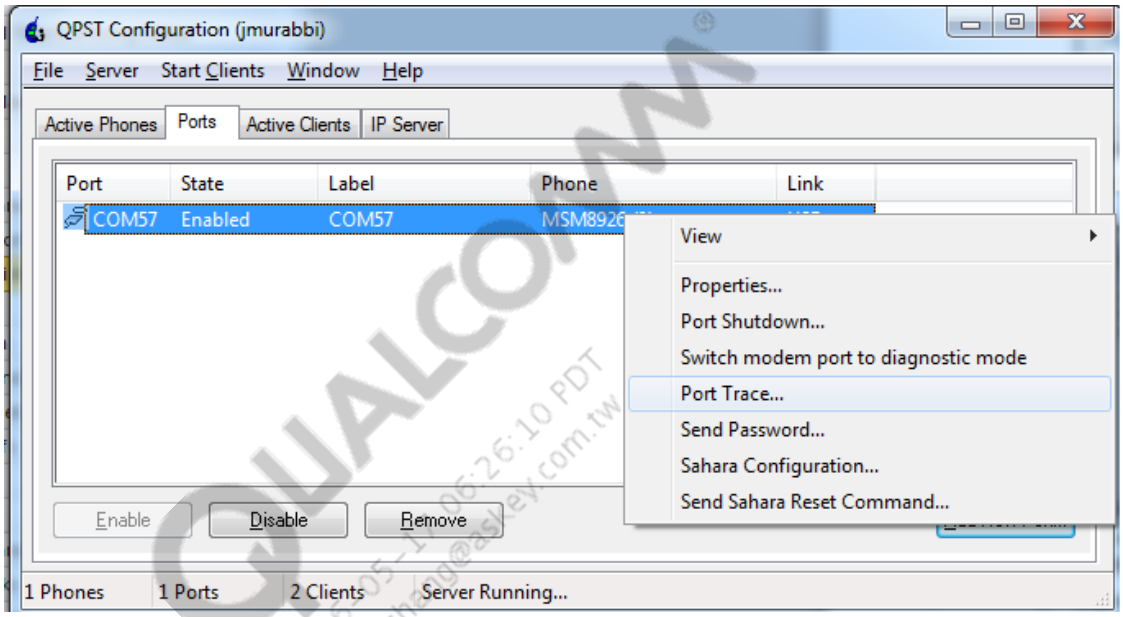


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

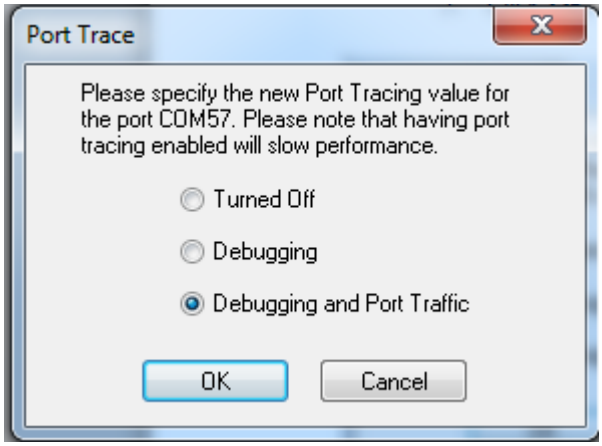
9.1.3 Check QPST port trace

Collect the port trace and verify if one of the polling commands is getting responses in a timely manner.

1. Go to QPST Configuration and select the Ports tab.
2. Hold shift and right-click on the device port.
3. Select Port Trace…



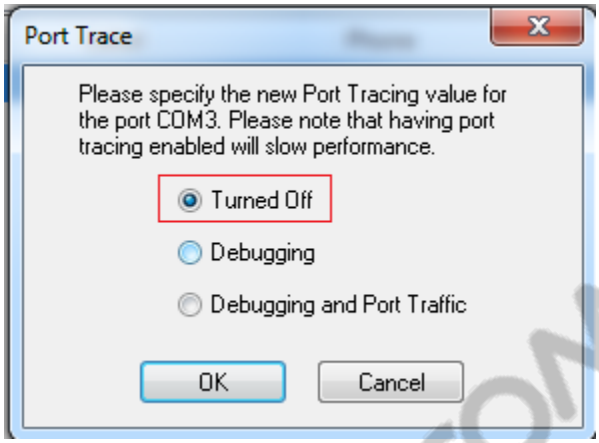
4. In the dialog, select Debugging and Port Traffic and click **OK**.



Port trace file (PortTrace_COMx) will be created in QPST Data Directory (QPST Configuration→Help→Open Log File Directory).

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

5. After a few seconds, if the port does not change to a valid device port, hold shift and right-click on the device port to reopen the Port Trace dialog.
6. Stop the port trace by selecting Turned Off from Port Trace options and click **OK**.



7. Analyze the generated PortTrace_COMx to investigate the issue as indicated in Section 9.1.3.1.

9.1.3.1 Port trace interpretation

The following is a snippet of a port trace:

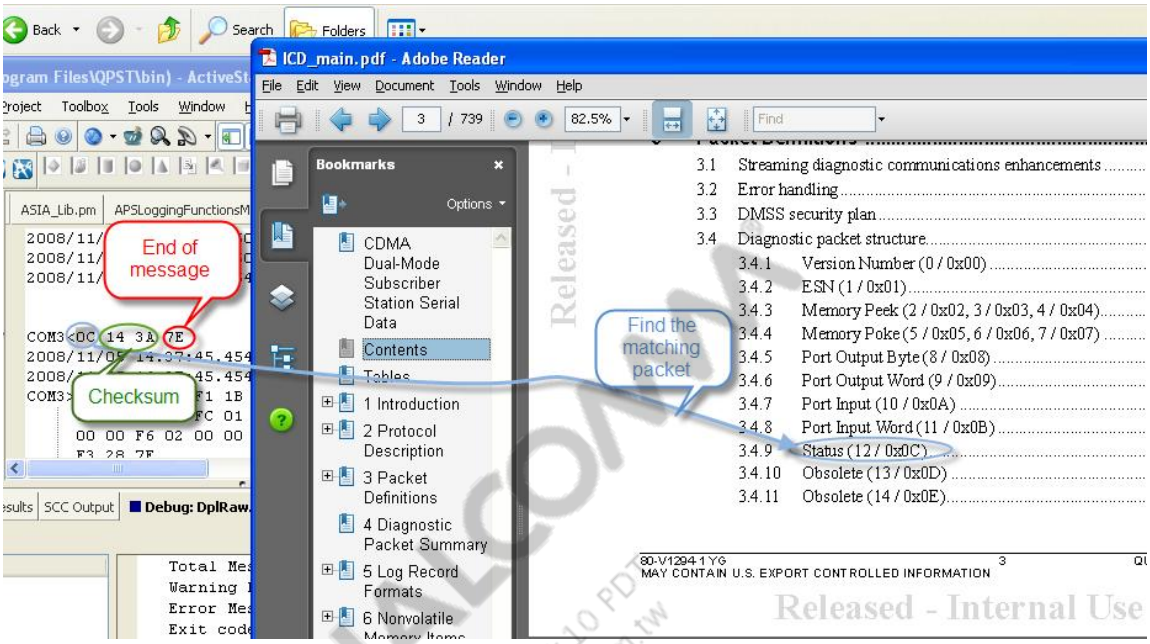
```
COM3<0C 14 3A 7E
2008/11/05 14:37:45.454:    780: P:  12  20 115200 size:    4 --> QueueCount=0
2008/11/05 14:37:45.454: 1 reads queued
COM3<0C E1 00 E1 F1 1B 04 74 01 00 C8 11 03 00 C8 11
     03 00 FC 01 FC 01 00 00 00 FF 00 00 00 00 00 00
     00 00 F6 02 00 00 00 00 00 00 00 00 00 00 00 00
     F3 28 7E
```

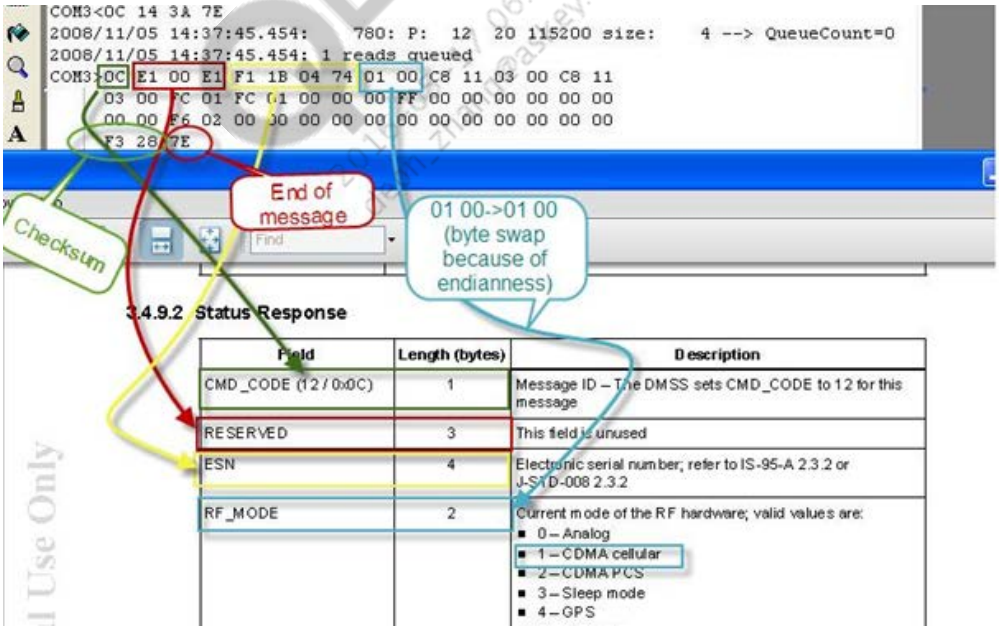
To interpret these bytes:

- Get a copy of the latest version of CDMA Dual-Mode Subscriber Station Serial Data Interface Control Document 80-V1294-1.
- The ">" and "<" shown in the snippet reflects the direction of the byte stream from/to QPST and the COM port. Examples:
  - "COM3<0C 14 3A 7E" means QPST sent 0C 14 3A 7E (the request packet) to COM3.
  - “COM3>0C E1 ... 7E” is the response packet sent from COM3 to QPST.
- In the snippet above:
  - The last byte "7E" is the end of message tag for every Diag packet.
  - The last two bytes before the last byte "7E" are the checksum.
  - The first one byte is the command code, and you need to find the matching command definition from the ICD.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

The following screenshots illustrate the request and response packet interpretations, respectively, in QPST port trace.





MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

9.1.3.2 QPST polling issues

QPST sends polling command requests to device every 600 ms (default value) and waits 400 ms (default value) to receive the corresponding response. If QPST does not receive response within the interval 3 (default value) times in a row, a diag/QPST disconnect occurs.

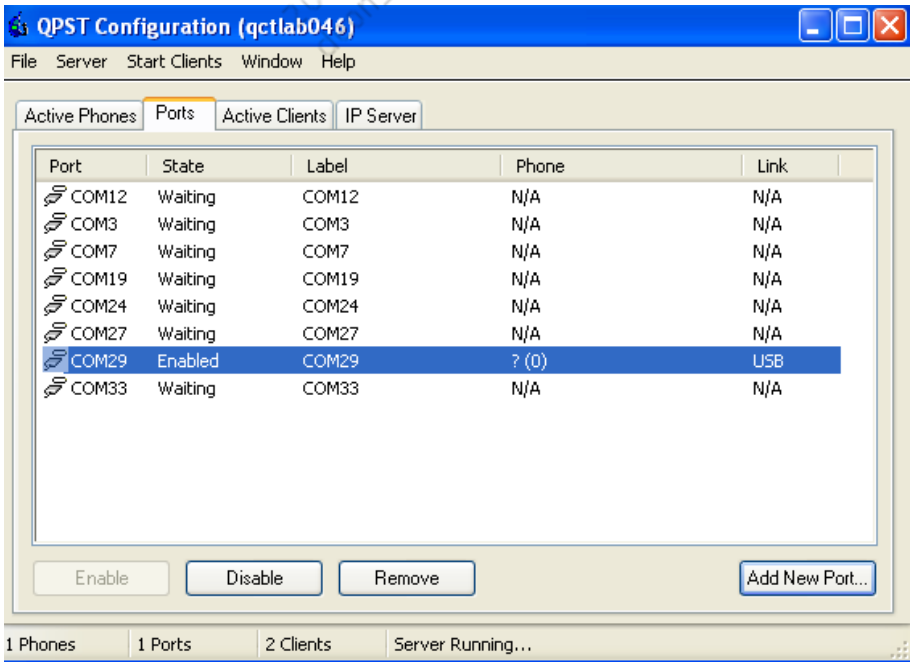
When experiencing a port disconnect, these values can be increased. For example:

- Polling timeout duration – 3000 ms
- Polling timer interval – 3000 ms
- Polling maximum retries – 5

See Section 7.4 for information on updating the polling parameters to address QPST port disconnection issues.

CDMA/WCDMA/GSM phone detection and status works as follows:

- When DEVICE is connected to QPST, the first request QPST sends to device is the status request.
- Status request is a one-byte command request – *0x0C*
  - See Section 3.4.9.1 in 80-V1294-1 for the definition of the Status Request message.
- The device will respond with a Status Response packet, which includes information such as the RF mode, ESN, etc.
  - See Section 3.4.9.2 in 80-V1294-1 for the definition of the Status Response message.
- QPST sends the **Extended Build ID command 124 (0x7C**) to query a code to display the appropriate string, otherwise it will display **?(0).** This is for cosmetic reasons only, if **?(0)** displays, that suggests a proper connection has been established.



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

QPST connection is established by continuously sending and receiving a response to one of the several polling commands. QPST has a state machine which sends these commands as well as other commands for other non-Diag modes (such as Download mode) to detect which mode the target is in. Once a response is received for one of these commands, QPST stays with this command and continuously polls with this command to keep the QPST connection alive.

To determine which polling commands are sent to the device, bear in mind that the QPST polling algorithm changes in terms of polling commands for the following four cases:

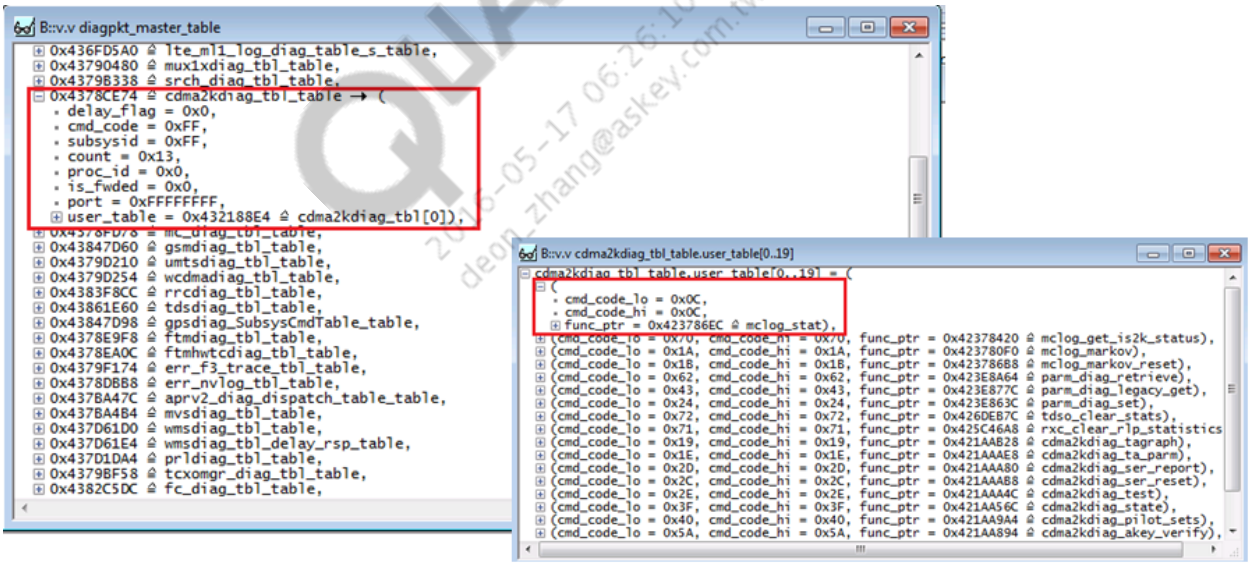
Case 1 – MPSS is available

If a processor has registered for one of the following polling commands, the polling command request will be forwarded to the registered processor:

- 0x0C
- 0x4B 04 0E 00
- 0x4B 08 02 00

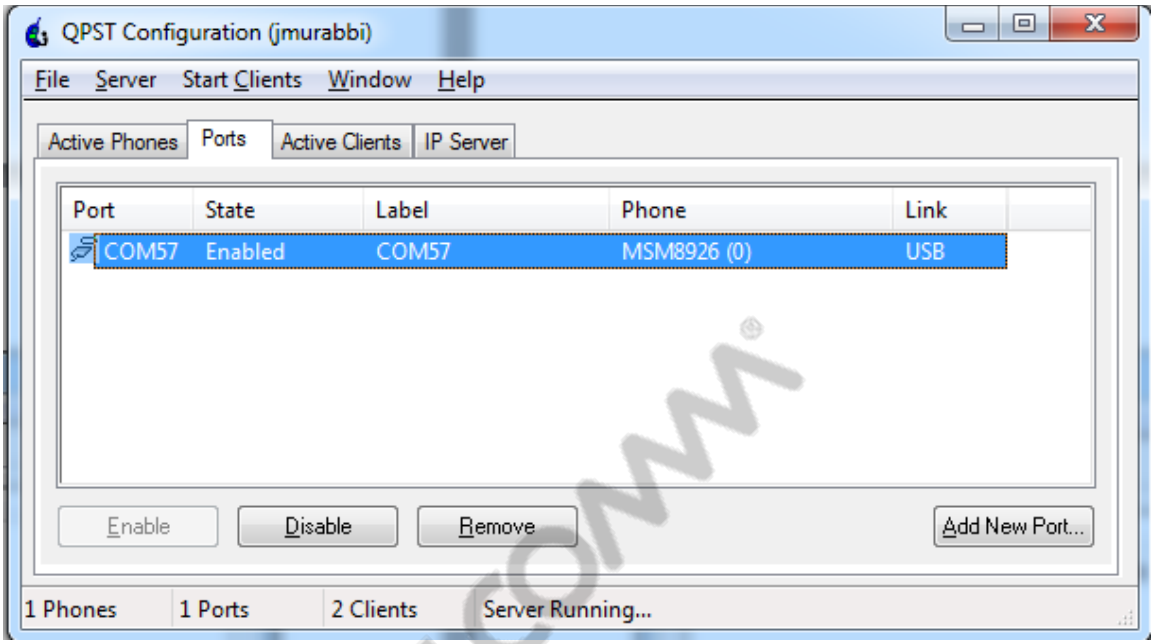
The registered handler will generate the response and if received by QPST, the device port connection will be established.

For example – for 0x0C, the handler function is *mclog_stat()*



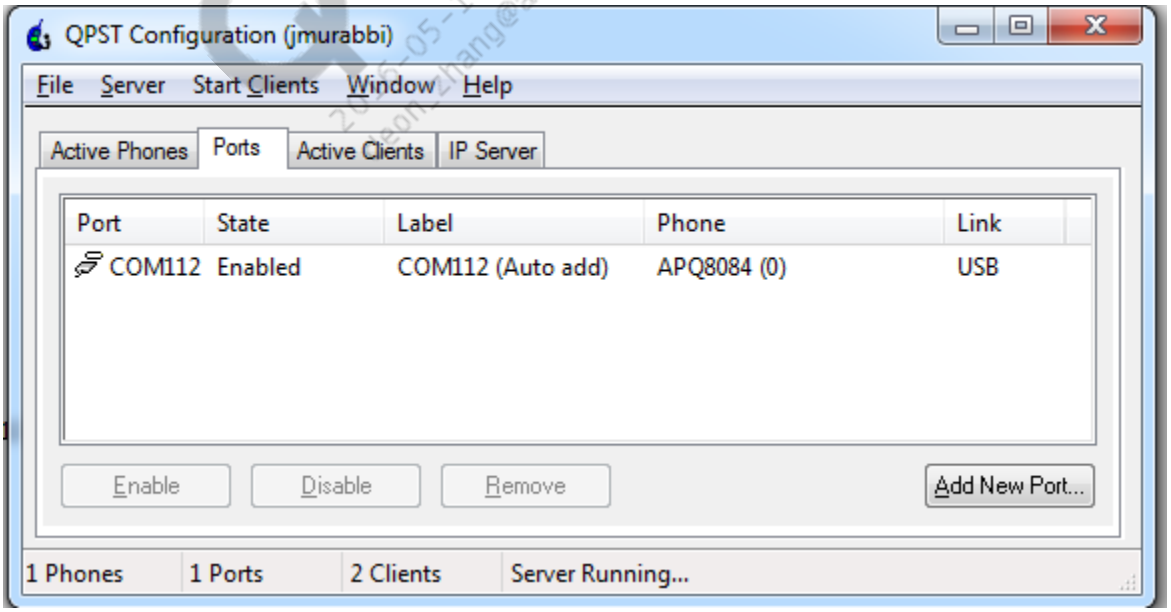
QUALCOMM®
2016-05-17 05:26:10 PDT
deon zhang@askey.com.tw

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION



Case 2 – MPSS is not available

An example of this case is APQ chipsets, which do not have a modem and thus the MPSS is not available.



If none of the MPSS polling commands are registered, the APSS (master Diag) will respond to following polling command:

- 0x4B 32 03 00

If the response is received by QPST, a connection will be established.

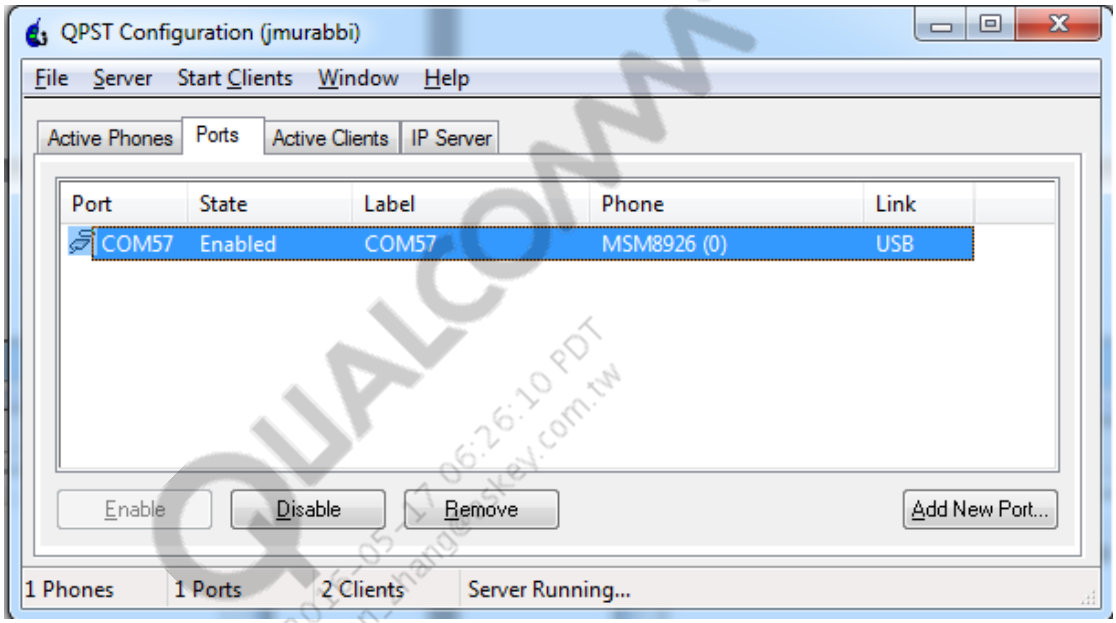
MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Case 3 – MPSS comes up late

- Phase 1 – If MPSS takes longer to come up, its polling commands will not be registered with the APSS during that time. At that point, the APSS should respond to its own polling command:

  ```
  0x4B 0x32 03 00
  ```

  The QPST connection will be established at this time.



- Phase 2 – Once MPSS is available, it will register its commands with APSS. APSS will then stop responding to its own polling command and forward the 0xC polling command to MPSS.

  QPST connection may be lost for a few seconds during this transition, but will quickly be re-established once MPSS sends a good response.

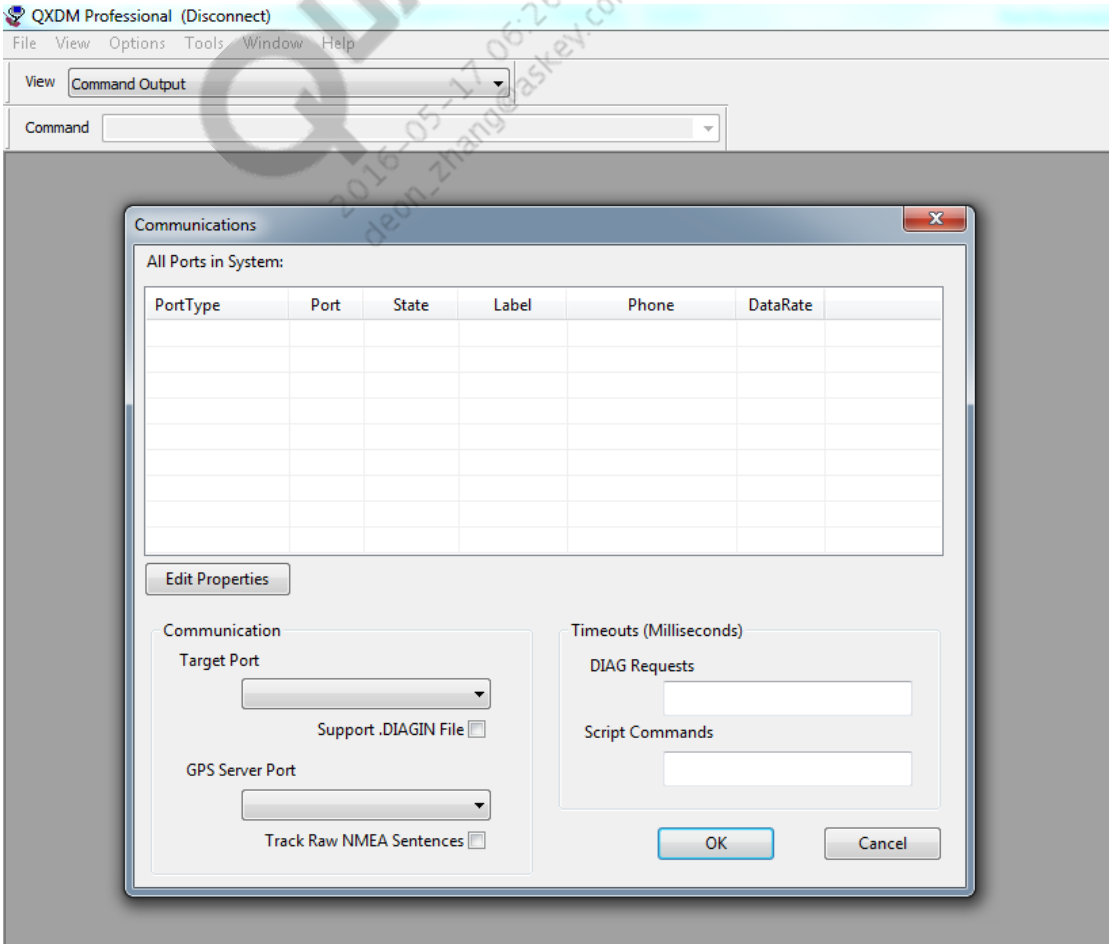
MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Case 4 – Subsystem restart is performed on MPSS

1. QPST connection is established with the 0xC command with the MPSS.
2. A command is sent to MPSS to perform subsystem restart.
3. MPSS closes SMD channels with APSS, which proceeds to remove the MPSS registrations from its slave table.
4. Subsequent 0xC requests from QPST result in an error response by the master Diag on the APSS. This forces QPST to retry the polling sequence.
5. Eventually the APSS polling request is sent and responded by the master Diag, and the QPST connection is re-established.
6. When MPSS comes back up, the algorithm follows, and the MPSS comes up late (case 3) above.
7. QPST connection may be temporarily lost for a few seconds during this transition, but will quickly be re-established.

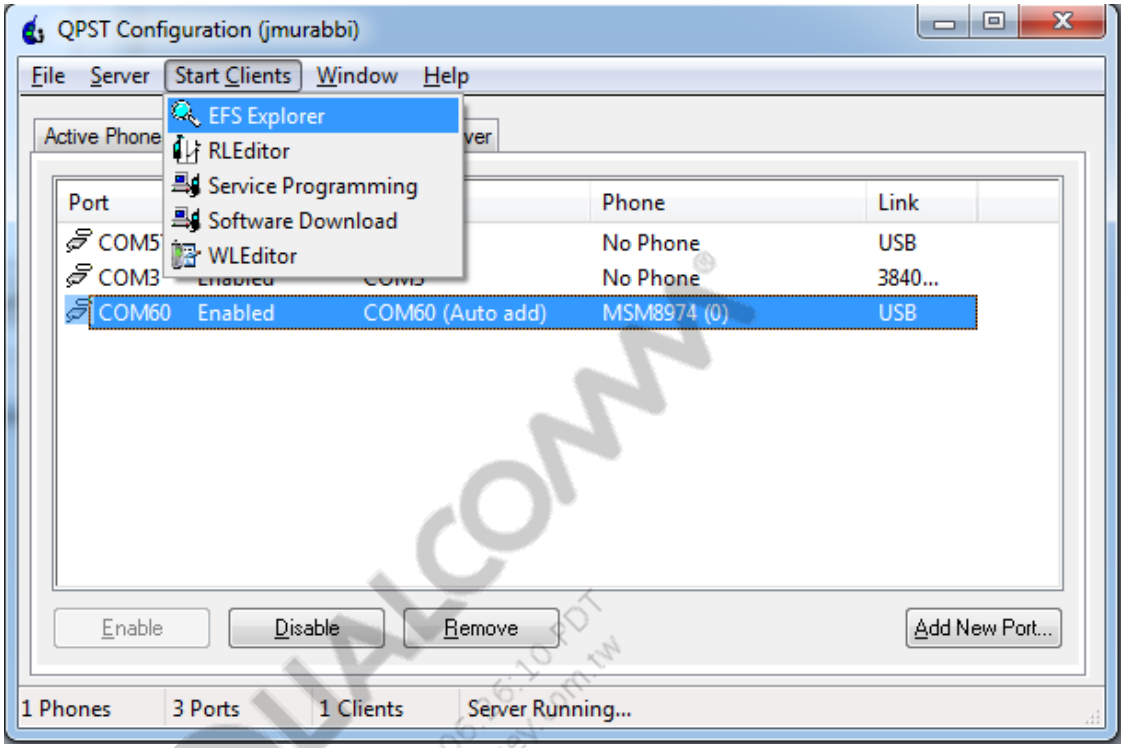
9.1.4 Check QXDM connection

If QPST shows device COM ports but no ports show up in QXDM Communication Port window, check for EFS support on device.

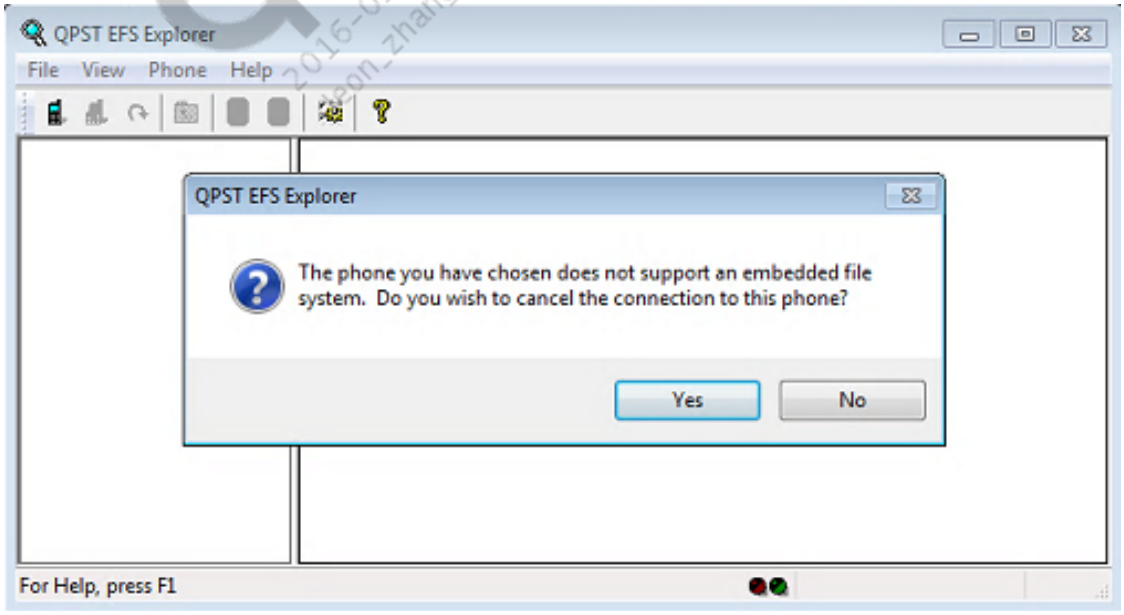


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

1. From the QPST Configuration tool, open EFS Explorer, Start Clients → EFS Explorer:



2. Check whether the following error dialog appears.



This indicates that the EFS on the DUT could be password-protected leading to no response for the EFS hello command. One way to send the password to DUT could be from QPST.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

9.1.5 Check device status

9.1.5.1 APSS

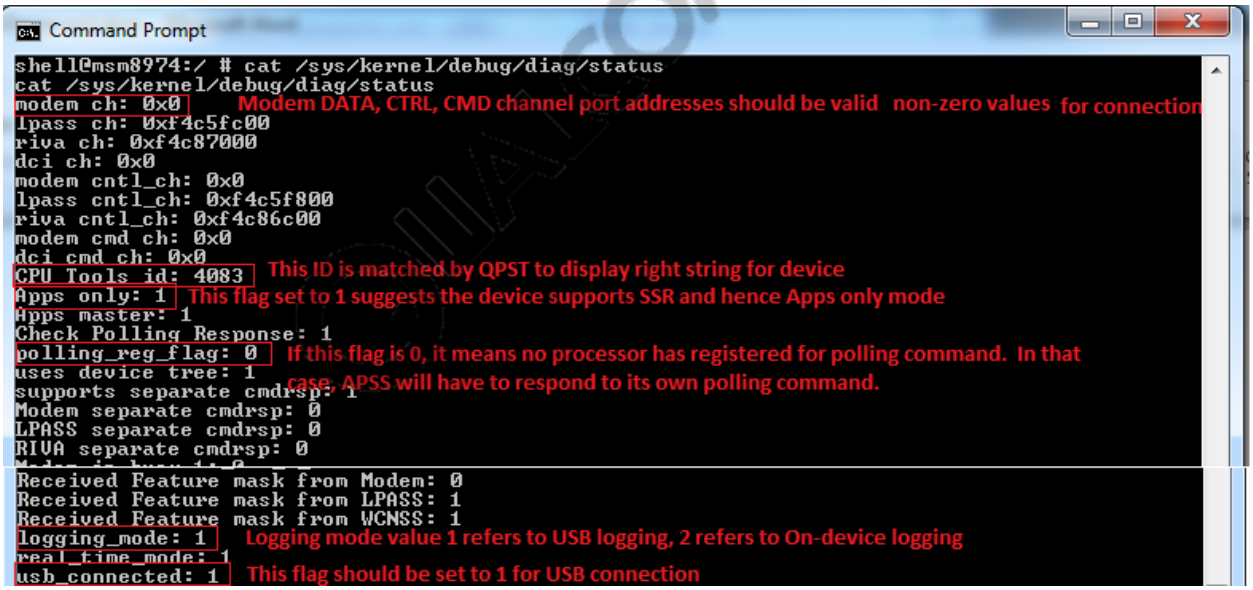
Check debugfs logs

1. Mount debugfs, if not already mounted with the commands:

```
mkdir /sys/kernel/debug
mount -t debugfs 0/sys/kernel/debug
```

2. Get the general diag status:

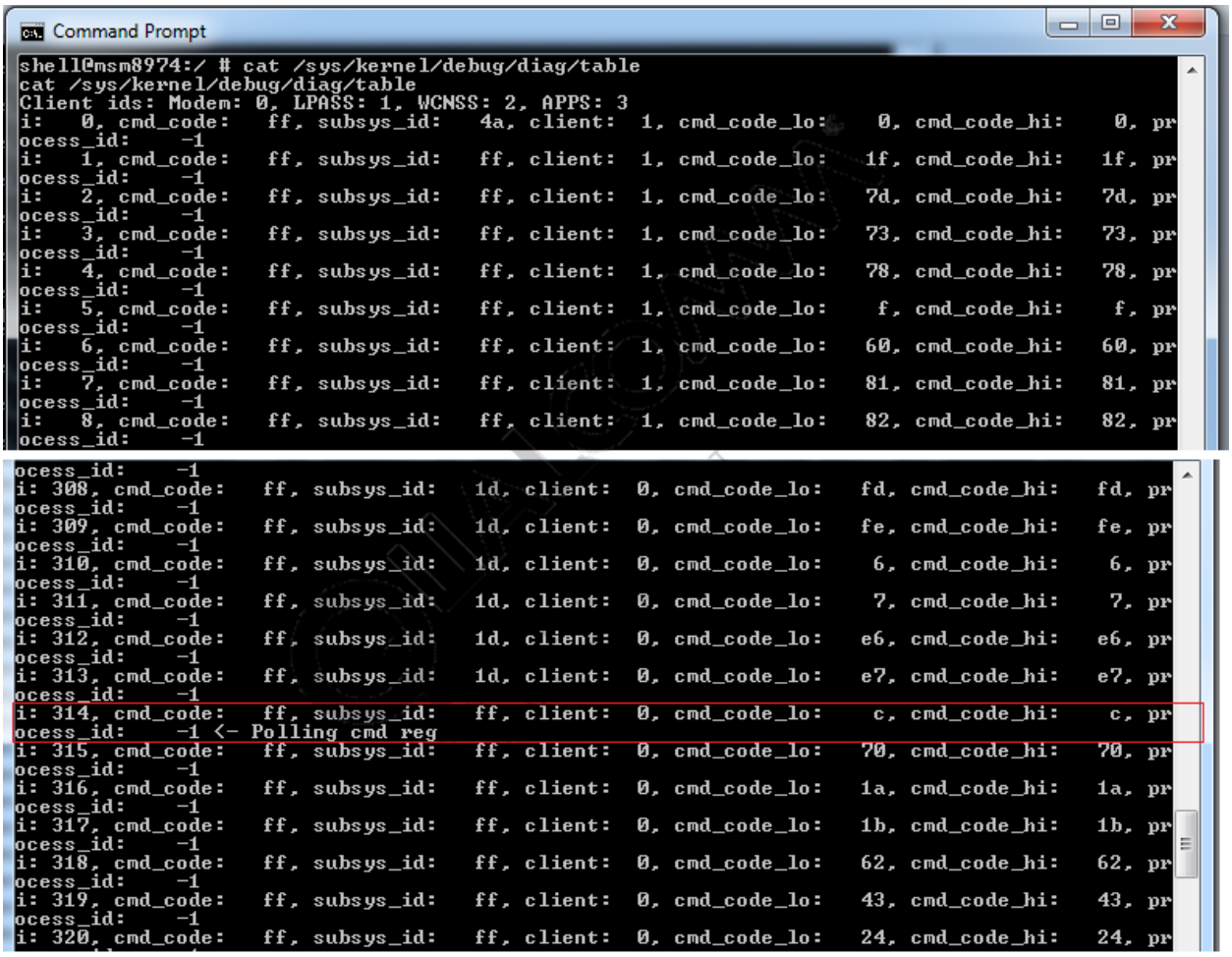
```
cat /sys/kernel/debug/diag/status
```



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3. Display the registration table, i.e., all messages that are currently registered with apps linux diag with the command:

   ```
   cat /sys/kernel/debug/diag/table
   ```



Highlighted entry is for MPSS polling command 0xC. Please note Cmd code and Subsys ID for this entry are set to 0xFF, and client value is 0 for MPSS.

For a command that is registered by an application on the applications processor, the registration table will show the Porcess ID information as:

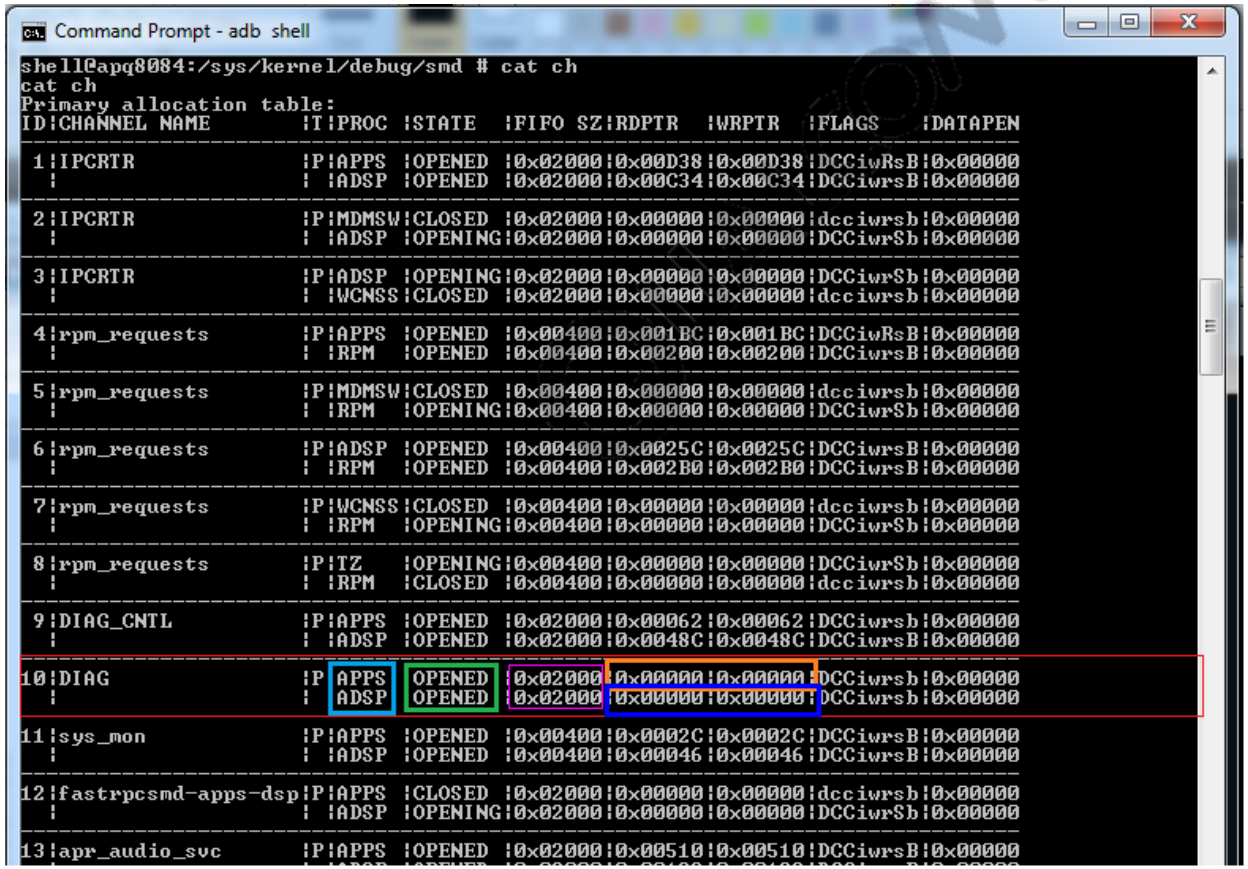
i: 655, cmd_code: ff, subsys_id: ff, **client: 3**, cmd_code_lo: 29, cmd_code_hi: 29, **process_id: 2664**

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4. Check for pending bytes on SMD channels the command:

Newer targets

```
cat /sys/kernel/debug/smd/ch
```



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Old targets

cd /sys/kernel/debug/smd

cat *

The table is shown as follows:

ch08: OPENED(0000/0000) DCCiwrsb <-> OPENED(0000/0000) DCCiwrsb : 2000

ch09: OPENED(0040/0040) DCCiwRsb <-> OPENED(0132/0132) DCCiwrsB : 2000

ch10: OPENED(0000/0000) DCCiwrsb <-> OPENED(0000/0000) DCCiwrsb : 2000

ch15: CLOSED(0000/0000) dcciwrsb <-> OPENING(0000/0000) DCCiwrSb : 2000

ch16: CLOSED(0000/0000) dcciwrsb <-> OPENING(0000/0000) DCCiwrSb : 2000

ch17: CLOSED(0000/0000) dcciwrsb <-> OPENING(0000/0000) DCCiwrSb : 400

ch18: OPENED(0000/0000) DCCiwrsb <-> OPENED(0000/0000) DCCiwrsb : 2000

ch19: OPENED(0952/0952) DCCiwrsb <-> OPENED(0507/0720) DCCiwrsB : 400

ch20: OPENED(6308/6308) DCCiwRsb <-> OPENED(4263/4263) DCCiwrsB : 2000

ch21: CLOSED(0000/0000) dcciwrsb <-> OPENING(0000/0000) DCCiwrSb : 2000

ch27: CLOSED(0000/0000) dcciwrsb <-> OPENING(0000/0000) dcciwrSb : 2000

ch28: OPENED(0000/0000) DCCiwrsb <-> OPENED(0256/0256) DCCiwrsB : 2000

ch29: CLOSED(0000/0000) dcciwrsb <-> OPENING(0000/0000) dcciwrSb : 2000

ch37: OPENED(0109/0109) DCCiwrsB <-> OPENED(0163/0163) DCCiwrsB : 400

name=DIAG_CNTL cid=7 ch type=1 xfer type=2 ref_count=2

name=DIAG cid=8 ch type=1 xfer type=2 ref_count=2

name=apr_audio_svc cid=9 ch type=1 xfer type=2 ref_count=1

name=SSM_RTR cid=16 ch type=0 xfer type=2 ref_count=1

name=sys_mon cid=17 ch type=0 xfer type=2 ref_count=1

name=DIAG_2 cid=18 ch type=0 xfer type=2 ref_count=2

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

name=DIAG_CNTL cid=19 ch type=0 xfer type=2 ref_count=2

name=DIAG cid=20 ch type=0 xfer type=2 ref_count=2

name=apr_voice_svc cid=21 ch type=0 xfer type=2 ref_count=1

name=DATA1 cid=22 ch type=0 xfer type=2 ref_count=2

In the above table, it is necessary to first find the number of the channel of interest and see if it is open.

For example: DIAG_CTNL is channel number 19, which is shown as OPEN.

Below is a list of channels supported by Diag and DCI:

DCI and DIAG share control channel

DIAG_2 and DIAG_2_CMD ::: DCI Channels

DIAG and DIAG_CMD ::: DIAG Channels

DIAG_CTRL: This is the control channel that is shared by both DCI and Diag

```
-------------------------------------------------------------------------------
 4|DIAG_2_CMD         |P|APPS |OPENED |0x02000|0x00000|0x00000|DCCiwrsb|0x00000
  |                   | |MDMSW|OPENED |0x02000|0x00000|0x00000|DCCiwrsb|0x00000
-------------------------------------------------------------------------------
 5|DIAG_2             |P|APPS |OPENED |0x02000|0x00000|0x00000|DCCiwrsb|0x00000
  |                   | |MDMSW|OPENED |0x02000|0x00000|0x00000|DCCiwrsb|0x00000
-------------------------------------------------------------------------------
 6|DIAG_CNTL          |P|APPS |OPENED |0x02000|0x00108|0x00108|DCCiwrsb|0x00000
  |                   | |MDMSW|OPENED |0x02000|0x0057D|0x0057D|DCCiwrsB|0x00000
-------------------------------------------------------------------------------
 7|DIAG_CMD           |P|APPS |OPENED |0x02000|0x00000|0x00000|DCCiwrsb|0x00000
  |                   | |MDMSW|OPENED |0x02000|0x00000|0x00000|DCCiwrsb|0x00000
-------------------------------------------------------------------------------
 8|DIAG               |P|APPS |OPENED |0x02000|0x00000|0x00000|DCCiwrsb|0x00000
  |                   | |MDMSW|OPENED |0x02000|0x00026|0x00026|DCCiwrsB|0x00000
-------------------------------------------------------------------------------
```

All channels open at boot. If DCI channels are not often visible, it is because DCI is blocked on the modem by the OEM.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

How to check the status of a DCI application

If a DCI application is making use of DCI, DCI_STATS can be used to check its status from the ADB shell.

This is shown when DCI_STATS ir run.

```
shell@msm8974:/sys/kernel/debug/diag $ cat dci_stats
cat dci_stats
number of clients: 0
dci proc active: 0
dci real time vote: 1
usb_connected: 1
dci power: active, relax: 0, 0
shell@msm8974:/sys/kernel/debug/diag $
```

If a client is active, the number of clients is shown: 1.

The status is also displayed, as shown:

```
i 1770          s 3320          c 2          t 0:3:24:101403
i 1771          s 2915          c 2          t 0:3:24:111126
i 1772          s 3264          c 2          t 0:3:24:114923
i 1773          s 1194          c 2          t 0:3:24:115064
i 1774          s 3341          c 2          t 0:3:24:140192
i 1775          s 3292          c 2          t 0:3:24:140360
i 1776          s 3607          c 2          t 0:3:24:190669
i 1777          s 3457          c 2          t 0:3:24:200851
i 1778          s 3147          c 2          t 0:3:24:201049
i 1779          s 1194          c 2          t 0:3:24:201177
i 1780          s 3264          c 2          t 0:3:24:218984
i 1781          s 3312          c 2          t 0:3:24:219151
i 1782          s 3264          c 2          t 0:3:24:225782
i 1783          s 3147          c 2          t 0:3:24:235949
i 1784          s 3147          c 2          t 0:3:24:236120
i 1785          s 3147          c 2          t 0:3:24:236267
i 1786          s 1222          c 2          t 0:3:24:236405
i 1787          s 3610          c 2          t 0:3:24:249904
i 1788          s 1194          c 2          t 0:3:24:250074
i 1789          s 3239          c 2          t 0:3:24:298925
i 1790          s 3292          c 2          t 0:3:24:309130
i 1791          s 2073          c 2          t 0:3:24:309307
i 1792          s 3264          c 2          t 0:3:24:325204
i 1793          s 3454          c 2          t 0:3:24:325412
i 1794          s 2243          c 2          t 0:3:24:325552
```

Dci_stats displays the last 100 reads on the DCI channel, the size at the timestamp for the incoming data. When DCI_STATS is run once, it may not print all the 100 iterations because the input buffer is just 4 kB. Thus, this is covered over two writes. There is a possibility that the data repeats when this is done several times.

This can be sorted by iteration number, which reveals whether DCI packets are being allocated.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

9.1.5.2 APSS callflow

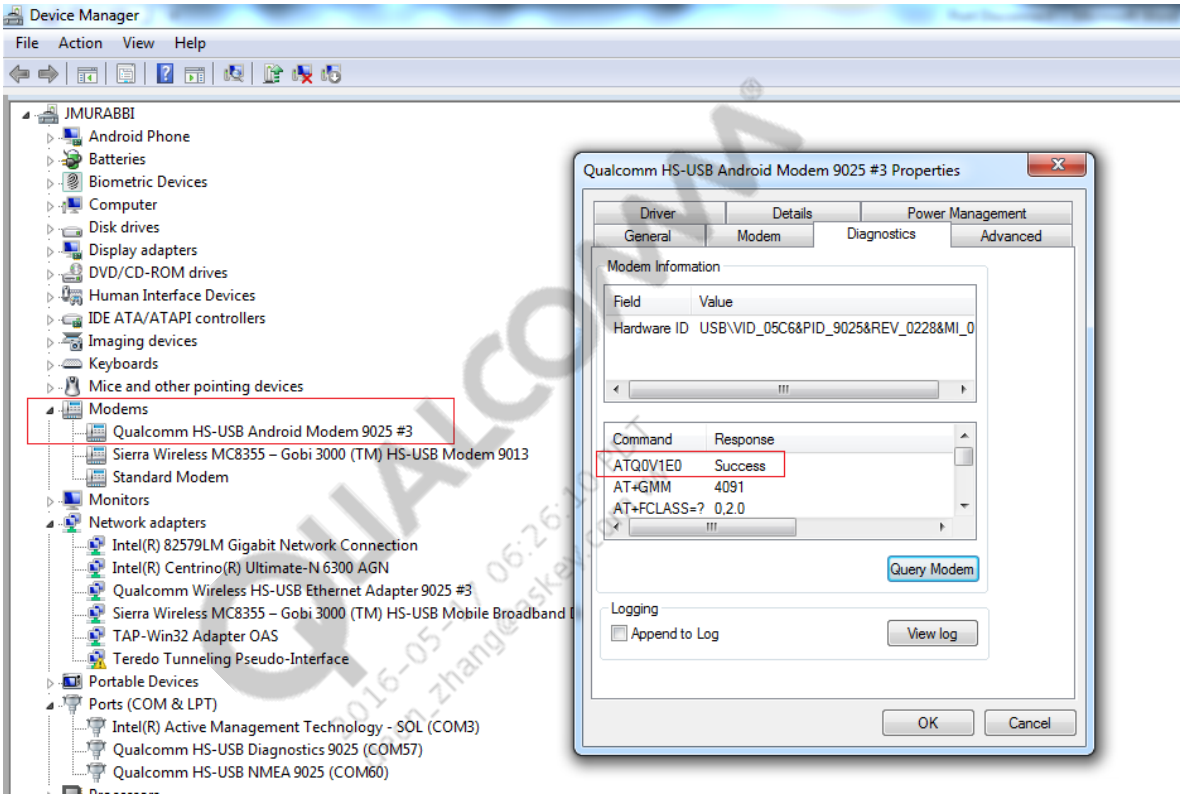
```
<6>[ 1943.837744] I[6:      swapper/6:    0] android_usb gadget: usb_diag_read  ← USB Enqueus read request
on OUT endpoint
<6>[ 1943.838988] I[6:      swapper/6:    0] android_usb gadget: diag_read_complete  ← READ DONE
received, USB layer will notify Diag
<6>[ 1943.839156]  [6:  kworker/u16:9: 7897] diag: HDLC decode fn, len of data  4  ← Diag was
notified
K;l;
```

QUALCOMM®
2016-05-17 06:26:10 PDT
deon_zhang@askey.com.tw

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

9.1.5.3 MPSS

1. Check whether the modem is running:
   - If the AT is enabled on the USB port, go to Device Manager → Modems → (Right click) Qualcomm USB Modem → Properties → Diagnostics → Query Modem

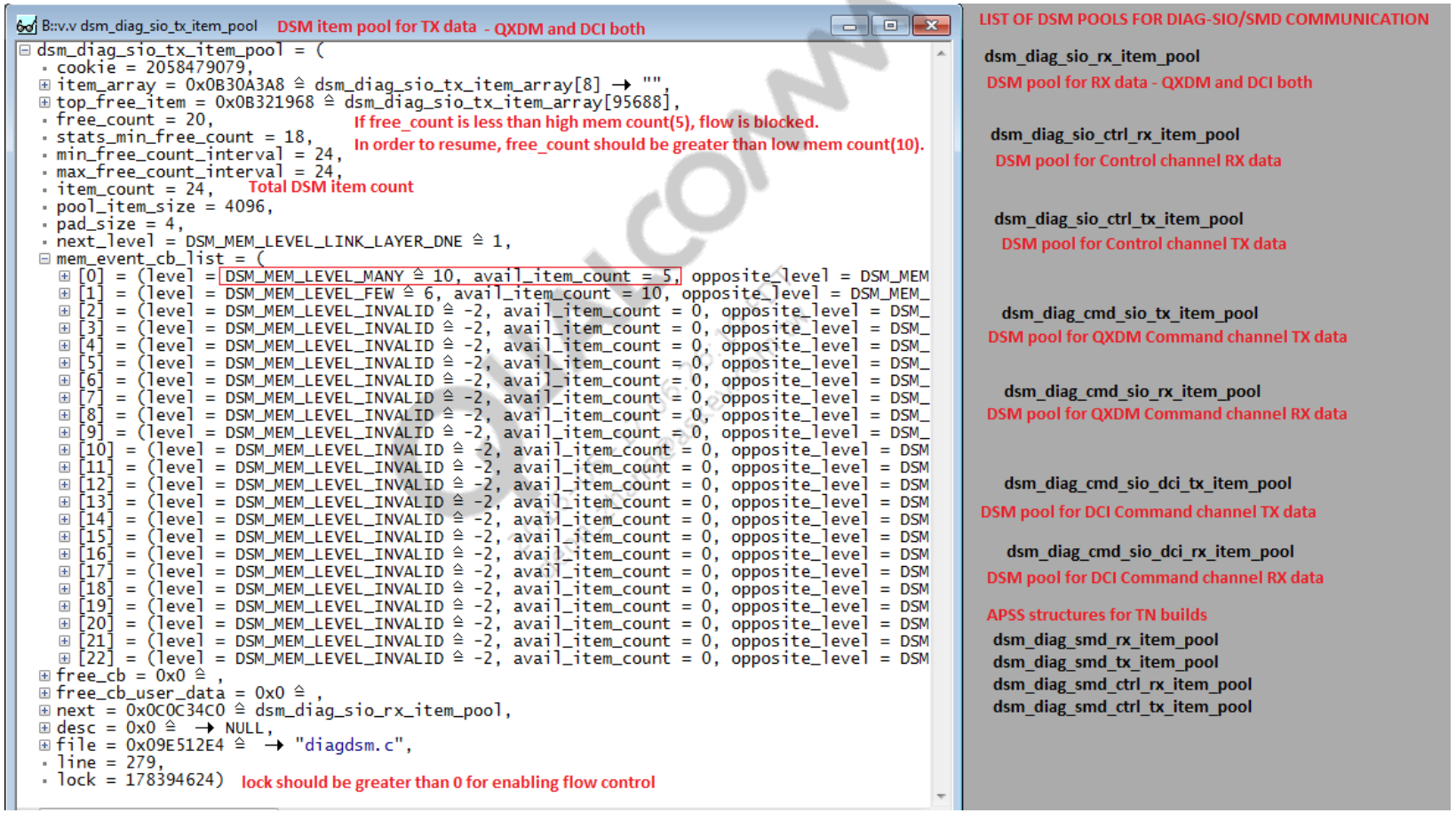


If the response is success, the modem is running.

- Use an alternate method to check the modem status; for example, make a voice call.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

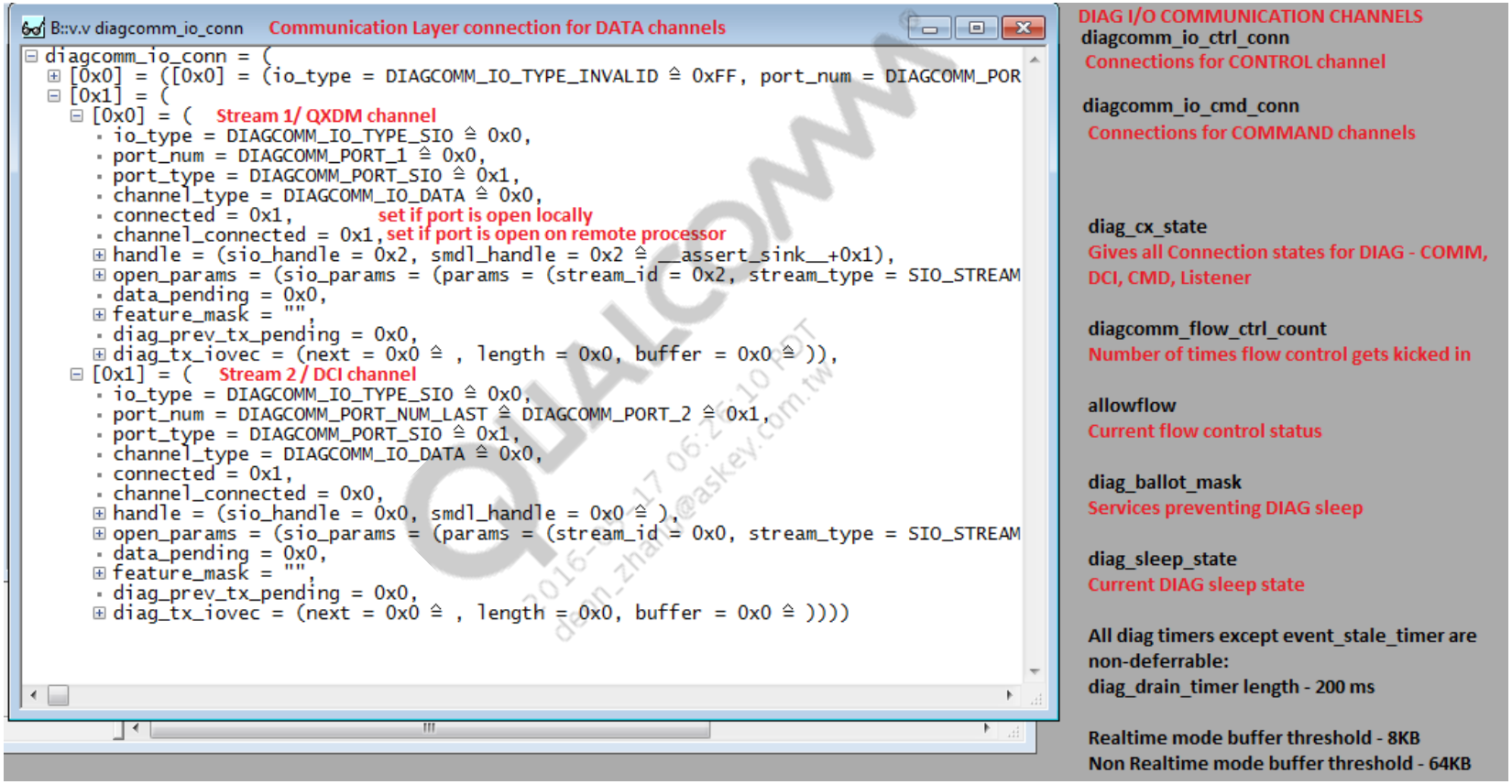
2. Check the MPSS Diag structures:
   - DSM pools – dsm_diag_cmd_sio_tx_item_pool, dsm_diag_cmd_sio_rx_item_pool



NOTE: item_count is the maximum number of items a pool can handle. free_count is the number of items that still need to be filled. It can be seen that only 8 are left, which means there are some items to be drained. If the pool is empty, free_count should equal item_count.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

- Communication layer connection – diagcomm_io_cmd_conn



9.2 Linux APSS

The global structure driver is the main data structure and can be viewed in Trace32 with the command

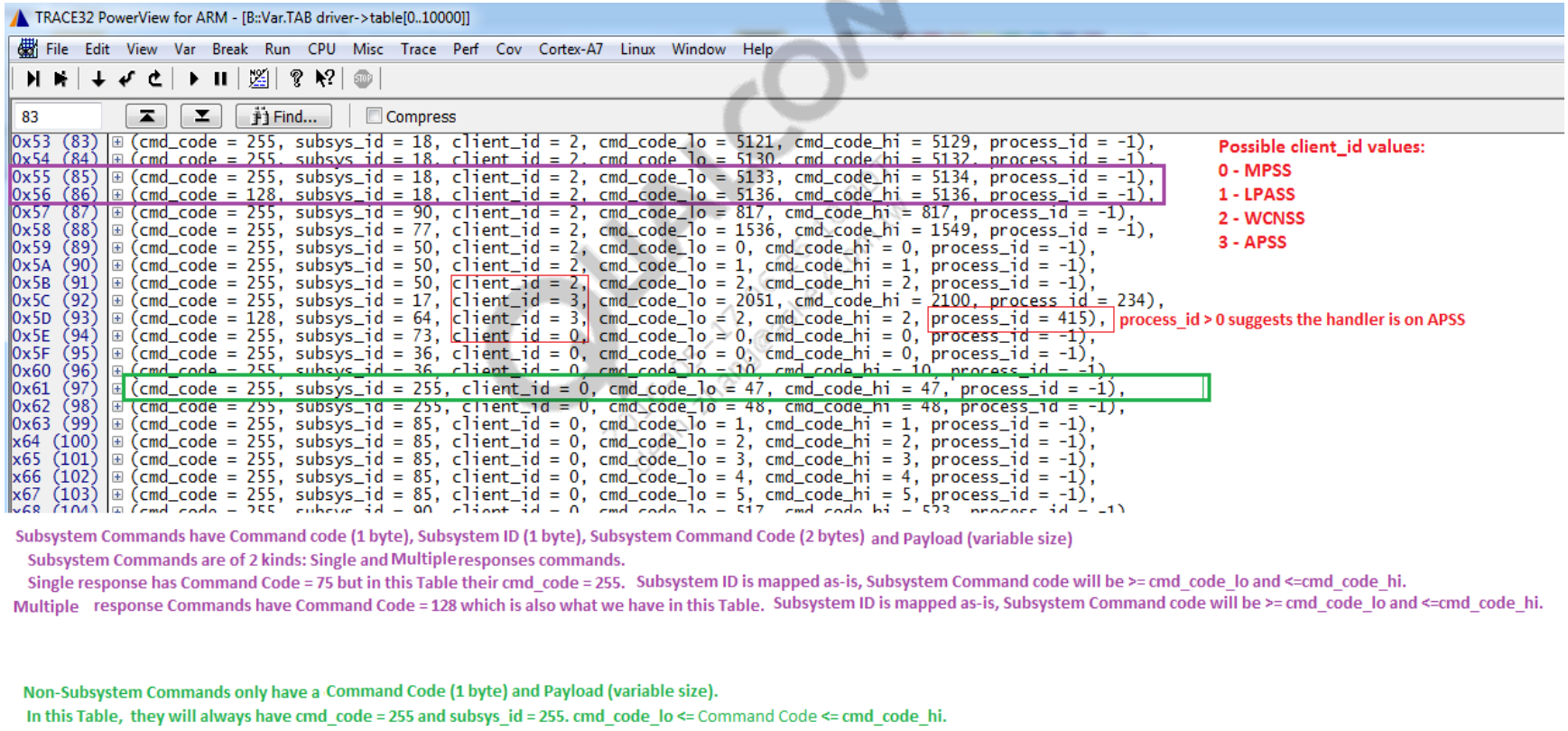
```
struct diagchar_dev *driver
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

9.2.1 Command/Response routing

APSS maintains a master dispatch table that tracks all commands to be serviced locally as well as by remote processors. For an incoming command request, it checks this table. If the request is not found in the table, an error response is sent to the client. Otherwise, the request is forwarded to the servicing handler.

SSIDs and commands can be seen under driver→table.



NOTE: If a command entry is not found in dispatch table, a response packet starting with 0x13 is sent by Diag.

NOTE: If a command is found in the dispatch table, and the command handler either returns a NULL response or fails to resturn any response packet, no response will be sent by Diag.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

9.2.1.1 Modem commands – loopback

Example – Modem loopback command 75 18 41 0 1 2 3

- Command code (1 byte) − 75
- Subsystem ID (1 byte) − 18
- Subsystem command code (2 bytes) − 41 0
- Payload (N bytes) − 1 2 3

Table 9-1 shows the modem loopback command entry in the command dispatch table (driver→table).

Table 9-1 Modem loopback command structure in APSS Linux dispatch table

cmd_code	subsys_id	client_id	cmd_code_lo	cmd_code_hi	process_id
255	18	0	41	41	-1

The applications processor forwards this command request to the modem according to client_id.

9.2.1.2 APSS commands – reset

Example – Mode reset command 41 02 00

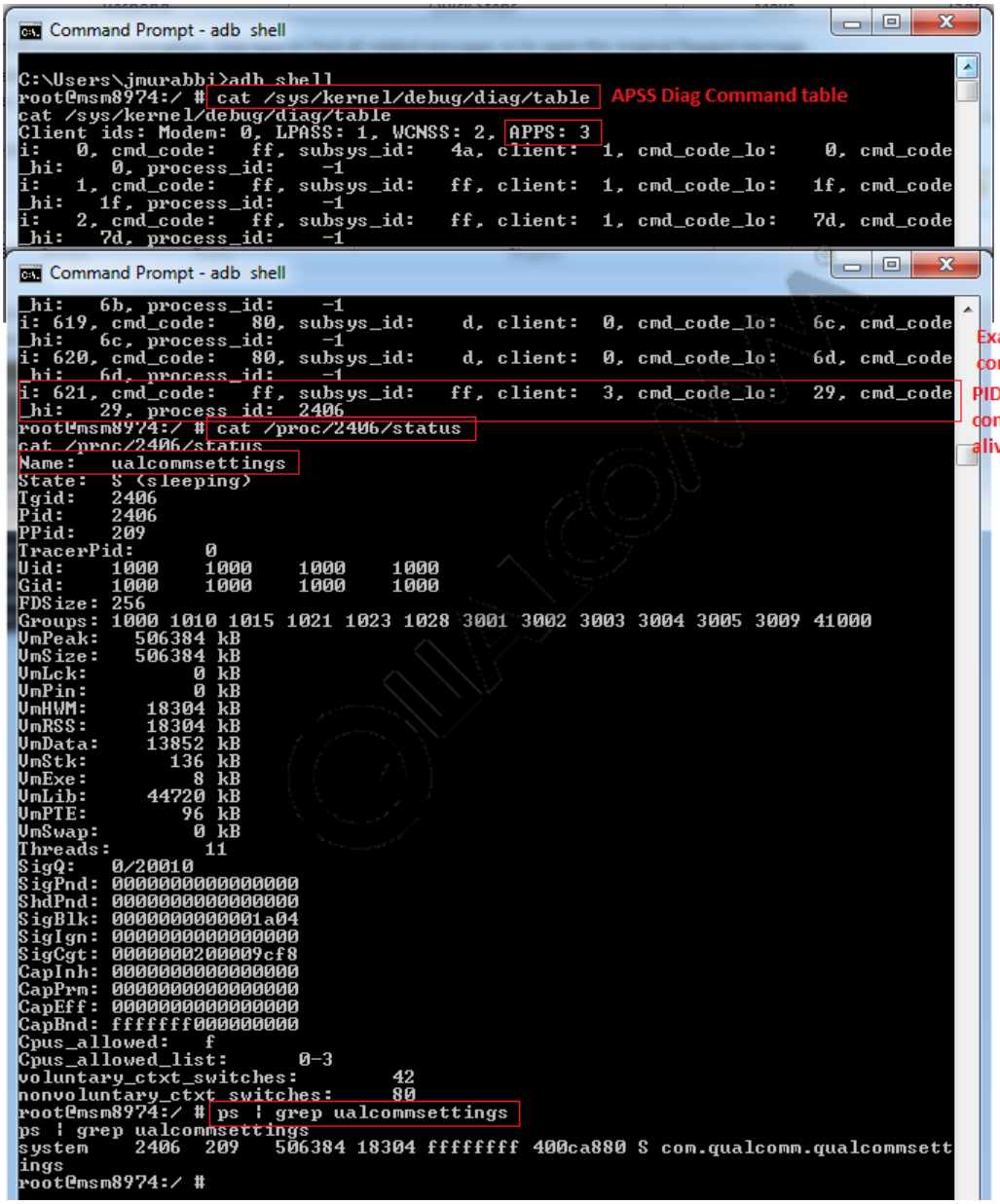
Table 9-2 shows the mode reset command structure in APSS Linux dispatch table.

Table 9-2 Mode reset command structure in APSS Linux dispatch table

cmd_code	subsys_id	client_id	cmd_code_lo	cmd_code_hi	process_id
255	255	3	41	41	2406

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

To troubleshoot this command, check whether process ID 2406 is alive.



The handler for the Reset command is defined in

`/vendor/qcom/proprietary/QualcommSettings/jni/DiagJNIInterface.cpp`

```
PACK(void *) android_diag_handler (PACK(void *)req_pkt, uint16 pkt_len)
{
    LOGD("android_diag_handler called!\n");
    /* TODO: Decode diag req_pkt */
    void *rsp_pkt = diagpkt_alloc(DIAG_CONTROL_F, pkt_len);
    if(rsp_pkt) {
        memcpy(rsp_pkt, req_pkt, pkt_len);
    }
    diag_callback_handler("reboot");
    return rsp_pkt;
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

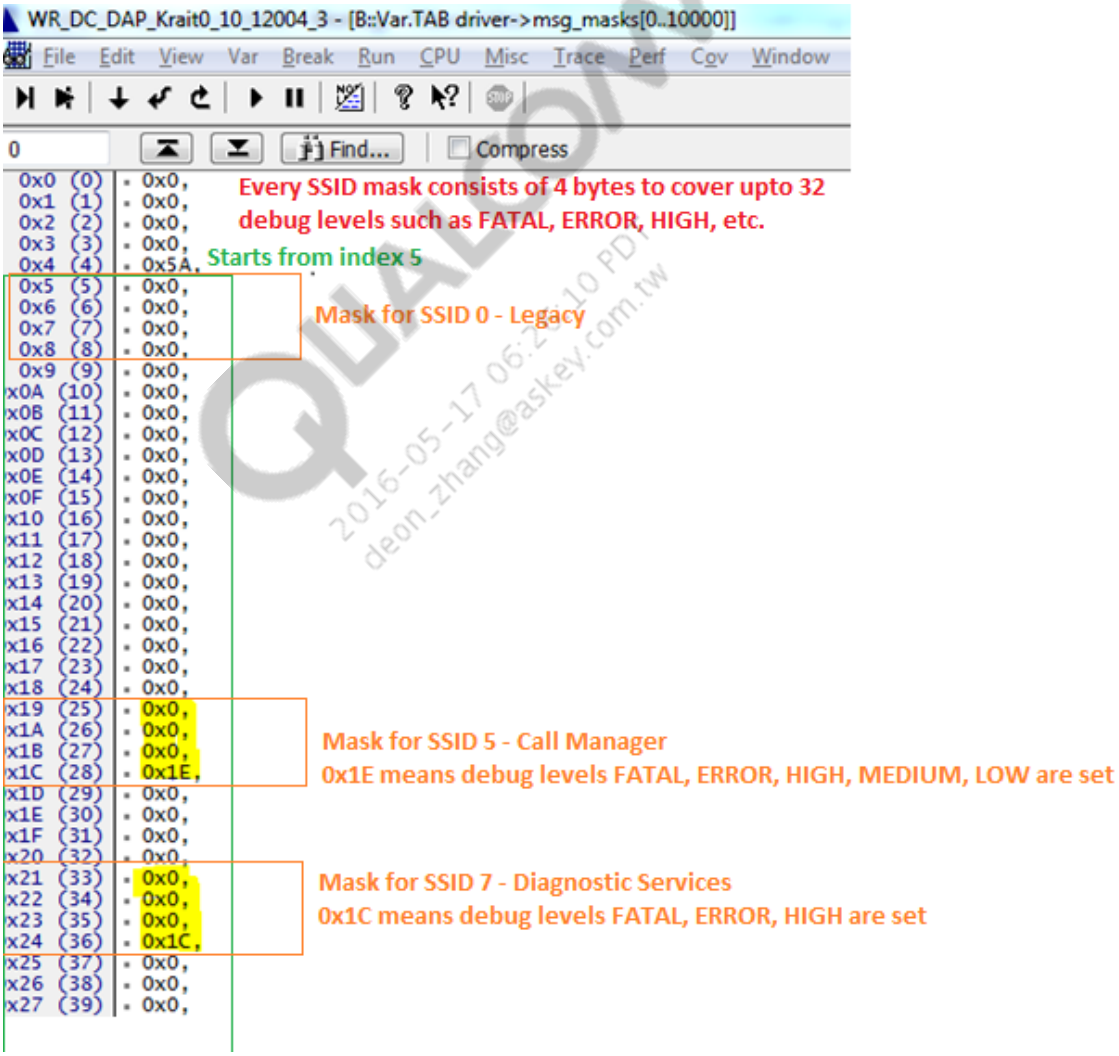
```
}

static const diagpkt_user_table_entry_type android_diag_handler_tbl[] =
{
    {DIAG_CONTROL_F, DIAG_CONTROL_F, android_diag_handler},
};
```

9.2.2 F3 masks

F3 masks can be seen as driver→msg_masks.

As shown here, every SSID is composed of four bytes. The masks start from index 5. Highlighted entries refer to the Call Manager (SSID 5) and Diagnostic Services (SSID 7) masks.



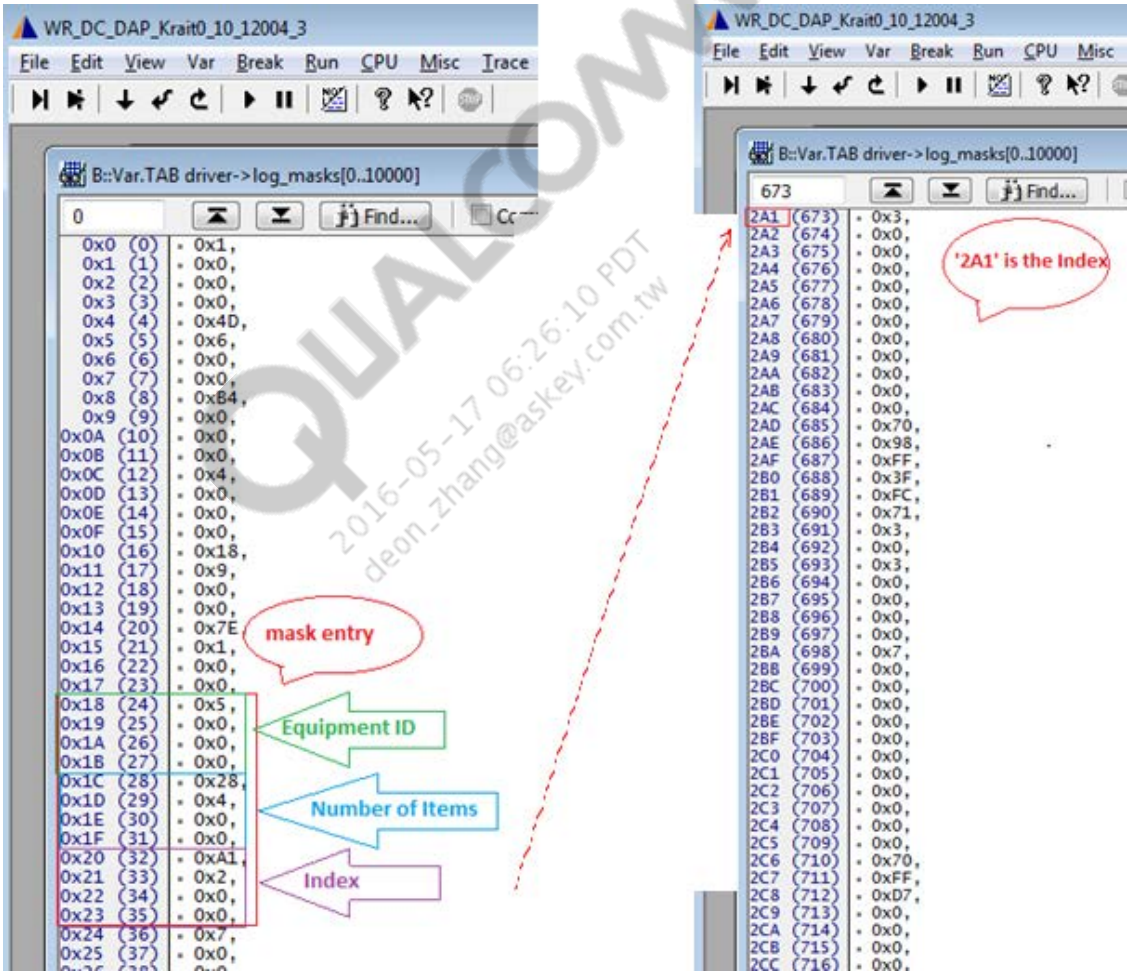
MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

9.2.3 Log masks

The log mask information is stored in driver→log_masks sequentially as mask entries. Each mask entry has the following members:

- equip_id (4 bytes) – Log equipment ID
- num_items (4 bytes) – Number of mask items
- index (4 bytes) – Index within the mask array to find the mask information for this equipment ID

In the following screenshot, use the index to find mask information for this particular equip_id within driver→log_masks.



QUALCOMM®
2016-05-17 05:26:10 PDT
deon_zhang@askey.com.tw

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

9.2.4 Kernel logs

To see or collect the kernel messages, enter either of the following two commands from a command window:

- The first command displays the kernel messages to the command window and places them in the kmsg.txt file. This command will continue running and updating with the latest messages as they occur. To stop, press **Ctrl+C** in the window.

```
adb shell cat /proc/kmsg | tee kmsg.txt
```

- The second command displays the kernel messages to the command window and places them in the dmesg.txt file. This command shows the kernel messages that currently exist and after displaying them, stops and returns to the command line.

```
adb shell dmesg | tee dmesg.txt
```

If there are difficulties using the Diag user space library when running a user space application, make sure the application was successful in its call to initialize the Diag user space library by checking the return value from Diag_LSM_Init():

- 1 (TRUE) – Call is successful
- 0 (FALSE) – Initialization has failed

One of the main causes of failure is permissions. The /dev/diag port has 660 permissions, which allows user and group read/write permissions. Running the application with any of the following should allow access to the /dev/diag port:

- Run as user root
- Run as user system
- Run with qcom_diag group permissions

9.2.5 Logcat

Logcat displays the user mode logs. In order to view only Diag logcat messages on the command prompt, use the following command:

Adb shell

Logcat –b main | grep –i diag

9.3 Modem processor

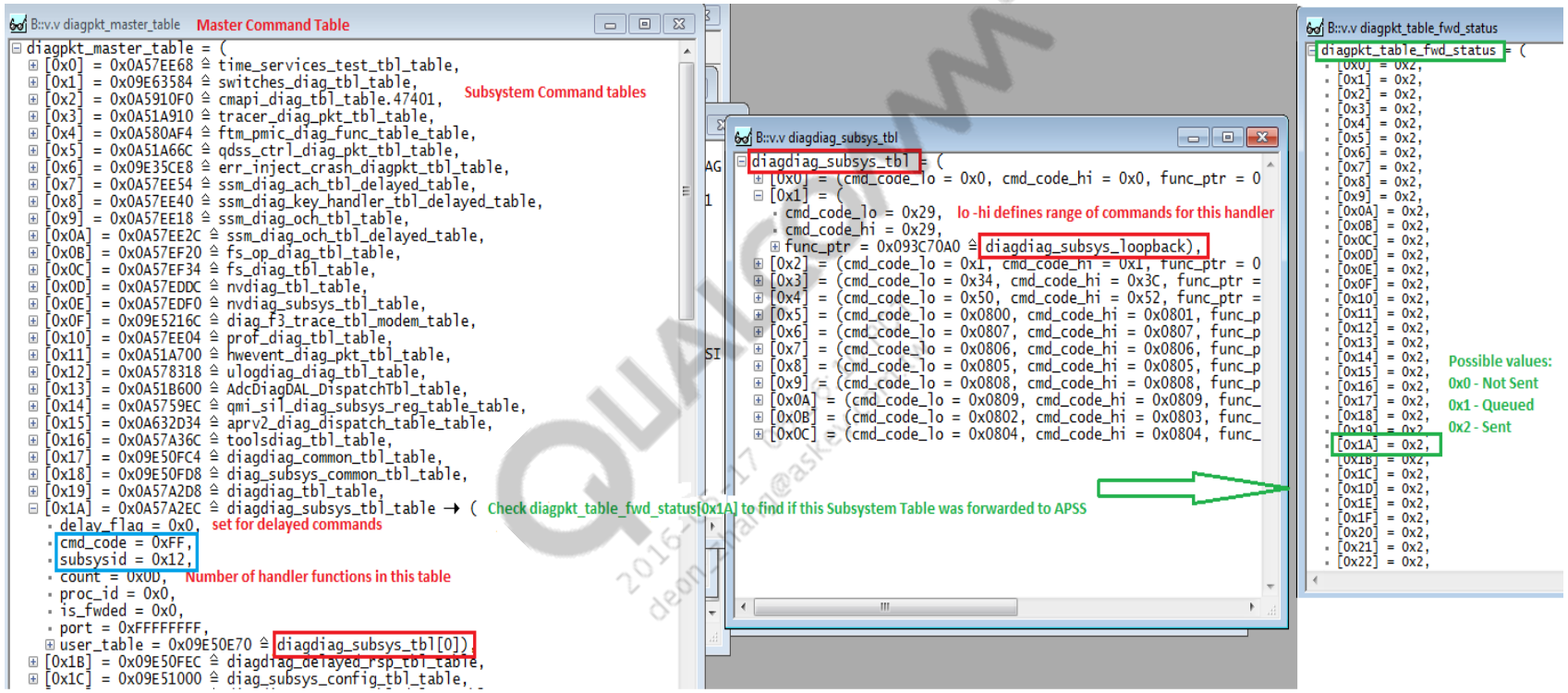
9.3.1 Command/Response routing

The modem maintains all registered command handlers in diagpkt_master_table structure.

diagpkt_table_fwd_status can be checked to find if a Subsystem Command table was forwarded to the APSS.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Example – Modem Loopback command 75 18 41 0 1 2 3



Single response Commands(Command code =75) have 'cmd_code' = 0xFF/255 in this Table. Subsystem ID is mapped as-is. cmd_code_lo <= Subsystem Command code <= cmd_co_hi.
'func_ptr' gives the handler API.
The mapping shown in this picture is for Modem Loopback command - 75 18 41 0 1 2 3

Multiple response Commands have the correct Command code in this Table = 0x80/128. Subsystem ID is mapped as-is. cmd_code_lo <=Subsystem Command code <= cmd_code_hi.
'func_ptr' gives the handler API.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

9.3.2 Log packet generation

Log packets (see Section 2.3.4) are generated by a client and sent to Diag for forwarding. Figure 9-1 shows the procedure. The client first checks to see if a mask for the desired log has been set. If so, it allocates memory from the Diag buffer. If there is insufficient memory, the log packet is dropped by the client.

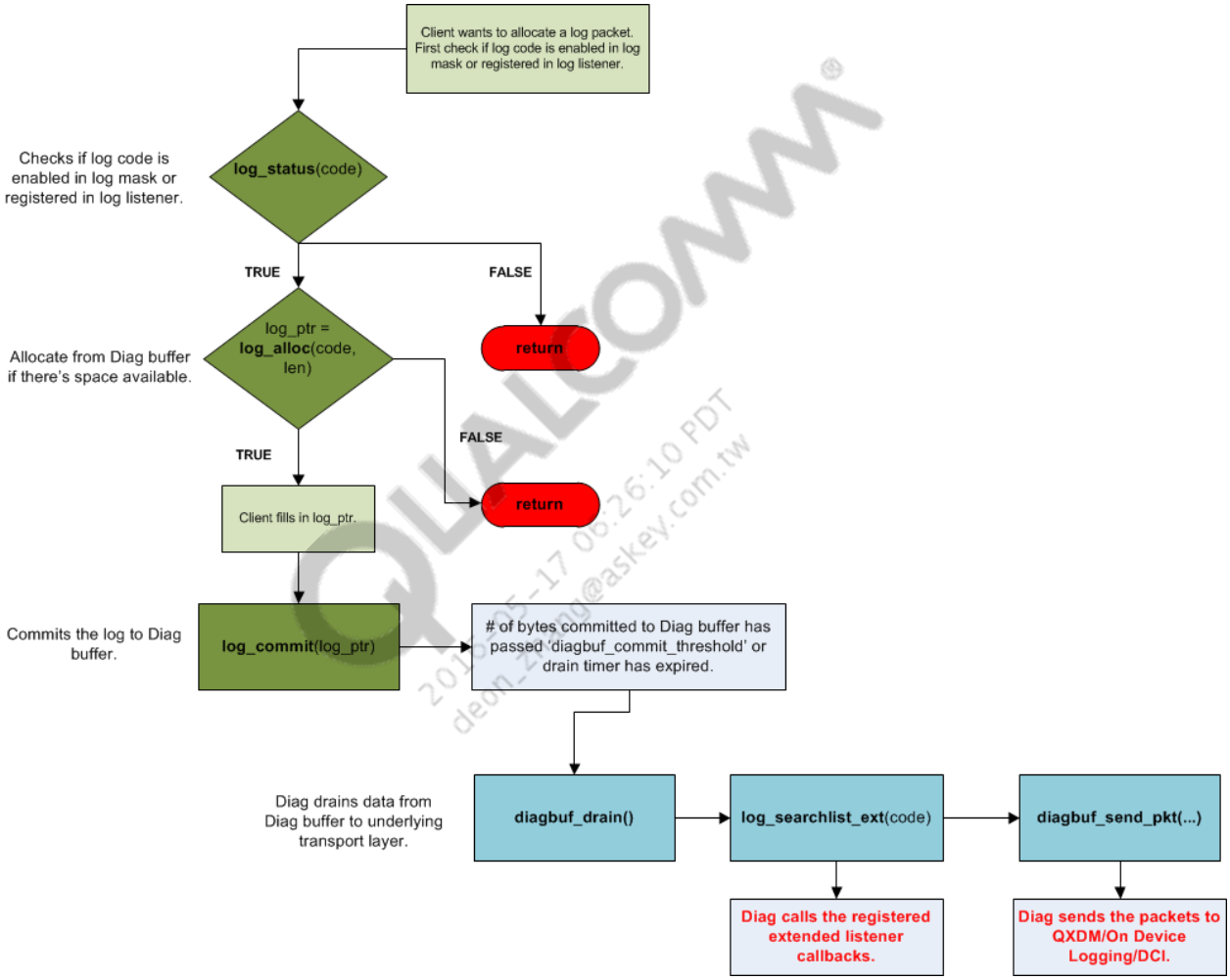


Figure 9-1 Log packet generation call flow

9.3.2.1 Log mask structures

- log_composite_mask – Composite log mask for all streams
- log_listener_mask – Single Log Listeners' mask
- log_listener_ext_mask – Extended Log Listeners' mask
- diag_log_preset_mask[0] – QXDM Pro/On-device logging stream log mask
- diag_log_dci_mask – DCI stream log mask

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

9.3.2.2 Checking the log mask value for a log code

The log code is 2 bytes as follows:

- Equipment ID – Most significant 4 bits
- Item number – Least significant 12 bits

To check the log mask value for a log code, find the correct index in the above log mask arrays:

Index = log_mask_offset_tbl[equipment id] + (Item number/8)

Log code = 0x11EB example:

- Equipment ID = 1
- Item number = 1EB
- log_mask_offset_tbl[1] = 1
  - 491/8 = 61 with remainder of 3
  - 8 * 61 = 491 – 488 = 3
  - Index of 1 + 61 = 62
  - Corresponds to the third bit in log_composite_mask[62]
- log_composite_mask[62] = 00001**0**00.
- Since the value in the third bit is 0, the mask value for this log code is not set.

QUALCOMM 2016-05-17 16:25:10 PDT deon_zhang@askey.com.tw

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

9.3.3 Event report generation

Event report generation is similar to log packet generation. The client checks for masks, requests memory, and submits event reports to Diag. Diag then forwards the reports to either the Listeners or QXDM/DCI client, as shown in Figure 9-2.

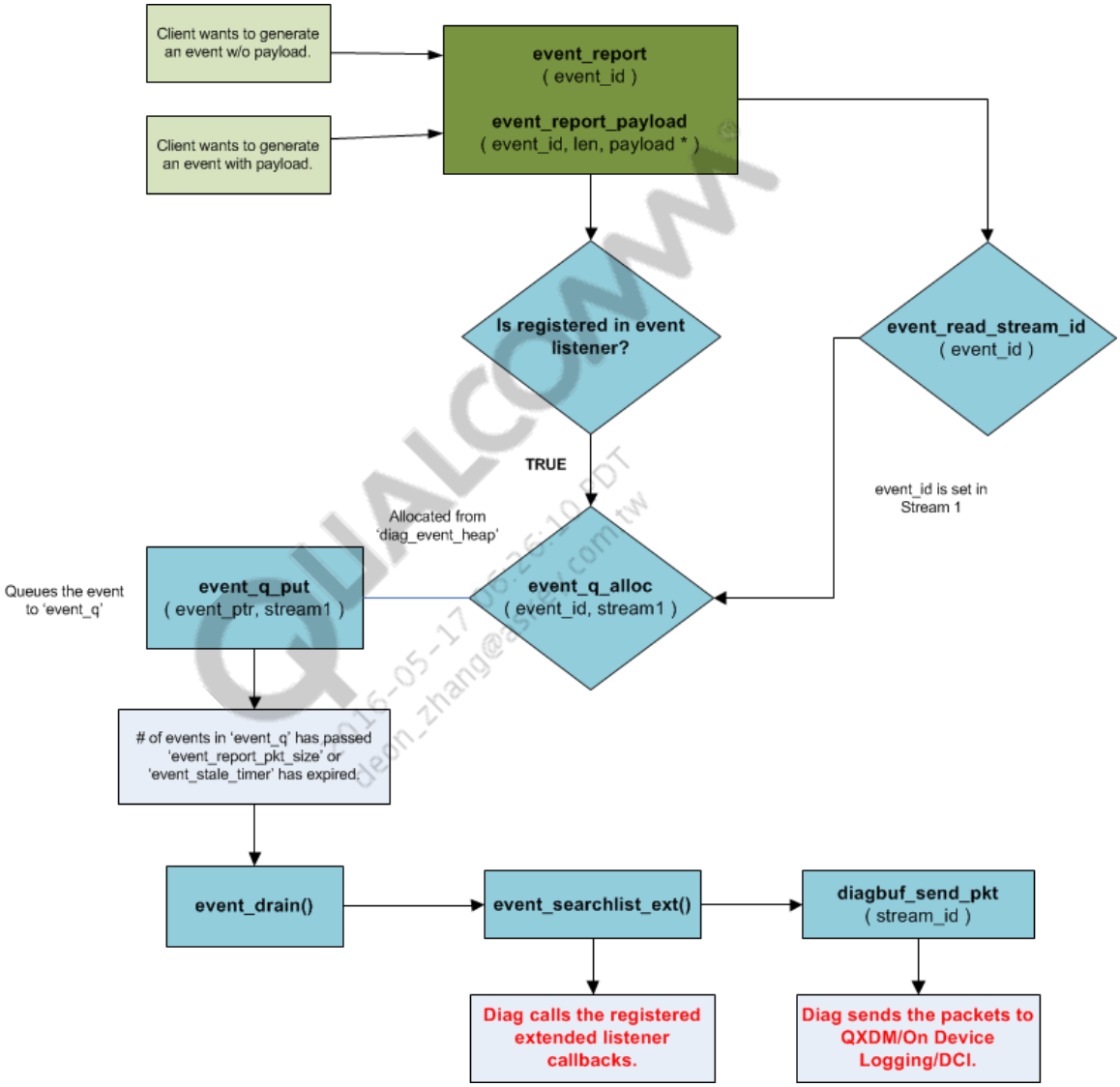


Figure 9-2 Event report generation call flow

9.3.3.1 Event mask structures

- diag_event_preset_mask[0] – QXDM Pro/On-device logging stream event masks
- diag_event_dci_mask – DCI stream event masks

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

9.3.3.2 Checking the event mask value for an event ID

Given an event ID, the index to the above array is calculated by:

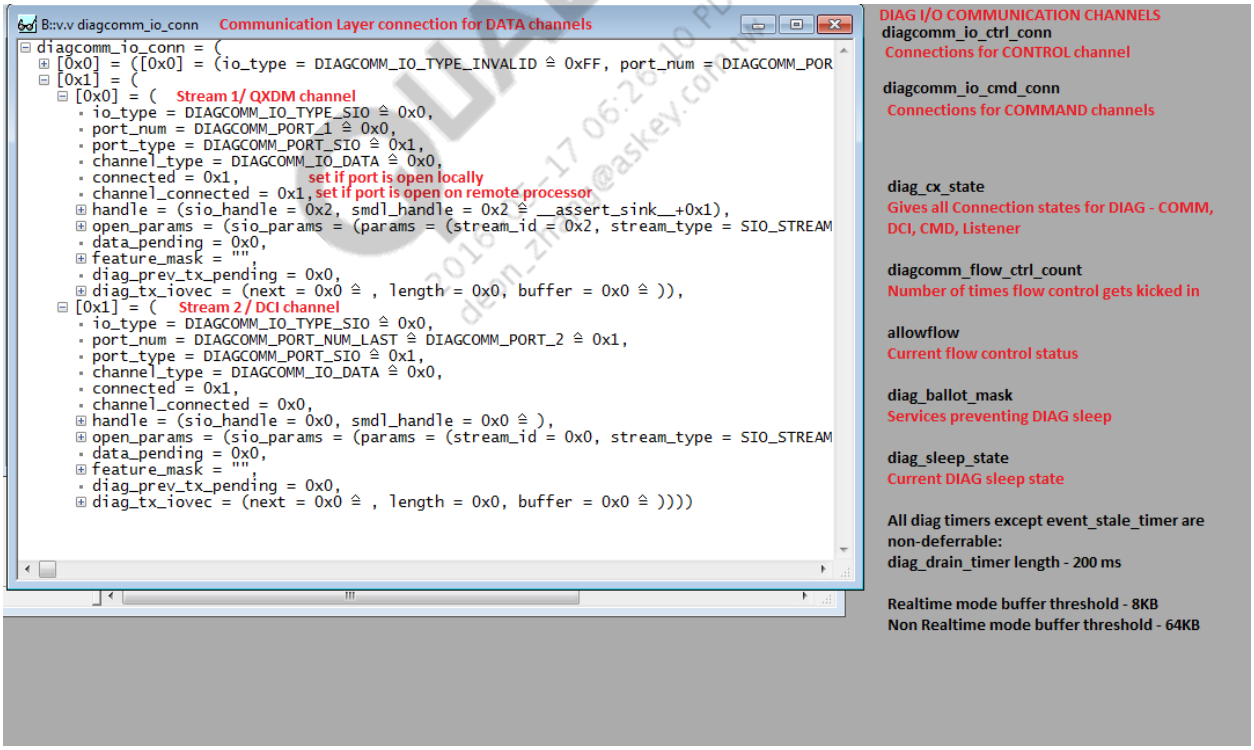
Index = (event_id/8)

event_id = 0x0112 = 274 example:

- Index = 274/8 = 34 with remainder of 2
- This corresponds to the second bit in diag_event_mask[34]
- diag_event_mask[34] = 11111011
- Since the second bit is 1, the mask is set.

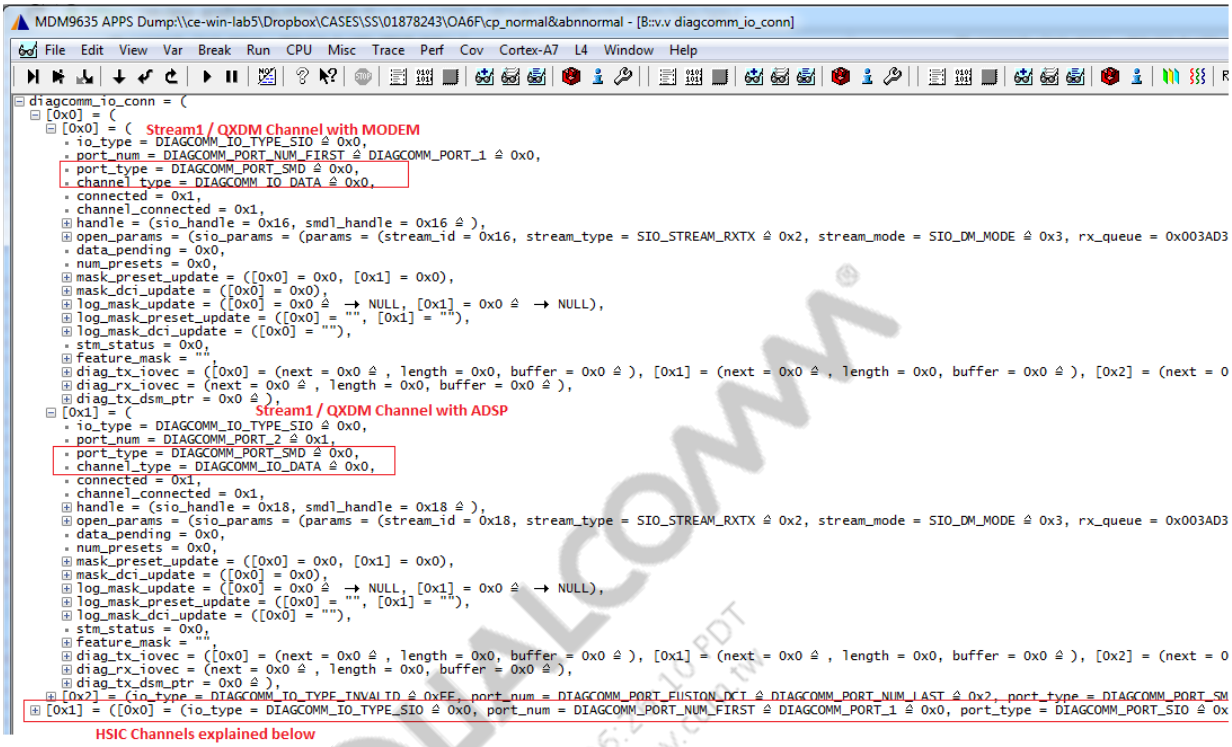
9.3.4 Diag I/O communication channels

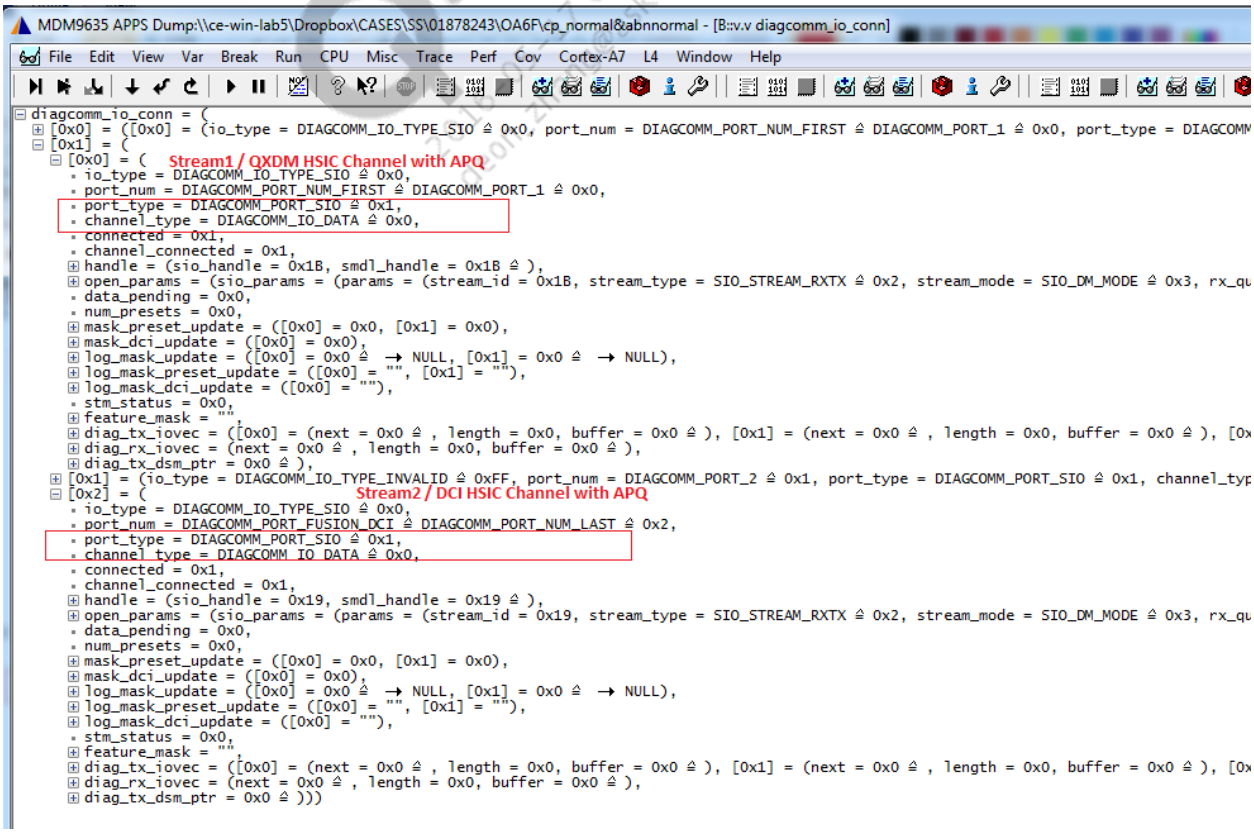
9.3.4.1 Modem



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

9.3.4.2 TN APSS





MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

9.3.5 F3 messages generation

As shown in Figure 9-3, a subsystem client pushes F3 messages to the diag buffer. Once the buffer threshold is touched or the drain timer expires, F3 messages are forwarded to the SIO layer. Notice that F3 messages are also saved in the F3 trace buffer to be extracted out of a RAM dump.

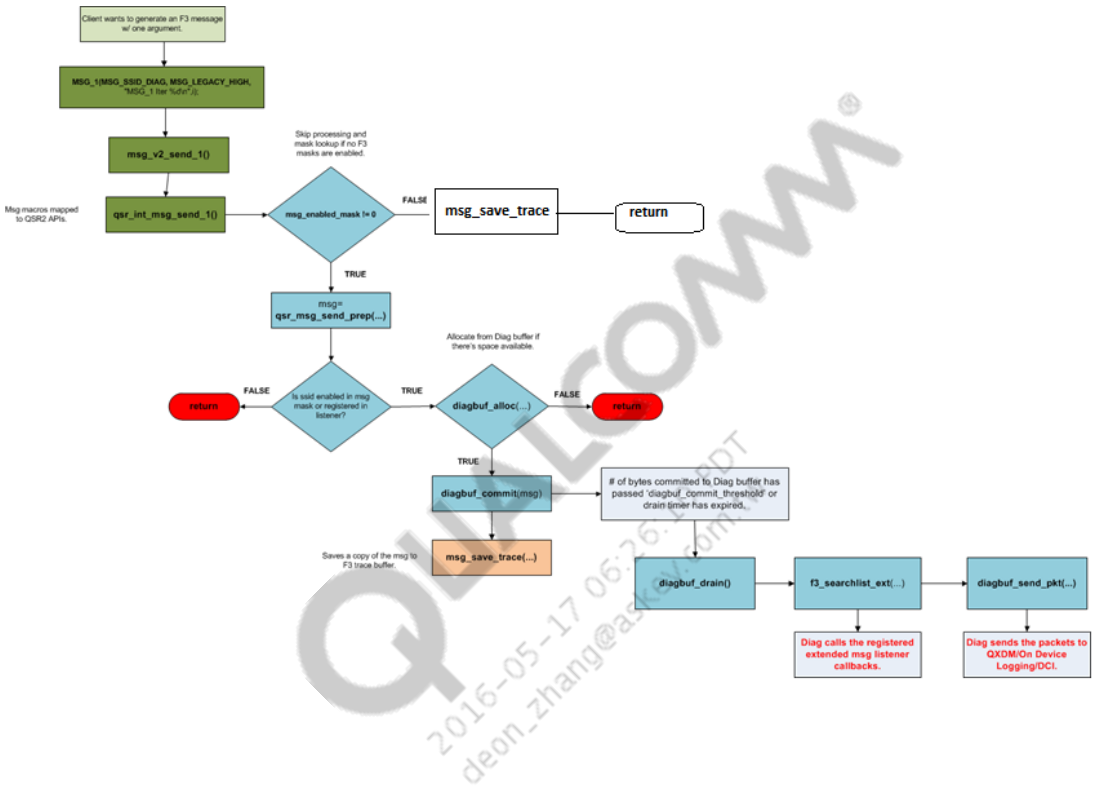


Figure 9-3 F3 message generation call flow

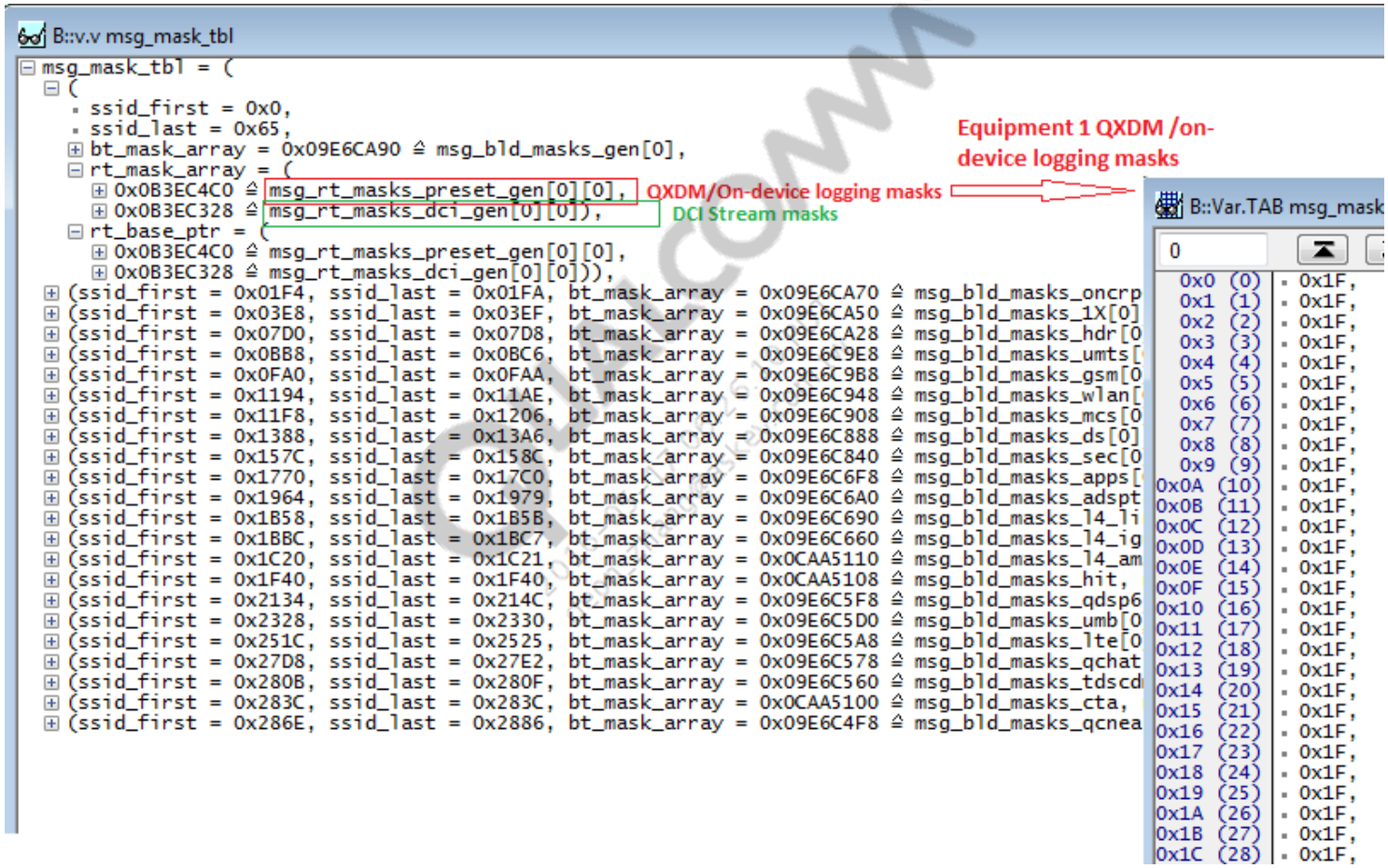
9.3.5.1 Checking the message mask value for a given SSID

msg_enabled_mask is a bitmask that suggests which stream masks are set. An entry in msg_mask_tbl consists of:

- ssid_first – First value of SSID in the range
- ssid_last – Last value of SSID in the range
- *bt_mask_array – Pointer to build time mask array
- *rt_mask_array – Pointer to runtime mask array

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Determine the range in which a given SSID belongs and once the range is obtained, check the runtime mask array to determine if the mask value is set.



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

9.3.6 Diag signals

Diag signals are handled in diag_handle_sigs(). They can be checked as diag_tcb->sigs. Major signals of interest are:

- DIAG_TX_SIG
  - Set whenever there is data committed to the Diag buffer
  - Handled by calling diagbuf_drain() to drain the data out
- DIAG_RX_SIG
  - Set whenever an inbound request is received by Diag and enqueued to the SIO Rx watermark queue
  - Handled by calling diag_process_rx() and eventually diagpkt_process_request() to route the command to the appropriate local handler or forward to the processor that registered the command
- DIAG_RX_RSP_SIG
  - Set in diagpkt_commit() whenever a response packet is queued to the response queue diagpkt_rsp_q
  - Handled by calling diagpkt_rsp_send() and eventually diagbuf_send_pkt() to dequeue and drain the response out
- DIAG_DRAIN_TIMER_SIG
  - Set whenever diag_drain_timer expires (default is 200 ms); this timer is set whenever there is data committed to the Diag buffer to drain data in low traffic scenario
  - Handled by calling diagbuf_drain() to drain data from the Diag buffer to transport layer
- DIAG_EVENT_DRAIN_SIG
  - Set whenever event_stale() is called, indicating that Diag is ready to send an event report packet for primary queue events
  - Handled by calling event_drain() to drain events from the primary event queue event_q to the transport layer
- DIAG_EVENT_DRAIN_SEC_SIG
  - Set whenever event_stale_sec() is called, indicating that Diag is ready to send an event report packet for secondary queue events
  - Handled by calling event_drain_sec() to drain events from the secondary event queue event_q_sec to the transport layer
- DIAG_EVENT_TIMER_SIG
  - Set whenever event_stale_timer expires
  - Handled by calling event_stale(), which calls event_drain() to drain events from event_q to the transport layer
- DIAG_EVENT_TIMER_SEC_SIG
  - Set whenever event_stale_timer_sec expires
  - Handled by calling event_stale_sec(), which calls event_drain() to drain events from event_q_sec to the transport layer

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

9.3.7 DSM debugging

DSM pool buffers are shared between the SIO Communications Layer and Diag to Rx/Tx data packets as shown in Figure 9-4.

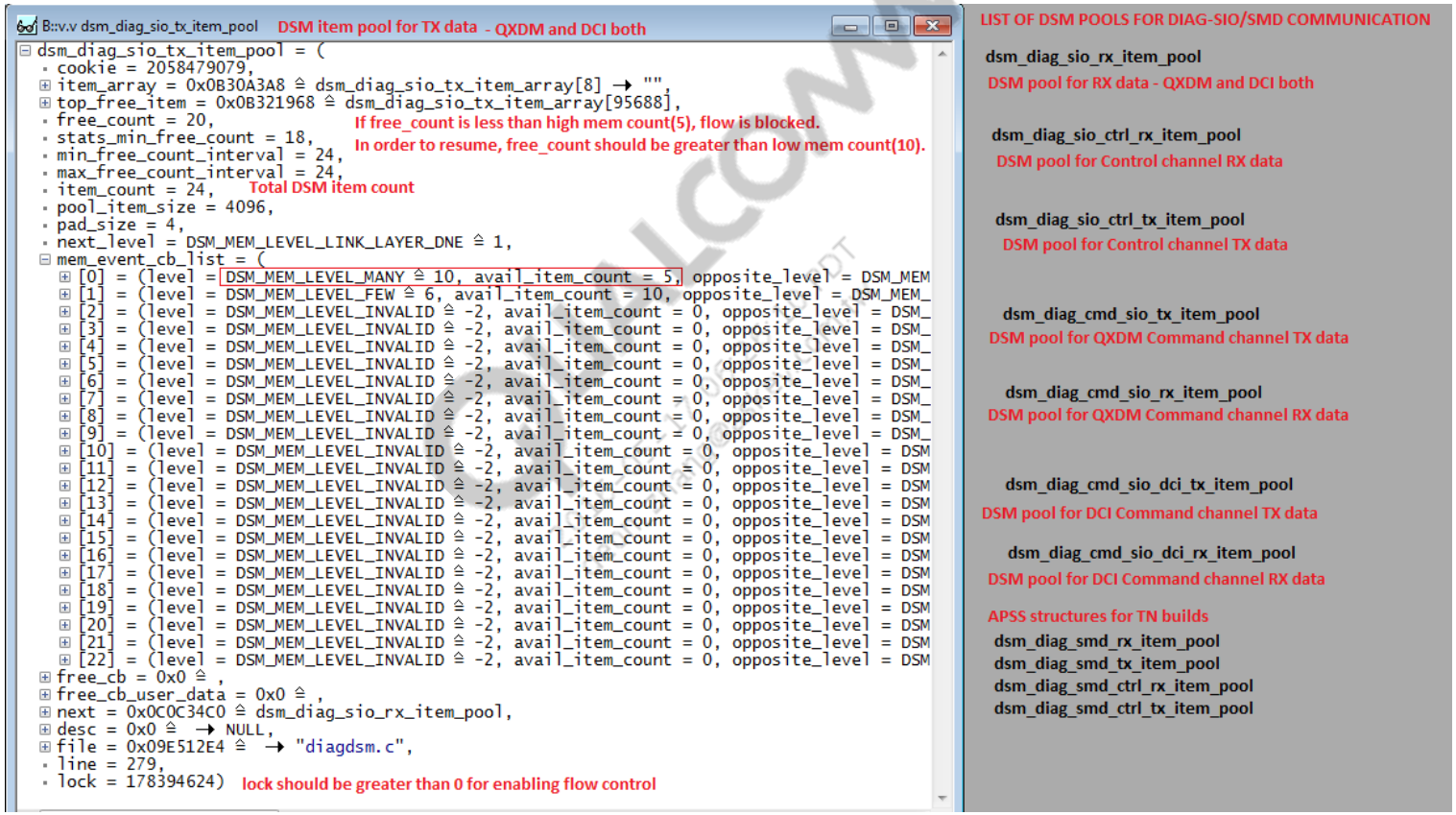


Figure 9-4 Diag DSM pools and watermark queues

The “lock” in DSM pool is a mutex lock used by the DSM layer to manage the flow control, as per the above callback threshold values. This should be a 4-byte positive value. Make sure FEATURE_DSM_REX or a similar DSM feature is defined to initialize the “lock” field in a DSM pool structure.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

To determine whether data flow is blocked or not, check:

- allowflow (outgoing traffic)
- ballowflow (incoming traffic on MDM products from the modem)

For example, say the total DSM items is 749, it is necessary to determine what is causing the Low mem event threshold to be so high, in this case, 707.

The idea behind this implementation was to include Buffering mode. However, the related Diag code still makes it to the CRM releases, which creates confusion. The Buffering mode is for APQ+MDM chipsets where the need to wake up APQ every time MDM has data to transmit is reduced. Rather, data is buffered as much as possible on the MDM ARM® Cortex-A5® processor (this is where the aforementioned DSM item pool comes in). In this example, this pool has 749 items, but only 50 are used in Regular mode, while the remaining 700 come in when Buffering mode is enabled, wasting a lot of space in the process.

This implementation was required since there was a QPST disconnect issue where polling responses were not being received. Due to this large DSM pool, many DSM items got queued (instead of dropping logs/events), which caused the required polling response to lag behind in the watermark queue, and hence caused a QPST timeout. Therefore, the window is now kept to 50 items for the regular logging mode; the pool size could not be changed dynamically, so it was adjusted, so that 50 was its apparent size.

QUALCOMM
2016-05-17 06:26:10 PDT
deon_zhang@askey.com.tw

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

9.3.8 Sleep voting

Figure 9-5 shows the Diag sleep voting call flow.

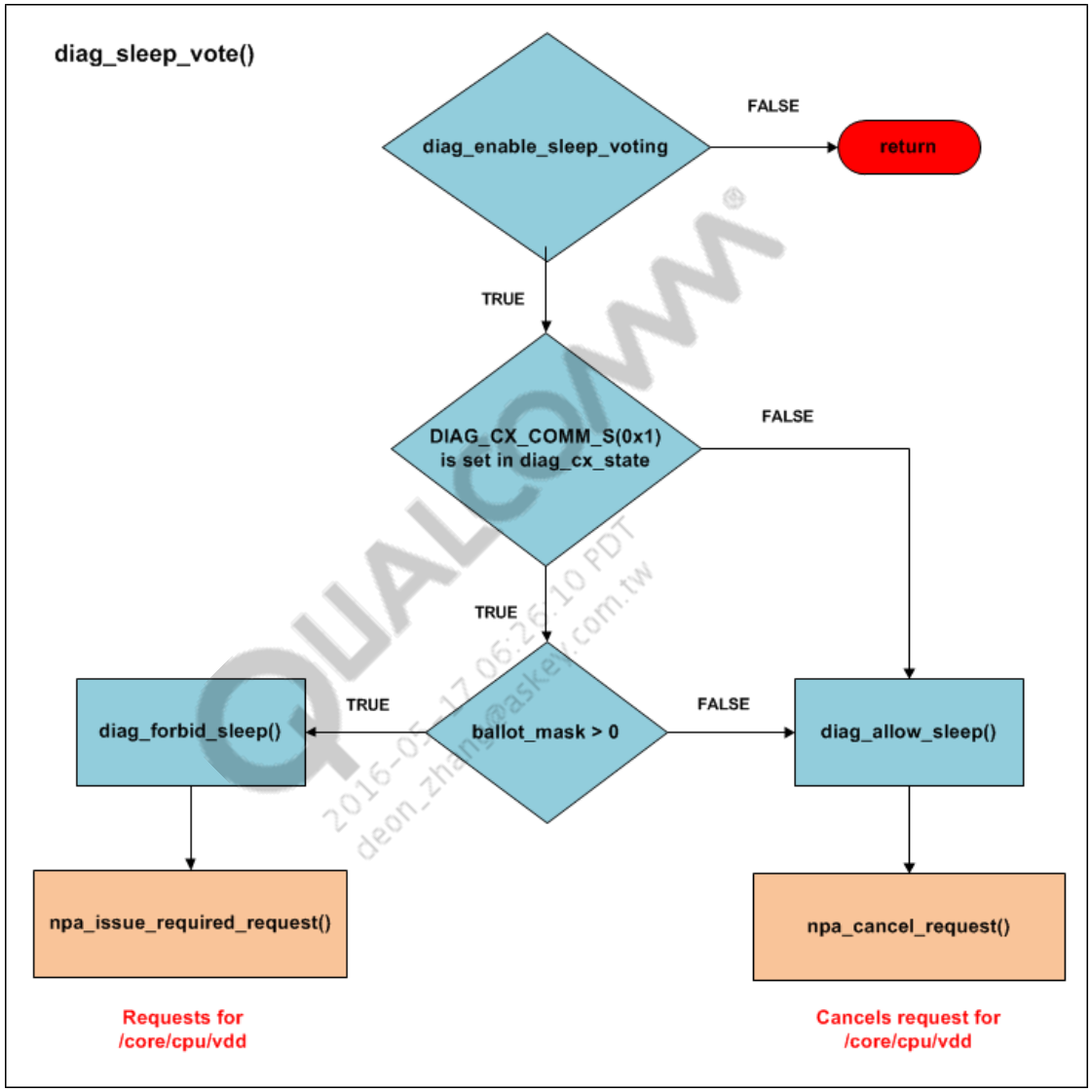


Figure 9-5 Diag sleep voting call flow

Check for the following:

- diag_enable_sleep_voting should be TRUE
- diag_sleep_state should be 1 (DIAG_SLEEP_STATE_ASLEEP)
- allowflow should be 1
- diag_ballot_mask should be 0
  - This is a bitmask for all services; that is, buffer, log, and F3 (event masks do not affect Diag sleep).
  - If any of these bits are set or if diag_ballot_mask > 0, Diag will prevent power collapse unless the Communications layer connection is broken.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

- diag_cx_state
  - First LSB is for DIAG_CX_COMM_S; that is, Communications Layer connection. If set to 0, the channel is not connected; hence, Diag will vote for sleep.
- diagbuf_head/diagbuf_tail
  - Both pointers should be equal, showing an empty buffer.
  - If diagbuf_head is less than diagbuf_tail, it is wrapped, and if diagbuf_head is more than 128 k, it is secretly wrapped.
- diag_tcb.sigs
  - DIAG_DRAIN_TIMER_SIG 0x00200000 (set on drain timer expiration)
  - DIAG_DRAIN_TIMER_LEN 200 ms
  - DIAG_TX_SIG 0x00000080 (when the client commits data to the Diag buffer)
  - DIAG_RX_RSP_SIG 0x00000200 (when responses are generated for Rx traffic)

9.3.9 Timestamps

Each Diag packet contains a timestamp field. The timestamp represents the time the packet was allocated, not the time the packet was sent. The timestamp is generated locally on each processor as follows:

- Diag reads the timestamp using the time API available on the image on which it is running
- Diag does not control or set the timestamp in any way

Timestamp differences or jumps in QXDM Pro logs between packets generally mean either one of the following:

- The time value was updated on the individual processor.

or

- The packets are being sent from different processors, and the timestamps between the processors are not synchronized.

9.3.10 Maximum Packet sizes

There might be variations between targets, so it is best to check the header file diagpkt.h (core/api/services) for that particular build.

For example,

Maximum Tx packet size

#define DIAG_MAX_TX_PKT_SIZ (0x4000) /* For long logs and command responses */

Maximum Rx packet size

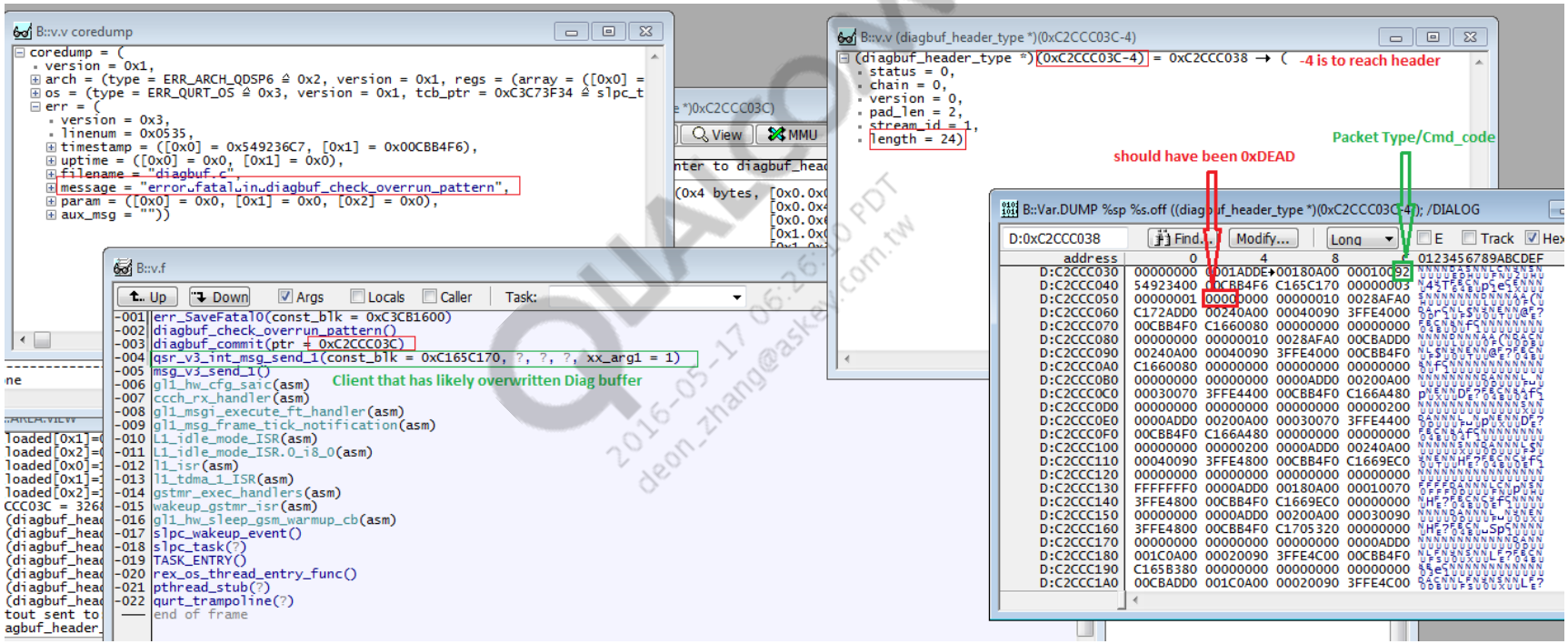
#define DIAG_MAX_RX_PKT_SIZ (0x4000) /* To support long command packets */

For logs and command packets, this means that the payload (excluding HDLC encoding) can be as long as the #define.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

9.3.11 Error fatal in diagbuf_check_overrun_pattern

This error fatal is usually due to a client overwriting the Diag buffer. A 2-byte 0xDEAD pattern is appended to every packet allocated from the Diag buffer. For example, if a client requests a 4-byte buffer using log_alloc() and writes more than 4 bytes to the log pointer, a subsequent log_commit() call by the client to commit the log packet to the Diag buffer results in this error fatal, as shown below.



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

9.3.12 Debugging listeners

There are 3 types of listeners:

- Event
- Log
- F3

F3 and Log listeners share many code items, while events have their own APIs.

A listener can be debugged in two ways:

- By adding debug messages that show up in F3 logs
- By ram dump analysis

Debugging event listeners

While debugging events, various other things can be checked:

- Number of registered listeners and their APIs



- Number of events that are allocated

  alloc_count_event
- Number of events that are dropped

  drop_count_event
- Whether event listener masks are set

  diag_event_listener_ext_mask: For extended listener

  diag_event_listener_mask: For single listener

Call flow:

CLIENT CALLS->Event_report_ext->checksmaks-<event_q_alloc

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Debugging log listeners

While debugging logs, these are the various places where a structure can be added to follow the log flow:

- Client code where the log is being allocated and commited
- Log_alloc; API that is called by the client. Add the code at various places:
  - When the client tries to allocate a packet with the wrong header size
  - After the log mask is checked
  - Once the packet is successfully allocated
  - After the log is dropped because it was not enabled
- Log_commit; called when the log is commited
- log_service_outbound; called when the packet is drained.

The following code snippet of a structure to capture the intended log packet can be used as an example:

```
         typedef struct
{
   unsigned long long ts;
   char src_ptr[20];
   char dest_ptr[20];
}_dbg_diag_extract_pkt_type;

_dbg_diag_extract_pkt_type dbg_diag_extract_pkt[500];
uint32 dbg_diag_extract_pkt_type_idx=0;
if(dbg_diag_extract_pkt_type_idx>=500)
        dbg_diag_extract_pkt_type_idx=0;
memcpy(&dbg_diag_extract_pkt[dbg_diag_extract_pkt_type_idx].src_ptr, hdlc-
>src_ptr, 20);
memcpy(&dbg_diag_extract_pkt[dbg_diag_extract_pkt_type_idx].dest_ptr, hdlc-
>dest_ptr, 20);
DalTimetick_GetTimetick64(hDebugTimerHandle,&dbg_diag_extract_pkt[dbg_diag_
extract_pkt_type_idx].ts);
dbg_diag_extract_pkt_type_idx++;
```

Add structures and counters to keep track of various log packets.

To use the timer:

```
DalTimetick_Attach("SystemTimer", &hDebugTimerHandle);
DalTimetick_Enable(hDebugTimerHandle,1);
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

10 Code structure

10.1 L4/Rex

10.1.1 Source files

Diag public API files

- core\api\services\
  - diag.h
  - diag_fusion.h
  - diag_LSM.h
  - diagbuf.h
  - diagcmd.h – Defines command codes and SSIDs
  - diagcomm.h
  - diagdiag.h
  - diagdsm.h
  - diagnv.h
  - diagpkt.h
  - event.h
  - event_defs.h – Defines event IDs
  - log.h
  - log_codes.h – Defines log codes (additional log codes defined in log_codes_umts.h, log_codes_wcdma.h, log_codes_gsm.h)
  - msg.h
  - msg_diag_service.h
  - msg_mask.h
  - msg_pkt_defs.h
  - msg_qsr.h
  - msg_wince.h
  - msgcfg.h – Defines F3 SSIDs
  - msgtgt.h – Defines levels for each SSID (high, med, low, etc.)

QUALCOMM®
2016-05-17 06:26:10 PDT
deon_zhang@asker.com.tw

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Diag source files

- core\services\diag\
  - DCM\common\src\
    - diag.c – Main driver file pertaining to task diag_tcb
    - diagbuf.c – Routines for managing diag output buffer
    - diagcomm_sio.c – SIO layer notification routines, meant for the slave processor
    - diagcomm_smd.c – Meant for the master processor only
    - diagcomm_cmd.c – For the command packet service
    - diagdsm.c – Diag DSM routines
    - diaglog.c – Log packet support
    - msg.c – F3 message support
    - event.c – Event report support
  - DCM\rtos\src\
    - log_api.c – External API for clients using the log service
    - msg_api.c – External API for clients using the F3 message service
    - event_api.c – External API for clients using the events service
    - diagpkt.c – Command/response packet support

10.1.2 Callflow

10.1.2.1 Initialization

`diag_task()` is the starting API for the Diag driver, registered with the RCINIT layer. It is called at the time of modem startup.

```
diag_task() ->
    diag_init() -> Initialize Diag task
         diag_sleep_init() ->
              npa_resource_available_cb() Request for NPA resource
         diagcomm_io_init() Initialize diagcomm_io_* structures
         diag_heap_init()
         diagdsm_init()
         diag_fwd_task_init() Creates a separate task on TN APSS for forwarding peripherals data
    <--
    diagcomm_sio_init()Register watermark level callbacks
    diagcomm_register_inbound(diag_rx_notify) Register callback diag_rx_notify()
    diag_sleep_vote(TRUE) Vote for sleep
    diag_handle_sigs(0)
```

`diag_task`() does not exit due to continuous while() loop in `diag_handle_sigs`(). All Diag operations are serviced based on a software signal (see Section 9.3.6) set either by itself or by an external task to complete some operation.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

10.1.2.2 Rx

SIO task

```
diag_rx_notify() ->
   osal_set_sigs(DIAG_RX_SIG)
```

Diag task

```
diag_handle_sigs(DIAG_RX_SIG) ->
   diag_process_rx() ->
```

`diagpkt_process_request()` Process command packets based on the command registration table

10.1.2.3 Tx

Client task

`diagpkt_commit()` Client submits a command response packet

`event_report()`Client submits an event packet

`msg_send()` Client uses MSG_* macros

`log_commit()` Client submits a log packet

```
   diagbuf_commit() ->
        diag_tx_notify() ->
             osal_set_sigs(DIAG_TX_SIG)
```

Diag task

```
   diag_handle_sigs(DIAG_TX_SIG) ->
        diagbuf_drain() ->
```

Drains the Diag buffer to the communication layer

```
             diagbuf_send_pkt() ->
                  diagcomm_send() ->
                       diagcomm_buffer_enqueue_stream() ->
                            dsm_enqueue()
                       diagcomm_io_transmit() ->
                            sio_transmit()
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

10.2 Linux

10.2.1 LA source files

Diag public files

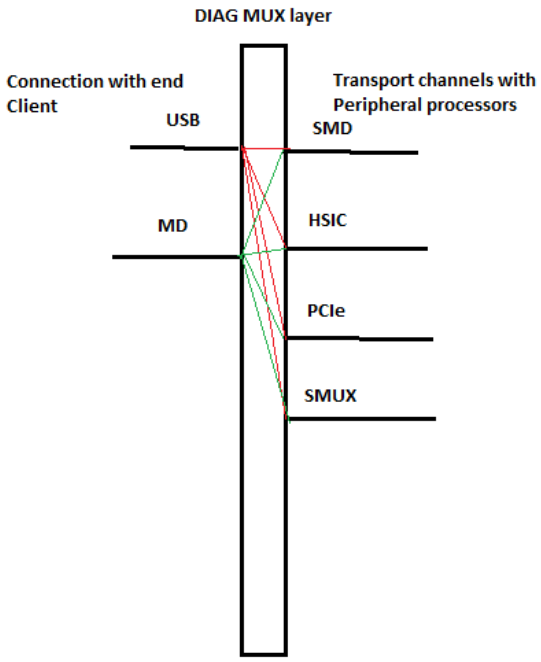
- kernel\drivers\char\diag\..

Diag proprietary code

- vendor\qcom\proprietary\diag\
  - src\.. – LSM layer source
  - mdlog\diag_mdlog.c – Writes Diag traffic to the SD card
  - uart_log\diag_uart_log.c – Writes Diag traffic to the UART port
  - socket_log\diag_socket_log.c – Writes Diag traffic to the socket (Ethernet/Wi-Fi/data connection)
  - dci_client\diag_dci_client.c – Sample DCI client
  - klog\diag_klog.c – Displays kernel messages on QXDM
  - PktRsptest\PktRspTest.c – Sample application to demo Command/Response service
  - test\test_diag.c – Demos usage of all Diag services

10.2.2 Call flow

The connection between USB and on-device logging with SMD/HSIC/PCI/SMUX is shown below.



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

10.2.2.1 Initialization

`diagchar_init()` is the main initialization function called via `module_init()`, at the time of kernel loading.

```
diagchar_init() ->
   diag_ws_init() -> Initialize MD and DCI wakelock variables
   diag_debugfs_init()
   diag_masks_init()
   diag_mux_init()
   diagfwd_init() SMD initialization
   diag_remote_init()
   diagfwd_bridge_init() HSIC, SMUX, PCIe initialization
   diag_dci_init()
```

10.2.2.2 SMD initialization

```
diagfwd_init() ->
   diag_smd_constructor()
   diag_mux_register() ->
        diag_usb_register() ->
             diagmem_init()
             usb_diag_open() Opens a Diag channel over USB. A notify handler is passed,
             which USB calls in case of CONNECT, DISCONNECT, READ_DONE, and
             WRITE_DONE events

        <--
        diag_md_register() For on-device logging
   <--
platform_driver_register() Loads the Diag SMD driver
```

10.2.2.3 USB connnect with SMD

When USB is connected, Diag is notified with a CONNECT event.

Diag prepares SMD structures and queues a read with USB.

```
diag_usb_notifier(CONNECT) ->
   queue_work(usb_connect_work_fn()) ->
        usb_connect() ->
             usb_diag_alloc_req()
             diagfwd_mux_open() ->
                  diag_smd_notify() Read SMD CNTL channel in case of data

             <--
             queue_work(usb_read_work_fn()) ->
                    usb_diag_read() Read data from USB diag channel. When data is
                    available diag_usb_notifier() is called with READ_DONE event.
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

10.2.2.4 USB disconnect with SMD

```
diag_usb_notifier(DISCONNECT) ->
   queue_work(usb_disconnect_work_fn()) ->
        usb_disconnect() ->
             diagfwd_mux_close()
```

10.2.2.5 USB read, SMD write

The following code shows the scenario when USB is done reading data from the PC Diag channel and has notified Diag driver with a READ_DONE event.

The Diag driver writes this data to the SMD channel and then queues another read with USB.

```
diag_usb_notifier(READ_DONE) ->
   queue_work(usb_read_done_work_fn()) ->
        diagfwd_mux_read_done() ->
             diag_process_hdlc() ->
                  diag_hdlc_deode()
                  diag_process_apps_pkt() ->
                       smd_write() Writes to the SMD channel

             <--
             diag_mux_queue_read() ->
                  diag_usb_queue_read() ->
                       queue_work(usb_read_work_fn()) ->
                            usb_diag_read() Reads from the USB Diag channel
```

10.2.2.6 USB write, SMD read

The following code shows the scenario when USB is done writing data on the Diag channel and has notified Diag with a WRITE_DONE event.

The Diag driver reads from the SMD channel and passes on data to USB.

```
diag_usb_notifier(WRITE_DONE) ->
   diag_usb_write_done() ->
        diagfwd_mux_write_done() ->
             diag_smd_reset_buf() ->
                  queue_work(diag_read_smd_work_fn()) ->
                       diag_smd_send_req() ->
                            smd_read() Read from SMD channel
                            diag_process_smd_read_data() ->
                                 diag_mux_write() ->
                                      diag_usb_write() ->
                                           usb_diag_write() Write on USB
             channel. Once done, callback diag_usb_notifier() is called

                            <--
                            diag_smd_notify() Read further from the SMD channel
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

10.2.2.7 HSIC initialization

```
diag_hsic_init() ->
   diagfwd_bridge_register() ->
        diag_mux_register() ->
             diag_usb_register() ->
                  diagmem_init()
                  usb_diag_open()
```

Opens a Diag channel over USB. A notify handler is passed, which USB calls in case of CONNECT, DISCONNECT, READ_DONE, and WRITE_DONE events

```
                  <--
                  diag_md_register()
```

For on-device logging

10.2.2.8 USB Connect, with HSIC

When USB is connected, Diag is notified with a CONNECT event.

Diag prepares HSIC structures and queues a read with USB.

```
diag_usb_notifier(CONNECT) ->
   queue_work(usb_connect_work_fn()) ->
        usb_connect() ->
             usb_diag_alloc_req()
             diagfwd_bridge_mux_connect() ->
                  hsic_open() ->
                       diag_bridge_open()
                       queue_work(usb_read_work_fn()) ->
                            usb_diag_read()
```

Read data from the USB Diag channel.

When data is available, diag_usb_notifier() is called with READ_DONE event.

10.2.2.9 USB disconnect with HSIC

```
diag_usb_notifier(DISCONNECT) ->
   queue_work(usb_disconnect_work_fn()) ->
        usb_disconnect() ->
             diagfwd_bridge_mux_disconnect() ->
                  hsic_close() ->
                       diag_bridge_close()
                       diag_remote_dev_close()
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

10.2.2.10 USB read, HSIC write

The following code shows the scenario when USB is done reading data from the PC Diag channel and has notified the Diag driver with a READ_DONE event.

The Diag driver writes this data to the HSIC channel. Once done, HSIC calls the Diag callback. Diag then queues another read with USB.

```
diag_usb_notifier(READ_DONE) ->
   queue_work(usb_read_done_work_fn()) ->
        diagfwd_bridge_mux_read_done() ->
             diagfwd_bridge_write() ->
                  hsic_write() ->
                       diag_bridge_write() -> HSIC API
                            usb_alloc_urb()
                            usb_submit_urb()
```

//Once write is done, HSIC calls the callback API diag_hsic_write_complete()

```
        diag_hsic_write_complete() ->
             diag_remote_dev_write_done() ->
                  diag_mux_queue_read() ->
                       diag_usb_queue_read() ->
                            queue_work(usb_read_work_fn()) ->
                                 usb_diag_read() Reads from USB Diag channel
```

10.2.2.11 USB write, HSIC read

The following code shows the scenario when USB is done writing data to the PC Diag channel and has notified the Diag driver with a WRITE_DONE event.

Diag queues a read with HSIC. Once data is available, HSIC notifies Diag, which passes this data to USB.

```
diag_usb_notifier(WRITE_DONE) ->
   diag_usb_write_done() ->
        diagfwd_bridge_mux_write_done() ->
             hsic_fwd_complete() ->
                  queue_work(hsic_read_work_fn()) ->
                       diag_bridge_read() ->
                            usb_alloc_urb()
                            usb_submit_urb()
```

If there is data available on HSIC, it calsl the callback diag_bridge_read_cb()

```
        diag_bridge_read_cb() ->
             diag_hsic_read_complete() ->
                  diag_remote_dev_read_done() ->
                       diag_mux_write() ->
                            diag_usb_write()  Writes on the USB channel. Once done,
callback diag_usb_notifier() is called.

                       <--
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
hsic_queue_read() ->
     queue_work()
     ...
     hsic_read_work_fn()
```

10.2.2.12 Packet masks over HSIC

```
diag_read_mask_file() ->
   diag_send_data() ->
        write()
```

write() is mapped to the Diag kernel driver's diagchar_write()

```
diagchar_write() ->
   diag_process_userspace_remote() ->
        diagfwd_bridge_write()
```

10.2.2.13 DCI initialization

```
diag_dci_init() ->
   diag_dci_init_ops_tbl() Initialize client structures
   diag_smd_constructor()
   platform_driver_register() Registers the DCI driver
```

10.2.2.14 DCI write to peripherals

The DCI client sends a packet using the write() API, which is mapped to diagchar_write().

```
diagchar_write() ->
   diag_user_process_dci_data() ->
        diag_process_dci_transaction() ->
```

//in case of log masks being set

`update_dci_cumulative_log_mask()` Sets a dirty bit to indicate this set of masks must be forwarded to the modem

```
diag_send_dci_log_mask() ->
     diag_dci_write_proc() ->
          smd_write() Writes to SMD
```

//in case event masks being set

```
update_dci_cumulative_event_mask()
diag_send_dci_event_mask() ->
     diag_dci_write_proc() ->
          smd_write()Writes to SMD
```

//in case if command packet is sent

```
diag_process_dci_pkt_rsp() ->
     diag_dci_process_apps_pkt() Check if its an AP serviced command
     diag_send_dci_pkt() ->  If it is a peripheral command
          diag_dci_write_proc() ->
               smd_write()Writes to SMD
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

10.2.2.15 DCI read from peripherals

SMD calls callback diag_smd_notify() when data is available on SMD.

```
diag_smd_notify() ->
   queue_work(diag_read_smd_work_fn()) ->
        diag_smd_send_req() ->
             diag_process_smd_dci_read_data() ->
                  diag_process_single_dci_pkt() ->
                       extract_dci_log()
                       extract_dci_events()
                       extract_dci_pkt_rsp()
                       extract_dci_ctrl_pkt()
                  <--
                  diag_dci_wakeup_clients()
                  dci_check_drain_timer()
```

10.2.3 LE source files

Diag public files

- kernel\drivers\char\diag\..

Diag proprietary code

- diag\
  - src\.. – LSM layer source
  - dci_client\diag_dci_client.c – Sample DCI client
  - klog\diag_klog.c – Displays kernel messages on QXDM
  - mdlog\diag_mdlog.c – Writes diag traffic to SD card
  - uart_log\diag_uart_log.c – Writes diag traffic to UART port
  - PktRsptest\PktRspTest.c – Sample app to demo Command/Response service
  - test\test_diag.c – Demos usage of all diag services

QUALCOMM® 2015-05-17 06:26:10 PDT dean_zhang@askey.com.tw

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

11 Case study

11.1 Packet drop

In this scenario, Diag packets (F3 messages, logs) were sometimes dropped if the device LCD was turned off. After turning the LCD back on, the Diag traffic flow resumed.

If APSS was made to hold a wake-up source all the time, the issue did not occur.

End-to-end debugging was needed across Diag and SMD modules to root cause the issue.

11.1.1 Creating an IPC buffer on APSS

The IPC buffer allows saving debug messages in an allocated RAM buffer.

The first parameter to this API is number of pages. A page size is usually 4 kB.

```
#include <linux/ipc_logging.h>
void *ipc_log_context_create(int max_num_pages, const char *mod_name);
int ipc_log_string(void *ilctxt, const char *fmt, ...);
```

11.1.2 Extracting IPC logs

From the ADB shell, run the following:

cat /sys/kernel/debug/ipc_logging/<name>/log_cont

For examples, cat /sys/kernel/debug/ipc_logging/diag_ipc/log_cont > /data/file.txt

It can be saved to a file on a local machine or on the device itself.

11.1.3 Using MPM timecount

The MPM timecount register is 64 bits, and it can be read by any processor. Hence, it is very useful in logging the timestamp for certain events and later on correlating these events from separate processors.

APSS API

```
#include arch_timer.h
cycle_t arch_counter_get_cntpct(void);
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

MPSS API

```
#include "DDITimetick.h"  //for DalTimetick APIs
#include "DalDevice.h"    //for DAL data types
#include "DALStdErr.h"  //for DAL return code
static __inline DALResult DalTimetick_Attach(const char *pSystemTimer,
DalDeviceHandle** phDalHandle);
static __inline DALResult DalTimetick_GetTimetick64(DalDeviceHandle *
_h, DalTimetickTime64Type * ticks);
```

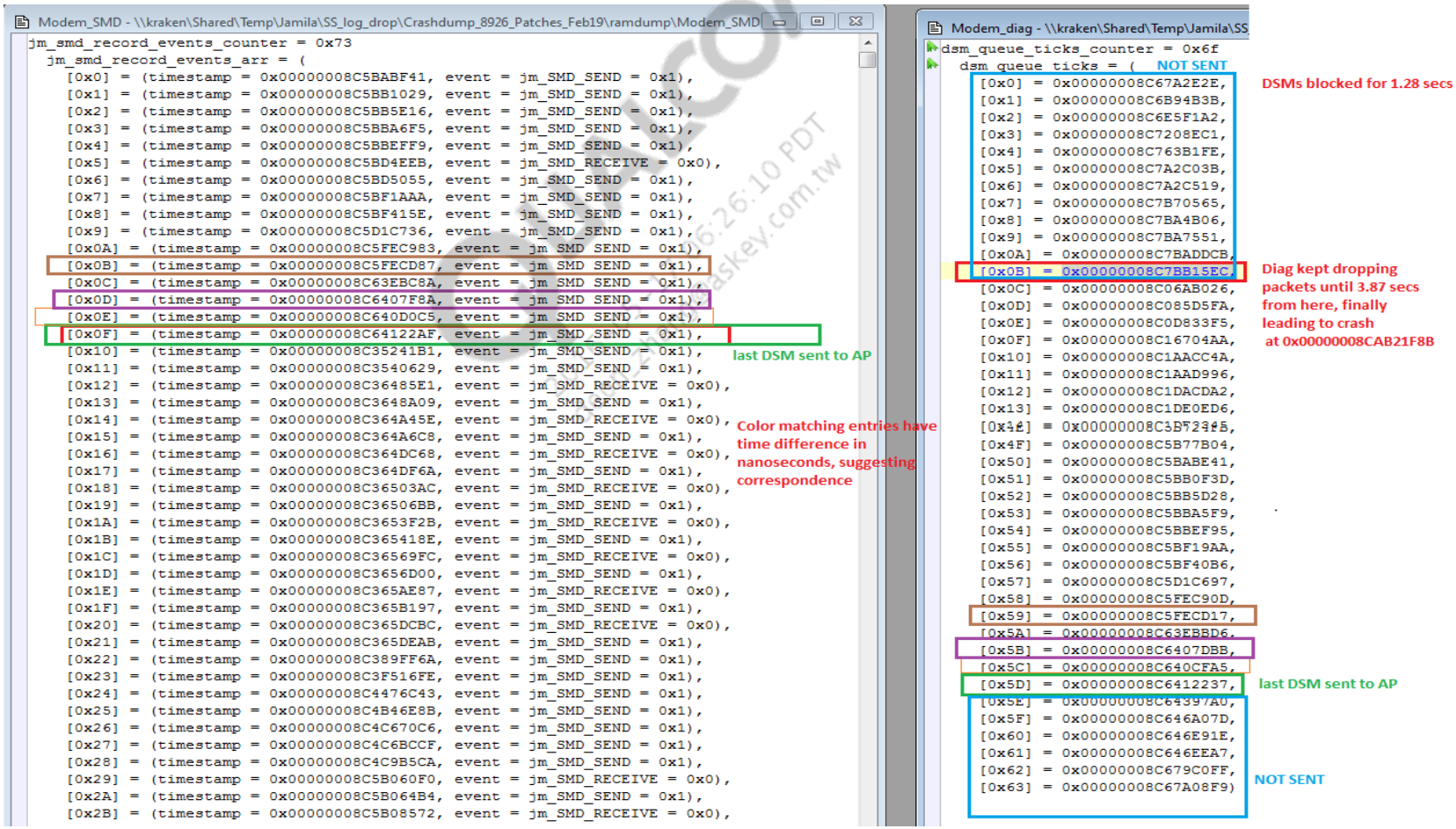
QUALCOMM®
2016-05-17 06:26:10 PDT
deon_zhang@askey.com.tw

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Debug patches are used to record events using a common MPM timecount on APSS and MPSS. MPSS Diag starts a timer when DSMs are exhausted. The timer is cleared when DSMs are freed up. When this timer expires, if DSMs are still not available, crash dumps of the SMD state are gathered.

1. MPSS SMD sent the last interrupt at 0x8C64122AF, for which Diag must have queued the DSM at 0x8C6412237.

   After that, several DSMs were queued with SIO until 0x8C7BB15EC. Diag started dropping packets from this point onward, finally leading to a crash at 0x8CAB21F8B.

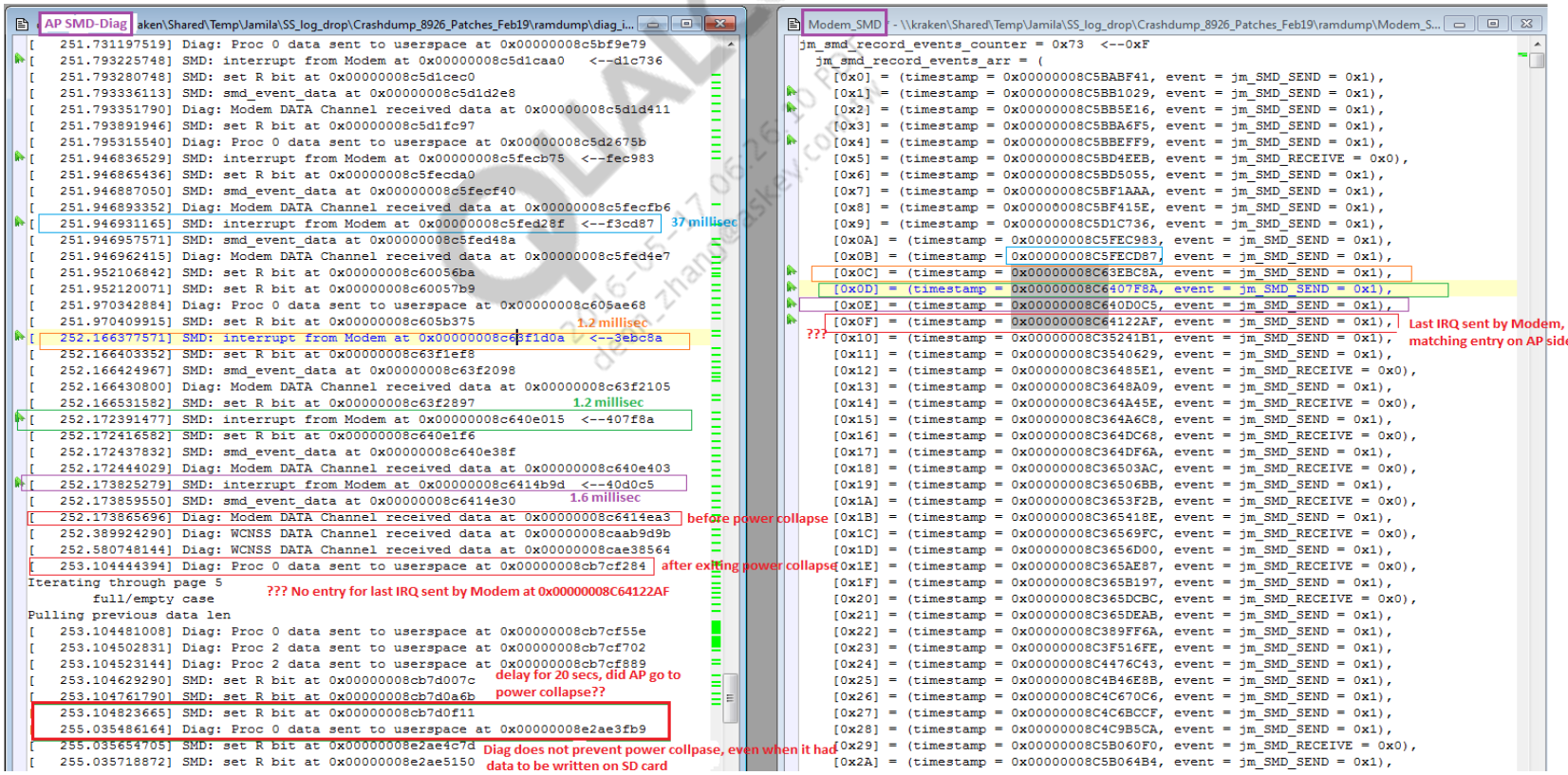


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

2. MPSS sent the last IRQ at 0x8C64122AF and APSS received the last IRQ at 0x8C6414B9D. By backtracking in these logs up to ~10 IRQs, it is possible to match a sent IRQ with a received one with the initial time gap in microseconds and then milliseconds.

   With that mapping, a matching received IRQ is not seen on APSS for the IRQ sent by MPSS. Since the last IRQ on APSS happened a little after the last one sent from MPSS, it is likely that APSS serviced the last two sent IRQs collectively only one time.

   After the last received IRQ, APSS goes into power collapse at the time when Diag had to write packets to the SD card. This is clearly an issue that must be fixed. A patch was released for the SS for holding a wake-up object from when Diag is notified to read SMD data until the time when it is written fully to the user space application. At this point, it is not possible to tell if this issue has led to the subsequent unusual events; that is, MPSS SMD waiting for an ACK interrupt from APSS to empty DSMs, etc.



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3. After correlating the Kernel log (yellow highlighting) with the rest of the logs (IPC log, red boldface), it can be seen that while the APSS goes into power collapse, MPSS sends it interrupts to read SMD data. The APSS SMD driver notifies Diag, but Diag fails to send packets to the user space since the APSS has gone into power collapse. The MPSS Diag then starts dropping packets and eventually crashes. Later, when APSS has resumed, Diag writes previously pending packets to the user space.

```
<6>[  251.902803] [0:    kworker/u:1:   35] PM: suspend entry 2014-
02-24 07:15:32.108895202 UTC            ←AP entering power collapse
<6>[  251.902854] [0:    kworker/u:1:   35] PM: Syncing filesystems
... done.
<4>[  251.938022] [0:    kworker/u:1:   35] Freezing user space
processes ... (elapsed 0.03 seconds) done.
<4>[  251.973559] [0:    kworker/u:1:   35] Freezing remaining
freezable tasks ... (elapsed 0.02 seconds) done.
<4>[  251.993855] [0:    kworker/u:1:   35] Suspending console(s)
(use no_console_suspend to debug)
…
  <MPSS Sends interrupt at 0x8C640D0C5>
  <MPSS Sends interrupt at 0x8C64122AF>
[ 252.173]  AP receives interrupt at 0x8C6414B9D
[ 252.173865696] AP Diag receives packets at 0x00000008c6414ea3

…
<4>[  252.381492] [0:    kworker/u:1:   35] Disabling non-boot CPUs
...
<6>[  252.381753] [0:    kworker/u:1:   35] msm_pm_enter
<6>[  252.381753] [0:    kworker/u:1:   35] msm_pm_enter: power
collapse
…
<6>[  252.381753] [0:    kworker/u:1:   35] Enabled clock count: 14
   [   POWER COLLAPSE starts, but Diag has not sent packets to the
userspace ]

<6>[  252.381753] [0:    kworker/u:1:   35] msm_pm_enter:
return
<4>[  252.381753] [0:    kworker/u:1:   35] gic_show_resume_irq: 174
triggered
<4>[  252.381753] [0:    kworker/u:1:   35] gic_show_resume_irq: 200
triggered
<6>[  252.388608] [0:    kworker/u:1:   35] PM: noirq resume of devices
complete after 6.084 msecs
<6>[  252.396527] [0:    kworker/u:1:   35] PM: early resume of devices
complete after 5.746 msecs
…
   <MPSS Diag is dropping packets from 0x00000008CAB1D190 to
                                      0x00000008CAB21F8B>

<3>[  252.473090]I[0:      swapper/0:    0] SMSM: MPSS SMSM state
changed to SMSM_RESET.+++ wakeup source Active =
smsm_snapshot                ←MPSS crashed
<3>[  252.473349]I[0:      swapper/0:    0] Fatal error on the MPSS.
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
<3>[  252.473434]I[0:      swapper/0:    0] MPSS subsystem failure
reason: diagbuf.c:975:Diag has dropped 50 Message packets, DSM items
are full.
…
<4>[  252.899014] [0:    kworker/0:1:   27] --- wakeup source
Deactive = qpnp-vadc-ee1e7c00
<4>[  252.899158] [0:    kworker/0:1:   27] +++ wakeup source Active
= qpnp-vadc-ee1e7c00
<4>[  252.899420] [0:    kworker/u:1:   35] done.
<6>[  252.900850] [0:    kworker/u:1:   35] PM: suspend exit 2014-
02-24 07:15:36.748986189 UTC
[   POWER COLLAPSE ends ]
…
<3>[  252.998431] [0:    kworker/0:1:   27] get_prop_batt_temp:
get_bat_temp: adc_code(32806) physical (256)
<3>[  253.097232] [0: UEventObserver:  917] get_prop_batt_temp:
get_bat_temp: adc_code(32806) physical (256)

[ 253.104444394] AP Diag sends pending packets to userspace at
0x00000008cb7cf284

<6>[  253.492406] [0:    kworker/u:1:   35] PM: suspend entry 2014-02-24
07:15:37.340529522 UTC
<6>[  253.492977] [0:    kworker/u:1:   35] PM: Syncing filesystems ...
done.
```

QUALCOMM
2016-05-17 06:16:16 PDT
deon_zhang@askey.com.tw

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

A Defining custom Android permissions for Diag

This appendix discusses the QCOM_DIAG custom Android permissions. It describes how to declare these permissions and how to enable an APK to use them. The implementation of these permissions provides a mapping between the high-level Android permission and the low-level Linux permission. See Declaring and Enforcing Permissions for information on the mapping.

A.1 Enabling use of custom permissions by a system APK

The framework of a system APK is used to create the QCOM_DIAG permissions. The APK can be a minimal APK because it is not required to execute anything or even to be selectable on the device applications. However, it must be present in the system image.

A.1.1 Declaring custom Android permission

The APK's AndroidManifest.xml file must declare the permission, using the following format:

```
<permission android:description="@string/desc"
   android:label="@string/label"
   android:name="com.qualcomm.permission.QCOM_DIAG"
   android:protectionLevel="signatureOrSystem"/>
```

If the APK needs to use these permissions, it must declare its need in the APK's AndroidManifest.xml file using this format:

```
<uses-permission android:name="com.qualcomm.permission.QCOM_DIAG"/>
```

The protection level for this permission is signatureOrSystem. This requires that the APK be either part of the system image or a signed APK to use either of these permissions.

A.1.2 Mapping custom Android permission to Linux permission

As part of the APK's source, there must be a mapping .xml file that declares the mapping between the Android permission and the Linux permission. The mapping format is:

<permissions>

<permission name="com.qualcomm.permission.QCOM_DIAG" >

<group gid="qcom_diag" />

</permission>

</permissions>

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

A.1.3 Additional modifications to the APK's Android.mk file

It is often recommended that a module be optionally included in the system image. The following will accomplish this.

```
LOCAL_MODULE_TAG := optional
```

For information on how to add the APK to the system image, see Section A.1.5.

The following causes the APK to be signed with the platform certificate:

```
LOCAL_CERTIFICATE := platform
```

A.1.4 Proper placement of the APK's mapping.xml file

This APK's mapping .xml file must be placed in the /system/etc/permissions folder. Addition to the APK's Android.mk file in the following manner will accomplish this:

```
LOCAL_MODULE := com.qualcomm.qcom_diag.xml
LOCAL_MODULE_TAGS := optional
LOCAL_MODULE_CLASS := ETC
LOCAL_MODULE_PATH := $(TARGET_OUT_ETC)/permissions
```

where the name of the mapping.xml file is com.qualcomm.qcom_diag.xml.

A.1.5 Making the APK part of the system image

The APK and its mapping .xml file must be part of the system image. Because of `LOCAL_MODULE_TAGS:= optional` in the APK's Android.mk file, these files are not automatically added to the system image. Adding the files can be accomplished by explicitly listing the APK and the mapping.xml file in the device-vendor.mk file. If these files were to be listed in the Diag project, the following lines would be added to the list of files in the Diag project:

```
DIAG += com.qualcomm.diag.qcom_diag
DIAG += com.qualcomm.qcom_diag.xml
```

where the APK name is com.qualcomm.diag.qcom_diag and the mapping.xml file is com.qualcomm.qcom_diag.xml.

The device-vendor.mk file can be found in <build_root>/vendor/qcom/proprietary/common/config, where build_root is the root folder of the Android source code.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

A.2 Enabling use of custom permissions by a nonsystem APK

Because the protection level of the QCOM_DIAG is signatureOrSystem, an APK that is not a part of the system image that wishes to use these permissions must declare that it needs them. It must also then be signed.

A.2.1 Using custom Android permissions

The APK must denote that it needs to use the custom permissions. This can be done by adding the following to its AndroidManifest.xml file:

```
<uses-permission android:name="com.qualcomm.permission.QCOM_DIAG"/>
```

A.2.2 Signing an unsigned APK

Because the custom permission declares the protection level as signatureOrSystem, any APK that is not a part of the system image must be signed in order to use that permission. Signing can be performed by exporting the APK as an unsigned APK and then signing it with signapk.jar tool as:

```
java –jar
<build_root>/out/host/<your_host>/framework/signapk.jar
<build_root>/build/target/product/security/platform.x509.pem
<build_root>/build/target/product/security/platform.pk8 unsigned_app.apk
signed_app.apk
```

where build_root is the root folder of the Android source code.

NOTE: This command signs the APK only for this particular build.

Signing an APK with the platform key allows the APK to use any permission protected by the platform key.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

B References

B.1 Related documents

Documents	
Qualcomm Technologies, Inc.	
Diagnostic Services Interface Specification and Operational Description	80-V6135-1
CDMA Dual-Mode Subscriber Station Serial Data Interface Control Document (ICD)	80-V1294-1
Diagnostic Consumer Interface API on Linux	80-N8469-2
QXDM Professional™ User Guide	80-V1241-21
Diagnostic Services Subsystem Interface Control Document (ICD)	80-V1294-1
Long Term Evolution (LTE) Interface Control Document (ICD)	80-VP457-1
Presentation: Secure Services Module (SSM) Overview for Windows Phone 8	80-NA027-1
Presentation: Secure Services Module (SSM) Overview for Linux Android™	80-NH283-1
QXDM Database Editor User Guide	80-V9396-1
Application Note: DCI Override Feature	80-NE668-1
Online Training: Using Diag Commands	VD80-NT650-1
Resources	
Declaring and Enforcing Permissions	http://developer.android.com/guide/topics/security/permissions.html#declaring

B.2 Acronyms and terms

Term	Definition
ADB	Android Debug Bridge
AMSS	advanced mobile subscriber station
APK	Android application package (file format)
APSS	applications processor subsystem
APU	address protection unit
CRC	cyclic redundancy check
BAM	bus access manager
BLSP	BAM low-speed peripheral
DCI	Diag consumer interface
DCM	Diag core module

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Term	Definition
DLF	diagnostic log file
DMC	demodulator card
DMSS	dual-mode subscriber station
DSM	data services memory
DUT	device under test
EFS	embedded file system
ELF	executable and linking format
eMMC	embedded multimedia card
ESN	electronic serial number
GSBI	general serial bus interface
HDLC	high-level data link control
HSIC	high-speed inter-chip interface
ICD	interface-specific documents
IPC	interprocessor communication
LPASS	low power audio subsystem
LSM	legacy service mapping
MPU	memory protection unit
NPA	node power architecture
NRT	non-real time
OTA	over the air
PCIe	peripheral component interconnect express
QCAT	Qualcomm CDMA Analysis Toolkit
QDI	Qualcomm display interface
QPST	Qualcomm Product Support Tool
QSPR	Qualcomm Sequence Profiling Resources
QTI	Qualcomm Technologies, Inc.
REX	Qualcomm real-time executive operating system
RT	real time
SD card	secure digital card
SIO	serial input/output
SLPI	serial link phy interface
SMD	shared memory driver
SSID	subsystem ID
SSM	secure services module
SSR	subsystem restart
TCB	task control block
UART	universal asynchronous receiver/transmitter
WCNSS	wireless connect subsystem

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION